

AI工具使用声明

本参赛队在竞赛过程中使用了 ChatGPT 等 AI 工具，主要用于建模方案讨论、程序编写与调试、结果整理及论文文字辅助，参赛队已人工修改建模方案并核验代码。详见支撑文件。

参考文献

- [1] KE G, MENG Q, FINLEY T, et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]//Advances in Neural Information Processing Systems 30. 2017: 3146-3154.
- [2] KOENKER R, HALLOCK K F. Quantile Regression[J]. Journal of Economic Perspectives, 2001, 15(4): 143-156. DOI: 10.1257/jep.15.4.143.
- [3] BELLMAN R. Dynamic Programming[M]. Princeton: Princeton University Press, 1957.
- [4] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119-142. DOI: 10.1007/s12532-017-0130-5.
- [5] The SciPy community. scipy.optimize.milp[EB/OL]. [2026-09-13]. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>.

附录

附录A 支撑文件清单与结果对应关系

本论文使用 Python 程序进行数据处理、优化求解、滚动回测、结果校验与可视化。支撑文件按下列目录组织，所有路径均相对于支撑材料压缩包根目录；逐文件清单列在本附录末尾。

1. AI工具使用详情.pdf、AI使用说明.docx 和 AI使用说明.txt：AI工具、使用环节、提示过程及人工修改与核验说明。
2. 源码/：第一问 MILP，第二问完整重跑版概率预测与反馈控制，第三问路线A预测更新与付费计划调整，第四问价格—净负荷联合误差模型，以及对应导出、核验和绘图程序。
3. 结果/：result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx；各问子目录保存汇总、逐日费用、主策略十分钟明细、调整账本、指定日期表及敏感性结果。
4. 图表/：由已保存结果生成的调度、累计费用、预报组合费用及指定日期库存图。
5. 环境/、核验/、准备复现.py、核验支撑包.py、README_支撑材料说明.txt、参考文献来源.txt和文件校验清单.json：依赖版本、运行记录、来源与哈希、Excel对照记录、复现与核验入口。

第二问采用 q2_fresh_20260912 版本，总费用为13,774,501.34元；第三问采用 mask_111，总费用为13,551,983.07元；第四问主策略为A42和A43_111，对应费用为14,505,015.50元和14,275,233.10元。第二至四问费用统计均为2—12月334天，1月暖启动另行记录。

LightGBM及条件分位预测的方法依据分别见文献[1]、[2]；反馈控制的动态规划递推见文献[3]；线性规划求解相关方法见文献[4]；第一问的混合整数线性规划调用接口见文献[5]。

附录B 主要程序与复现说明

第一问核心为 q1_solution.py 中 solve；第二问核心为 fresh_engine.py 中 plan_lp、markov、bellman、action；第三问核心为 q3_engine.py 中 revision_lp、bellman_batch、day_run；第四问核心为 model.py 中 scenario、plan、bellman、transact、simulate。完整实现及依赖在附录C按文件逐一给出。

第一问和第二至四问采用不同的边界设定：第一问求单日循环调度；后续问题以1月1日6000 kWh起点连续暖启动，不每日重置库存。物理量以10分钟时段为单位，功率转换为电量时乘以1/6 h；充放电效率分别为0.9。

从支撑包的准备复现.py建立独立运行目录，放入原始赛题、附件1—4及结果模板，再按说明先运行第二问以形成上游预测与签名，之后运行第一问、第三问和第四问。程序不依赖绝对个人目录；原程序中的可选本机Excel库回退不影响正常安装openpyxl环境。

大型可重建预测缓存、策略数组和检查点未重复打包。保留主策略完整数值明细和调整账本，其中.csv.xz为无损压缩，可用Python标准库lzma读取；完整源码可重新生成其余中间产物。历史自动检查记录随包提供，须与最终正文版本对应审阅。

附录C 完整源程序

代码/q1_solution.py

```
"""第一问：确定性 MILP，输入只读，结果为待审核版。"""
from pathlib import Path
import sys, json, hashlib, csv, platform
from datetime import time
import numpy as np
import scipy
from scipy.optimize import milp, Bounds, LinearConstraint
from scipy.sparse import lil_matrix

# 仅复用桌面运行时的纯 Python Excel 包；不修改全局环境。
try:
    import openpyxl
except ModuleNotFoundError:
    sys.path.append(str(Path.home()/'/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/Lib/site-packages'))
    import openpyxl
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT/'代码/results/q1'
FIG = ROOT/'代码/figures/q1'
N = 144
DT = 1/6
ETA = .9
CAP = 5000*DT
TOL = 1e-6

def sha(p):
    return hashlib.sha256(p.read_bytes()).hexdigest()

def clock(m):
    return f'{m//60:02d}:{m%60:02d}'

def load():
    wb = openpyxl.load_workbook(ROOT/'基础数据/附件1.xlsx', read_only=True, data_only=True)
    rows = list(wb.active.values)
    assert len(rows)==145 and len(rows[0])==4
    assert rows[0]==('时间', '电价', '小区负载', '光伏发电预测功率'), rows[0]
    for k, row in enumerate(rows[1:], 1):
        v=row[0]
        if isinstance(v, time): minute=v.hour*60+v.minute
        elif v=='0:00+1': minute=1440
        else:
            h, m=map(int, str(v).split(':')); minute=h*60+m
        assert minute==k*10, (k, v)
    a=np.array([r[1:] for r in rows[1:]], float)
    assert a.shape==(144, 3) and np.isfinite(a).all() and (a>=0).all()
    assert (a[:, 0]>0).all()
    wb.close()
    return a.T

def solve(p, L, V, relax=False):
    # x=[g, c, d, w, E(145), z]; 能量变量均为 kWh。
    dim=6*N+1
    G, C, D, W, E, Z=0, N, 2*N, 3*N, 4*N, 5*N+1
    cost=np.zeros(dim); cost[G:G+N]=p
    lo=np.zeros(dim); hi=np.full(dim, np.inf)
    hi[C:C+N]=CAP; hi[D:D+N]=CAP
    hi[W:W+N]=V*DT # 弃光不超过可用光伏
    lo[E:E+N+1]=1200; hi[E:E+N+1]=10800
    hi[Z:Z+N]=1
```

```

a=lil_matrix((4*N+1,dim)); lower=np.full(4*N+1,-np.inf); upper=np.zeros(4*N+1)
# 第一问为日循环：初末相等，其共同值是决策变量。
a[4*N,[E,E+N]]=1,-1]
lower[4*N]=upper[4*N]=0
for t in range(N):
    a[t,[G+t,C+t,D+t,W+t]]=1,-1,1,-1]
    lower[t]=upper[t]=(L[t]-V[t])*DT
    a[N+t,[E+t+1,E+t,C+t,D+t]]=1,-1,-ETA,1/ETA]
    lower[N+t]=upper[N+t]=0
    a[2*N+t,[C+t,Z+t]]=1,-CAP]
    a[3*N+t,[D+t,Z+t]]=1,CAP]; upper[3*N+t]=CAP
integer=np.zeros(dim); integer[Z]=0 if relax else 1
r=milp(cost,integrality=integer,bounds=Bounds(lo,hi),
        constraints=LinearConstraint(a.tocsc(),lower,upper),
        options={'mip_rel_gap':1e-9,'time_limit':120})
assert r.success,(r.status,r.message)
return r, [r.x[s:s+N].copy() for s in [G,C,D,W]],r.x[E:E+N+1].copy()

def main():
    OUT.mkdir(parents=True,exist_ok=True); FIG.mkdir(parents=True,exist_ok=True)
    p,L,V=load()
    r,(g,c,d,w),e=solve(p,L,V)
    lp,_=solve(p,L,V,relax=True)
    baseline=np.maximum((L-V)*DT,0)
    checks={
        'balance_max_abs_kWh':float(np.max(abs(g+V*DT+d-L*DT-c-w))),
        'state_max_abs_kWh':float(np.max(abs(np.diff(e)-ETA*c+d/ETA))),
        'cyclic_boundary_max_abs_kWh':float(abs(e[0]-e[-1])),
        'soc_bound_violation_kWh':float(max(0,1200-e.min(),e.max()-10800)),
        'power_bound_violation_kWh':float(max(0,c.max()-CAP,d.max()-CAP)),
        'simultaneous_charge_discharge_kWh':float(np.minimum(c,d).max()),
        'nonnegative_violation_kWh':float(max(0,-min(g.min(),c.min(),d.min(),w.min()))),
        'curtailment_bound_violation_kWh':float(max(0,np.max(w-V*DT))),
        'cost_recalculation_error_yuan':float(abs(sum(float(p[i])*float(g[i]) for i in range(N))-r.fun)),
        'MILP_minus_LP_yuan':float(r.fun-lp.fun),
        'daily_energy_error_kWh':float(abs(g.sum()+V.sum()*DT-L.sum()*DT-w.sum()-(1-ETA)*c.sum()-(1/ETA-1)*d.s
um
    )))
    }
    assert all(abs(v)<TOL for k,v in checks.items() if k!='MILP_minus_LP_yuan'),checks
    assert r.fun <= p@baseline+TOL and r.fun >= lp.fun-TOL
    labels=[f'{clock(t*10)}-{clock((t+1)*10)}' for t in range(N)]
    header=['时间段','原始区间结束标签','电价_元每kWh','负载_kW','光伏_kW','购电_kWh','充电_kWh','放电_kWh','
弃
    光_kWh','期初储电_kWh','期末储电_kWh','购电费_元']
    data=[[labels[t],clock((t+1)*10),p[t],L[t],V[t],g[t],c[t],d[t],w[t],e[t],e[t+1],p[t]*g[t]] for t in
range(N)
    ]
    with (OUT/'完整调度.csv').open('w',encoding='utf-8-sig,newline='') as f:
        writer=csv.writer(f);writer.writerow(header);writer.writerows(data)
    wb=openpyxl.Workbook(); ws=wb.active;ws.title='计划购电量'
    ws.append(['时间段','计划购电量 (kWh)'])
    for label,val in zip(labels,g):ws.append([label,float(val)])
    ws=wb.create_sheet('充放电量');ws.append(['时间段','充电量 (kWh, 电网侧)','放电量 (kWh, 电网侧)'])
    blocks=[]
    for k in range(6):
        row=[f'{clock(k*240)}-{clock((k+1)*240)}',float(c[k*24:(k+1)*24].sum()),float(d[k*24:(k+1)*24].sum())]
        ws.append(row);blocks.append(row)
    ws.append(['0:00储电量 (kWh)',float(e[0])]);ws.append(['24:00储电量 (kWh)',float(e[-1])])
    ws=wb.create_sheet('表1');ws.append(['时间段','购电量 (kWh)'])
    selected=[]
    for hour in [10,12,14,16,18,20]:
        row=[labels[hour*6],float(g[hour*6])];ws.append(row);selected.append(row)
    ws.append(['全天购电量 (kWh)',float(g.sum())];ws.append(['全天购电费 (元)',float(p@g)])
    ws=wb.create_sheet('十分钟调度明细');ws.append(header)
    for row in data: ws.append(row)
    ws=wb.create_sheet('口径与状态')
    for row in [
        ['状态','计算审核版;按用户要求恢复结束时标;未冻结'],

```

```

        ['时间映射', '标签为区间结束时刻; 00:10对应00:00-00:10; 末行对应23:50-24:00'],
        ['效率', '充电、放电效率各0.9; 充放电量均为电网侧; 往返效率0.81'],
        ['边界', '日初与日末相等, 共同值由优化决定; 全部145状态均在1200-10800 kWh'],
        ['弃光', '允许弃光, 不售电; 第一问不设紧急购电'],
        ['模板', '原始官方模板只读; 当前结果时段标签已按用户指定结束时段调整, 与官方标签有差异']
    ]]:ws.append(row)
    from openpyxl.styles import Font, PatternFill, Alignment
    from openpyxl.utils import get_column_letter
    for ws in wb:
        ws.freeze_panes='B2'; ws.auto_filter.ref=ws.dimensions
        for cell in ws[1]:cell.font=Font(bold=True, color='FFFFFF');cell.fill=PatternFill('solid', fgColor='2453
6B
    ')
    for column in ws.columns:
        width=min(65, max(18, max(len(str(cell.value or '')) for cell in column)+2))
        ws.column_dimensions[get_column_letter(column[0].column)].width=width
    for row in ws.iter_rows(min_row=2):
        for cell in row:
            if isinstance(cell.value, (int, float)):cell.number_format='0.000000'
            cell.alignment=Alignment(vertical='center', wrap_text=True)
    target=OUT/'result1_审核版.xlsx';wb.save(target)
    reread=openpyxl.load_workbook(target, data_only=True)
    exported=np.array([row[1] for row in list(reread['计划购电量'].values)[1:])]
    assert len(exported)==144 and np.max(abs(exported-g))<TOL
    assert abs(exported@p-r.fun)<TOL
    exported_states=np.array([row[9:11] for row in list(reread['十分钟调度明细'].values)[1:]], dtype=float)
    assert np.max(abs(exported_states[:,0]-e[:-1]))<TOL
    assert np.max(abs(exported_states[:,1]-e[1:]))<TOL
    assert abs(reread['充放电量']['B8'].value-e[0])<TOL
    assert abs(reread['充放电量']['B9'].value-e[-1])<TOL
    for ws in reread:
        assert not any(cell.data_type=='e' for row in ws for cell in row)
    reread.close()
    # 覆盖先前开始时标版本, 避免同目录留下两套冲突结果。
    official_path=OUT/'result1.xlsx'
    wb.save(official_path)
    checks['xlsx_roundtrip_max_abs_kWh']=float(np.max(abs(exported-g)))
    plt.rcParams.update({'font.family':'Microsoft YaHei', 'font.size':10, 'axes.unicode_minus':False})
    fig, axes=plt.subplots(3, 1, figsize=(7.2, 7.4), sharex=True, layout='constrained')
    x=np.arange(N)*DT
    axes[0].step(x, L-V, where='post', label='净负载', color='#3C5488')
    axes[0].step(x, g/DT, where='post', label='购电功率', color='#E64B35')
    axes[0].set_ylabel('功率 (kW)');axes[0].legend(ncol=2)
    axes[1].bar(x, c/DT, width=DT, align='edge', label='充电', color='#4DBBD5')
    axes[1].bar(x, -d/DT, width=DT, align='edge', label='放电 (负向)', color='#E64B35')
    axes[1].set_ylabel('功率 (kW)');axes[1].legend(ncol=2)
    axes[2].plot(np.arange(N+1)*DT, e, color='#3C5488', label='储电量')
    for y in [1200, 10800]:axes[2].axhline(y, ls='--', color='gray', lw=8)
    axes[2].set_ylabel('储电量 (kWh)');axes[2].set_xlabel('时刻 (h)');axes[2].legend()
    axes[2].set_xlim(0, 24);axes[2].set_xticks(range(0, 25, 4))
    for ax in axes:ax.grid(axis='y', alpha=.2)
    for ext in ['png', 'svg']:fig.savefig(FIG/f'q1_dispatch.{ext}', dpi=300)
    plt.close(fig)
    summary={'optimal_cost_yuan':float(r.fun), 'purchase_kWh':float(g.sum()), 'no_storage_cost_yuan':float(p@bas
el
    ine),
            'saving_yuan':float(p@baseline-r.fun), 'saving_percent':float((1-r.fun/(p@baseline))*100),
            'no_storage_purchase_kWh':float(baseline.sum()), 'charge_kWh':float(c.sum()), 'discharge_kWh':float(d.su
m(
    )),
            'curtailment_kWh':float(w.sum()), 'load_kWh':float(L.sum()*DT), 'pv_kWh':float(V.sum()*DT),
            'initial_soc_kWh':float(e[0]), 'terminal_soc_kWh':float(e[-1]),
            'soc_min_kWh':float(e.min()), 'soc_max_kWh':float(e.max()), 'LP_lower_bound_yuan':float(lp.fun),
            'solver_message':r.message, 'mip_gap':float(r.mip_gap), 'table1':selected, 'table2':blocks, 'validation':c
he
    cks)
    (OUT/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
    report=['# 第一问计算结果 (待用户审核)', '',
            f'购电费 **{r.fun:.6f} 元**, 全天购电量 **{g.sum():.6f} kWh**.',

```

```

        f' 不使用储能基线费用 {p@baseline:.6f} 元；节省 {p@baseline-r.fun:.6f} 元（
(summary["saving_percent"]:.4f
    }%）。',
    ' 基线允许弃光、无售电；所有结果均基于附件1预测值，不代表未知实际发电下的执行保证。',' ',
    ' ## 表1 指定时段购电量',' |时间段|购电量（kWh）|',' |---|---:|'|]
    report += [f'|{label}|{val:.6f}|' for label,val in selected]
    report += [' ',' ## 表2 分段充放电量（电网侧）',' |时间段|充电（kWh）|放电（kWh）|',' |---|---:|---:|'|]
    report += [f'|{label}|{cv:.6f}|{dv:.6f}|' for label,cv,dv in blocks]
    report += [' ',' f' 日初储电量 {e[0]:.6f} kWh，日末储电量 {e[-1]:.6f} kWh。两者仅约束相等，共同值由优化决定，
未
    固定为6000 kWh。',' ',
    f' LP 松弛下界 {lp.fun:.9f} 元；MILP 与下界差 {r.fun-lp.fun:.3g} 元。这验证当前目标的最优性，不保证调度
唯一
    一。',
    ' 低价充电、高价放电的解释受容量、功率及效率共同约束；不能仅按电价排序独立安排各时段。',' ',
    ' ', ' ## 数值验收',' ````json', json.dumps(checks, ensure_ascii=False, indent=2), '````', ' ',
    ' 所有物理残差按 1e-6 kWh 检查，费用复算按 1e-6 元检查；Excel 写入后重新读取校验。',' ',
    ' ', ' ## 尚未完成',
    ' - 按用户要求恢复结束标；result1.xlsx与审核版均为此口径，不能声称与官方模板标签一致。',' ',
    ' - 0:10映射00:00-00:10，0:00+1映射23:50-24:00；原始附件及官方模板未修改。',' ',
    ' - 效率仍各90%；初末仅相等，不固定6000；第二问沿用原结束标回溯。结果待审核、未冻结。']
    (OUT/' 第一问结果审核.md').write_text('\n'.join(report), encoding='utf-8')
    command=f' & "{sys.executable}" "代码/q1_solution.py"'
    manifest={'runtime':sys.executable,'python':platform.python_version(),'versions':{m.__name__:m.__version__
f
    or m in [np, scipy, openpyxl, matplotlib]},
        'command_from_project_root':command, 'random_seed':None, 'deterministic':True,
        'parameters':{'dt_hours':DT, 'eta_charge':ETA, 'eta_discharge':ETA, 'power_kW':5000, 'soc_bounds_kWh':[120
0,
    10800], 'boundary_condition':'E_0 = E_144; common value optimized', 'tolerance':TOL, 'mip_rel_gap':1e-9},
        'inputs':{'str(p.relative_to(ROOT)):sha(p) for p in [ROOT/' 基础数据/附件1.xlsx', ROOT/' 基础数据/C题.
pdf', P
    ath(__file__)]},
        'outputs':{'str(p.relative_to(ROOT)):sha(p) for p in [target, official_path, OUT/' summary.json', OUT/' 完整
调度
    度.csv', OUT/' 第一问结果审核.md', FIG/' q1_dispatch.png', FIG/' q1_dispatch.svg']},
        'status':' 真实计算完成；按用户指定结束标映射，存在官方模板标签差异；结果待审核', 'validation':checks}
    (OUT/' 复现清单.json').write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding='utf-8')
    print(json.dumps(summary, ensure_ascii=False, indent=2))

```

```

if __name__ == '__main__': main()

```

代码/实验/q2_fresh_20260912/fresh_engine.py

""" 本次完整重训所用独立模型函数；不读取历史预测缓存或旧运行结果。 """

```

from pathlib import Path
import sys
sys.path.insert(0, str(Path(__file__).resolve().parent/' .deps'))
from datetime import datetime, timedelta
import numpy as np
import scipy
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from numpy.lib.stride_tricks import sliding_window_view
import lightgbm as lgb
N=144;DT=1/6;ETA=.9;CAP=5000/6;EMIN=1200.;EMAX=10800.
QUANTILES=np.array([.2,.5,.65,.8,.9]);SEED=20260911
GRID=np.arange(EMIN,EMAX+1,60.);TOL=2e-5;SAMPLES=7;FIT_CALLS=0

```

```

def features(y, d):
    """All rows of day d use exclusively days < d, including training rows."""
    assert d>=1
    t=np.arange(N)/N;wd=(datetime(2025,1,1)+timedelta(days=d)).weekday()
    cols=[t,np.sin(2*np.pi*t),np.cos(2*np.pi*t),np.sin(4*np.pi*t),np.cos(4*np.pi*t)]
    cols += [np.full(N,x) for x in [wd,wd>=5,np.sin(2*np.pi*wd/7),np.cos(2*np.pi*wd/7)]]
    ids=[j for j in range(max(0,d-35),d) if (d-j)%7==0]
    if not ids: ids=list(range(max(0,d-7),d))
    for a in y:
        hist=a[max(0,d-7):d]

```

```

        cols.extend([a[d-1], a[max(0, d-7)], a[max(0, d-3):d].mean(axis=0), hist.mean(axis=0), hist.std(axis=0), a[ids]
s]
        .mean(axis=0)])
        cols.extend([np.full(N, a[d-1].mean()), np.full(N, hist.mean())])
    return np.array(cols, dtype=np.float64).T

def one_fit(X, Z, x, days, alpha):
    global FIT_CALLS
    FIT_CALLS += 1
    params=dict(objective='quantile', alpha=float(alpha), verbosity=-1, num_threads=2, seed=SEED,
        deterministic=True, force_col_wise=True, num_leaves=15, max_depth=4, learning_rate=.05,
        min_data_in_leaf=48, lambda_l2=5., feature_fraction=1., bagging_fraction=1.)
    w=np.repeat(2, **(-(days[-1]-days)/28), N)
    model=lgb.train(params, lgb.Dataset(X, label=Z/1000, weight=w), num_boost_round=120)
    return model.predict(x, num_threads=2)*1000

def forecast_day(y, d, Xall):
    ids=np.arange(max(7, d-56), d)
    X=Xall[ids].reshape(-1, Xall.shape[-1]); x=Xall[d]
    net=y[0]-y[1]
    direct=np.array([one_fit(X, net[ids].ravel(), x, ids, a) for a in QUANTILES])
    separate=np.array([np.maximum(one_fit(X, y[v, ids].ravel(), x, ids, .5), 0) for v in range(2)])
    return np.sort(direct, axis=0), separate

def plan_lp(p, net, E0, kappa):
    """Deterministic forecast LP only proposes g; recourse is never executed."""
    H=len(p); dim=5*H+1; g=0; c=H; q=2*H; w=3*H; e=4*H
    obj=np.zeros(dim); obj[:H]=p; obj[-1]=-kappa
    bounds=[(0, None)]*dim
    A=lil_matrix((2*H, dim)); rhs=np.r_[net, np.zeros(H)]
    for t in range(H):
        A[t, [g+t, c+t, q+t, w+t]]=[1, -1, 1, -1]
        A[H+t, [e+t+1, e+t, c+t, q+t]]=[1, -1, -ETA, 1/ETA]
        bounds[c+t]=bounds[q+t]=(0, CAP)
    for t in range(H+1): bounds[e+t]=(EMIN, EMAX)
    bounds[e]= (E0, E0)
    res=linprog(obj, A_eq=A.tocsr(), b_eq=rhs, bounds=bounds, method='highs')
    assert res.success, res.message
    assert abs(A@res.x-rhs).max()<TOL
    return np.maximum(res.x[:H], 0)

def grid_value(v, r, p, grid):
    """Exact minimization of the linearly interpolated continuation on a grid.
    Actual actions respect direction; surplus charging is weakly dominant.
    """
    if r<=0:
        end=np.minimum(grid+ETA*min(-r, CAP), EMAX)
        return np.interp(end, grid, v)
    distance=min(r/ETA, CAP/ETA)
    lo=np.maximum(EMIN, grid-distance)
    width=min(len(grid), int(np.floor(distance/(grid[1]-grid[0])+1e-10))+1)
    width=min(width, len(grid))
    x=v+5*p*ETA*grid
    # Window consists of current grid point and preceding width-1 points.
    pad=np.pad(x, (width-1, 0), constant_values=np.inf)
    knot_min=sliding_window_view(pad, width).min(axis=1)
    boundary=np.interp(lo, grid, v)+5*p*ETA*lo
    return 5*p*(r-ETA*grid)+np.minimum(knot_min, boundary)

def action(E, r, p, future, grid=GRID):
    if r<=0:
        c=max(0, min(-r, CAP, (EMAX-E)/ETA)); return c, 0., 0., max(0, -r-c), E+ETA*c
    lo=max(EMIN, E-min(r, CAP)/ETA)
    knots=grid[(grid>=lo-1e-9)&(grid<=E+1e-9)]
    knots=np.unique(np.r_[lo, E, knots]); knots=np.clip(knots, lo, E)
    costs=5*p*np.maximum(r-ETA*(E-knots), 0)+np.interp(knots, grid, future)
    nxt=float(knots[np.argmin(costs)])
    q=max(0, ETA*(E-nxt)); e=max(0, r-q)
    return 0., q, e, 0., nxt

```

```

def execute(actual, p, g, E0, m, V, grid=GRID, greedy=False):
    rows=[];E=float(E0)
    for t in range(N):
        net=float(actual[t])*DT;r=net-g[t]
        if greedy:
            if r<=0:
                c=max(0, min(-r, CAP, (EMAX-E)/ETA));q=e=0.;w=max(0, -r-c)
            else:
                q=max(0, min(r, CAP, (E-EMIN)*ETA));c=w=0.;e=max(0, r-q)
            nxt=E+ETA*c-q/ETA
        else:
            s=int(np. digitize((actual[t]-m['center'])[t])/m['scale'][t], [-.84,.84]))
            future=V[t+1, s] if t==N-1 else m['P'][t, s]*V[t+1]
            c, q, e, w, nxt=action(E, r, p[t], future, grid)
            rows.append([g[t], c, q, e, w, E, nxt]);E=nxt
    arr=np. asarray(rows);validate(arr, actual, p)
    return arr

def validate(arr, actual, p):
    a=np. asarray(arr);g, c, q, e, w, before, after=a.T
    errs={'balance':float(abs(g+e+q-c-w-actual*DT).max()),
        'state':float(abs(after-before-ETA*c+q/ETA).max()),
        'continuity':float(abs(before[1:]-after[:-1]).max()) if len(a)>1 else 0.,
        'simultaneous':float(np. minimum(c, q).max()),
        'soc':float(max(0, EMIN-before.min(), EMIN-after.min(), before.max()-EMAX, after.max()-EMAX)),
        'power':float(max(0, c.max()-CAP, q.max()-CAP)),
        'nonnegative':float(max(0, -a[:, :5].min())),
        'emergency_charging':float(np. minimum(c, e).max())}
    assert np. isfinite(a).all() and max(errs.values())<TOL, errs
    return errs

def metrics(a, p):
    daily=np. sum(a[:, :, 0]*p+5*a[:, :, 3]*p, axis=1)
    return {'normal_cost':float(np. sum(a[:, :, 0]*p)), 'emergency_cost':float(5*np. sum(a[:, :, 3]*p)),
        'total_cost':float(daily. sum()), 'planned_kWh':float(a[:, :, 0]. sum()), 'emergency_kWh':float(a[:, :, 3]. sum())
    },
    {'spill_kWh':float(a[:, :, 4]. sum()), 'charge_kWh':float(a[:, :, 1]. sum()), 'discharge_kWh':float(a[:, :, 2]. sum())
    },
    {'emergency_days':int(np. sum(np. any(a[:, :, 3]>TOL, axis=1))),
        'emergency_intervals':int(np. sum(a[:, :, 3]>TOL)), 'E_start':float(a[0, 0, 5]), 'E_end':float(a[-1, -1, -1]),
        'max_daily_cost':float(daily. max()), 'top_5pct_daily_mean':float(np. sort(daily)[-int(np. ceil(.05*len(daily)))].mean())}

def base_markov(y, pred, d):
    """Only preceding dates enter this probability model."""
    ids=np. arange(max(14, d-30), d);assert len(ids)>0
    real=(y[0]-y[1])[ids]
    scale_hist=np. maximum((pred[ids, 3]-pred[ids, 0])/1.683, 100.)
    z=(real-pred[ids, 1])/scale_hist
    states=np. digitize(z, [-.84,.84])
    scale=np. maximum((pred[d, 3]-pred[d, 0])/1.683, 100.)
    means=np. empty((N, 3));P=np. empty((N-1, 3, 3))
    count=np. ones((3, 3))
    np. add. at(count, (states[:, :-1].ravel(), states[:, 1:].ravel()), 1)
    global_P=count/count. sum(axis=1, keepdims=True)
    fallback=np. array([-1.4, 0., 1.4])
    for t in range(N):
        a=max(0, t-3);b=min(N, t+4);zs=z[:, a:b];ss=states[:, a:b]
        for s in range(3):
            vals=zs[ss==s]
            means[t, s]=(vals. sum()+2*fallback[s])/(len(vals)+2)
        if t<N-1:
            a=max(0, t-3);b=min(N-1, t+4)
            cnt=3*global_P. copy()
            np. add. at(cnt, (states[:, a:b].ravel(), states[:, a+1:b+1].ravel()), 1)

```

```

        P[t]=cnt/cnt.sum(axis=1,keepdims=True)
        initial=np.bincount(states[:,0],minlength=3)+np.array([.6,1.8,.6])
        initial=initial/initial.sum()
        net=(pred[d,1,:None]+scale[:,None]*means)*DT
        assert np.isfinite(net).all() and np.allclose(P.sum(axis=2),1)
        return dict(net=net,P=P,initial=initial,scale=scale,center=pred[d,1])

def atomize(values,limit=SAMPLES):
    """Retain both empirical extremes; compress interior into weighted bins.
    Mean preserved, unlike arbitrary normal draws; remaining compression explicit.
    """
    a=np.sort(np.asarray(values,float))
    if len(a)<=limit:return a,np.full(len(a),1/len(a))
    groups=[a[1:],*np.array_split(a[1:-1],limit-2),a[-1:]]
    return np.array([x.mean() for x in groups]),np.array([len(x)/len(a) for x in groups])

def markov(y,pred,d,limit=SAMPLES):
    m=base_markov(y,pred,d)
    ids=np.arange(max(14,d-30),d)
    z=((y[0]-y[1])[ids]-pred[ids,1])/np.maximum((pred[ids,3]-pred[ids,0])/1.683,100.)
    states=np.digitize(z,[-.84,.84])
    samples=np.zeros((N,3,limit));weights=np.zeros_like(samples)
    for t in range(N):
        sl=slice(max(0,t-3),min(N,t+4));zs=z[:,sl];ss=states[:,sl]
        for s in range(3):
            vals=zs[ss==s]
            if len(vals)==0:vals=np.array([[-1.4,0,1.4][s]])
            atoms,w=atomize(vals,limit)
            samples[t,s,:len(w)]=(pred[d,1,t]+m['scale'][t]*atoms)*DT
            weights[t,s,:len(w)]=w
    m['samples']=samples;m['weights']=weights
    # Expected candidate now uses the same empirical conditional means as Bellman.
    m['net']=(samples*weights).sum(axis=2)
    assert np.allclose(weights.sum(axis=2),1)
    return m

def bellman(p,g,m,kappa,grid=GRID):
    H=len(p);V=np.empty((H+1,3,len(grid)));V[H]=-kappa*grid
    for t in range(H-1,-1,-1):
        for s in range(3):
            future=V[t+1,s] if t==H-1 else m['P'][t,s]@V[t+1]
            if 'samples' not in m:
                V[t,s]=grid_value(future,m['net'][t,s]-g[t],p[t],grid)
            else:
                V[t,s]=0.
                for value,w in zip(m['samples'][t,s],m['weights'][t,s]):
                    if w>0:V[t,s]+=w*grid_value(future,value-g[t],p[t],grid)
    assert np.isfinite(V).all() and np.diff(V,axis=2).max()<TOL
    return V

```

代码/实验/q2_fresh_20260912/run_fresh.py

"""原始附件起步：每次完整重训，不加载任何预测或调度缓存。"""

```

from pathlib import Path
from datetime import datetime,timedelta
import argparse, csv, hashlib, json, sys, time, platform
import numpy as np
import openpyxl
import fresh_engine as m

```

```

BASE=Path(__file__).resolve().parent
ROOT=BASE.parents[2]

```

```

def dump(path,value):path.write_text(json.dumps(value,ensure_ascii=False,indent=2),encoding='utf-8')
def sha(path):return hashlib.sha256(path.read_bytes()).hexdigest()
def log(*x):print(datetime.now().isoformat(timespec='seconds'),*x,flush=True)
def date(d):return (datetime(2025,1,1)+timedelta(days=int(d))).strftime('%Y-%m-%d')
def csvout(path,heads,rows):

```

```

with path.open('w', encoding='utf-8-sig', newline='') as f:
    w=csv.writer(f);w.writerow(headers);w.writerows(rows)

def load(data):
    wb=openpyxl.load_workbook(data/'附件2.xlsx', read_only=True, data_only=True)
    arrays=[];headers=[]
    for name in ['小区负载', '光伏发电实际功率']:
        rows=list(wb[name].values)
        assert len(rows)==366 and all(len(row)==145 for row in rows)
        assert [row[0] for row in rows[1:]]==[datetime(2025,1,1)+timedelta(days=d) for d in range(365)]
        arrays.append(np.asarray([row[1:] for row in rows[1:]],float));headers.append(rows[0][1:])
    wb.close()
    wb=openpyxl.load_workbook(data/'附件1.xlsx', read_only=True, data_only=True)
    rows=list(wb.active.values);wb.close()
    def minute(x):
        if hasattr(x,'hour'):return x.hour*60+x.minute
        if x=='0:00+1':return 1440
        h,v=map(int, str(x).split(':'));return h*60+v
    assert len(rows)==145
    assert [minute(x) for x in headers[0]]==[minute(x) for x in headers[1]]==[minute(r[0]) for r in
rows[1:]]==1
    ist(range(10, 1441, 10))
    y=np.asarray(arrays);p=np.asarray([r[1] for r in rows[1:]],float)
    assert y.shape==(2, 365, 144) and np.isfinite(y).all() and y.min()>=0
    assert p.shape==(144,) and np.isfinite(p).all() and p.min()>0
    return y,p

def forecast_fresh(y,out):
    X=np.zeros((365,144,25))
    for d in range(1,365):X[d]=m.features(y,d)
    net=y[0]-y[1];direct=np.zeros((365,5,144));separate=np.zeros((365,2,144))
    base=np.zeros_like(direct);pred=np.zeros_like(direct);family='direct';selection={};trace=[]
    for d in range(1,365):
        if d<14:
            direct[d]=net[d-1][None,:];separate[d]=y[:,d-1]
        else:
            # 必须调用真实拟合, 没有exists/cache/read预测文件分支。
            direct[d], separate[d]=m.forecast_day(y,d,X)
            trace.append([date(d), date(max(7,d-56)), date(d-1), 7, m.FIT_CALLS])
        if d==21:
            a=float(abs(direct[14:21,1]-net[14:21]).mean())
            b=float(abs(separate[14:21,0]-separate[14:21,1]-net[14:21]).mean())
            family='direct' if a<=b else 'separate_center'
            selection=dict(validation='2025-01-15..2025-01-21', frozen_at='2025-01-22 00:00',
                            direct_MAE_kW=a, separate_MAE_kW=b, family=family)
            log('forecast family', selection)
        base[d]=direct[d]
        if family=='separate_center':base[d]+=separate[d,0]-separate[d,1]-direct[d,1]
        pred[d]=base[d]
        ids=np.arange(max(14,d-30),d)
        if len(ids)>=4:
            for block in range(4):
                sl=slice(block*36, (block+1)*36)
                for k,q in enumerate(m.QUANTILES):
                    pred[d,k,sl]+=np.quantile(net[ids,sl]-base[ids,k,sl],float(q))
            pred[d]=np.sort(pred[d],axis=0)
        if d%15==0 or d==364:
            log('fresh forecast', date(d), 'fitted_models', m.FIT_CALLS)
    assert m.FIT_CALLS==351*7
    np.savez_compressed(out/'fresh_forecasts.npz', pred=pred, base=base, direct=direct, separate=separate)
    csvout(out/'training_trace.csv', ['forecast_date', 'training_start', 'training_end', 'models_fit', 'cumulative_
mo
dels'], trace)
    dump(out/'forecast_selection.json', selection)
    return pred, selection

def warmup(y,p):
    net=y[0]-y[1];E=6000.;days=[]
    for d in range(31):

```

```

        g=np.zeros(144) if d==0 else np.maximum(net[d-1]/6,0)
        a=m.execute(net[d],p,g,E,None,None,greedy=True);days.append(a);E=float(a[-1,-1])
    return np.asarray(days)

def select_kappa(y,p,pred,warm,out):
    ref=float(.9*p.mean());grid=ref*np.array([0,.25,.5,.75,1,1.25]);rows=[]
    for k in grid:
        E=float(warm[20,-1,-1]);start=E;days=[]
        for d in range(21,31):
            model=m.markov(y,pred,d)
            g=m.plan_lp(p,pred[d,3]/6,E,0.)
            V=m.bellman(p,g,model,k)
            a=m.execute((y[0]-y[1])[d],p,g,E,model,V);days.append(a);E=float(a[-1,-1])
            a=np.asarray(days);cost=float(np.sum(a[:,0]*p+5*a[:,3]*p))
            rows.append(dict(kappa=float(k),bill=cost,inventory_adjusted=cost+ref*(start-E),start=start,end=E))
    best=min(rows,key=lambda x:(x['inventory_adjusted'],x['kappa']))
    record=dict(candidates=rows,selected=best,reference_inventory_price=ref,
                validation='2025-01-22..2025-01-31',frozen_at='2025-02-01 00:00',
                model='fixed q80, day-ahead LP terminal coefficient zero, own continuous SOC')
    dump(out/'controller_selection.json',record);log('new controller selection',record)
    return best['kappa'],record

def backtest(y,p,pred,warm,kappa):
    E0=float(warm[-1,-1,-1]);states={name:E0 for name in ['main','same_plan_greedy','independent_greedy']}
    runs={name:[] for name in states};net=y[0]-y[1]
    for d in range(31,365):
        model=m.markov(y,pred,d);k=0. if d==364 else kappa
        g=m.plan_lp(p,pred[d,3]/6,states['main'],0.)
        V=m.bellman(p,g,model,k)
        main=m.execute(net[d],p,g,states['main'],model,V)
        pair=m.execute(net[d],p,g,states['same_plan_greedy'],None,None,greedy=True)
        gi=m.plan_lp(p,pred[d,3]/6,states['independent_greedy'],0.)
        independent=m.execute(net[d],p,gi,states['independent_greedy'],None,None,greedy=True)
        for name,a in zip(states,[main,pair,independent]):runs[name].append(a);states[name]=float(a[-1,-1])
        if (d-31)%50==0 or d==364:log('fresh backtest',date(d))
    return {name:np.asarray(a) for name,a in runs.items()}

def verify(y,p,pred,runs,kappa):
    checks={name:m.validate(a.reshape(-1,7),(y[0]-y[1])[31:].ravel(),np.tile(p,334)) for name,a in
    runs.items()}
    assert np.array_equal(runs['main'][:,0],runs['same_plan_greedy'][:,0])
    changed=y.copy();changed[:,31:]+=10000
    assert np.array_equal(m.features(y,31),m.features(changed,31))
    model=m.markov(y,pred,31);other=m.markov(changed,pred,31)
    for key in model:np.testing.assert_array_equal(model[key],other[key])
    g=runs['main'][0,0];V=m.bellman(p,g,model,kappa);actual=(y[0]-y[1])[31]
    modified=actual.copy();modified[72:]+=20000
    a=m.execute(actual,p,g,runs['main'][0,0,5],model,V)
    b=m.execute(modified,p,g,runs['main'][0,0,5],model,V)
    np.testing.assert_array_equal(a[:72],b[:72])
    # 类内样本均值不能替代真实缺口函数。
    toy=dict(samples=np.tile([0.,100.],[1,3,1]),weights=np.full((1,3,2),.5),P=np.empty((0,3,3)))
    values=m.bellman(np.array([1.]),np.array([50.]),toy,0.)
    assert abs(values[0,0,0]-125)<1e-8
    return dict(physical=checks,dayahead_causal_features_and_probabilities=True,
                future_observation_perturbation=True,within_class_shortfall_counterexample=125.)

def export(y,p,pred,runs,out,summary,kappa):
    heads=['date','slot','g_kwh','charge_kwh','discharge_kwh','emergency_kwh','waste_kwh','start_kwh','end_kwh',
    ,
    'price','net_kwh']
    all_daily=[];dates=[date(d) for d in range(31,365)]
    for name,a in runs.items():
        np.save(out/f'{name}.npy',a)
        rows=((dates[i],t+1,*a[i,t],float(p[t]),float((y[0,i+31,t]-y[1,i+31,t])/6)) for i in range(334) for t
    in
    range(144))
        path=out/f'{name}_intervals.csv';csvout(path,heads,rows)
        with path.open(encoding='utf-8-sig',newline='') as f:read=list(csv.DictReader(f))

```

```

        nc=sum(float(r['g_kwh'])*float(r['price'])) for r in read)
        ec=sum(5*float(r['emergency_kwh'])*float(r['price'])) for r in read)
        assert len(read)==48096 and abs(nc-summary[name]['normal_cost'])<1e-5 and
abs(ec-summary[name]['emergenc
y_cost'])<1e-5
        for i,ds in enumerate(dates):
            normal=float(a[i,:,0]@p);em=float(5*a[i,:,3]@p)
            all_daily.append([name,ds,normal,em,normal+em,float(a[i,:,3].sum()),float(a[i,0,5]),float(a[i,-1,-
1])])
        csvout(out/'daily_summary.csv',['policy','date','normal_cost','emergency_cost','total_cost','emergency_kwh'
,
        'start_kwh','end_kwh'],all_daily)
        wb=openpyxl.Workbook(write_only=True);main=runs['main']
        ws=wb.create_sheet('计划购电量');ws.append(['日期']+f'{t*10//60:02d}:{t*10%60:02d}-{(t+1)*10//60:02d}:{(t
+1
)*10%60:02d}' for t in range(144)]+'全天购电量','全天购电费'])
        for i,ds in enumerate(dates):ws.append([ds,*main[i,:,0].tolist(),float(main[i,:,0].sum()),float(main[i,:,0]
@
p)])
        ws=wb.create_sheet('十分钟明细');ws.append(heads)
        for i,ds in enumerate(dates):
            for t in range(144):ws.append([ds,t+1,*main[i,t].tolist(),float(p[t]),float((y[0,i+31,t]-y[1,i+31,t])/
6)
            ])
            ws=wb.create_sheet('四个指定日');ws.append(heads)
            for ds in ['2025-03-20','2025-06-21','2025-09-23','2025-12-21']:
                i=dates.index(ds)
                for t in range(144):ws.append([ds,t+1,*main[i,t].tolist(),float(p[t]),float((y[0,i+31,t]-y[1,i+31,t])/
6)
                ])
            ws=wb.create_sheet('充放电量');ws.append(['日期','四小时时段','充电量kWh','放电量kWh','日初库存kWh','日末
库
存kWh'])
            for i,ds in enumerate(dates):
                for block in range(6):ws.append([ds,f'{block*4:02d}:00-{(block+1)*4:02d}:00',float(main[i,block*24:(b1
oc
k+1)*24,1].sum()),float(main[i,block*24:(block+1)*24,2].sum()),float(main[i,0,5]),float(main[i,-1,-1])])
            ws=wb.create_sheet('紧急购电量');ws.append(['日期','时段序号','应急电量kWh','应急费用元'])
            for i,ds in enumerate(dates):
                for t in range(144):
                    if main[i,t,3]>1e-8:ws.append([ds,t+1,float(main[i,t,3]),float(5*p[t]*main[i,t,3])])
            ws=wb.create_sheet('说明');ws.append(['项目','内容'])
            for key,value in dict(forecast='原始附件完整重训2457个LightGBM模型;未读取旧预测'
,quantile=.8,lp_terminal=0,
            dp_terminal=kappa,dec31_terminal=0,observation='当前时段功率即可测'
,period='2025-02-01..2025-12-31',status
            ='独立审核结果,区间终点按00:10至24:00解释').items():ws.append([key,str(value)])
            wb.save(out/'result2.xlsx')
            wb=openpyxl.load_workbook(out/'result2.xlsx',read_only=True,data_only=True)
            rr=list(wb['计划购电量'].values);assert len(rr)==335
            assert abs(sum(float(row[-1]) for row in rr[1:])-summary['main']['normal_cost'])<1e-5
            rr=list(wb['十分钟明细'].values);assert len(rr)==48097
            assert abs(sum(5*float(row[5])*float(row[9]) for row in rr[1:])-summary['main']['emergency_cost'])<1e-5
            wb.close()

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--data-dir',type=Path)
    parser.add_argument('--output',type=Path,default=BASE/'results')
    args=parser.parse_args();out=args.output.resolve()
    if out.exists() and any(out.iterdir()):raise RuntimeError('输出目录非空;请选择新的--output,禁止复用缓存
或
覆盖旧结果')
    out.mkdir(parents=True,exist_ok=True)
    data=args.data_dir or next(p for p in [ROOT/'基础数据/C题/附件',ROOT/'基础数据']) if (p/'附件
2.xlsx').exists(
    ))
    started=time.perf_counter();y,p=load(data)

```

```

input_hashes={file:sha(data/file) for file in ['附件1.xlsx','附件2.xlsx']}
log('raw inputs read; no prior prediction or results loaded',input_hashes)
pred,forecast_selection=forecast_fresh(y,out)
warm=warmup(y,p);np.save(out/'january_warmup.npy',warm)
kappa,controller_selection=select_kappa(y,p,pred,warm,out)
runs=backtest(y,p,pred,warm,kappa)
checks=verify(y,p,pred,runs,kappa)
summary={name:m.metrics(a,p) for name,a in runs.items()}
err=(y[0]-y[1])[31:]-pred[31:,1]
forecast_metrics=dict(MAE_kW=float(abs(err).mean()),RMSE_kW=float(np.sqrt((err**2).mean()))),q80_coverage=f
lo
at(np.mean((y[0]-y[1])[31:]<=pred[31:,3]))
export(y,p,pred,runs,out,summary,kappa)
manifest=dict(created=datetime.now().isoformat(),status='completed_checked_pending_review',raw_inputs=inp
t_
hashes,
old_prediction_reads=0,old_result_reads=0,models_trained=m.FIT_CALLS,
elapsed_seconds=time.perf_counter()-started,summary=summary,checks=checks,
forecast_metrics=forecast_metrics,forecast_selection=forecast_selection,controller_selection
=c
ontroller_selection,
python=sys.version,numpy=np.__version__,scipy=m.scipy.__version__,lightgbm=m.lgb.__version__,
o
penpyxl=openpyxl.__version__,
source_hashes={f:sha(BASE/f) for f in ['fresh_engine.py','run_fresh.py']},seed=m.SEED,
command=[sys.executable,str(Path(__file__).relative_to(ROOT))],
output_hashes={str(f.relative_to(out)):sha(f) for f in out.iterdir() if f.is_file()})
dump(out/'manifest.json',manifest)
dump(out/'summary.json',dict(summary=summary,forecast_metrics=forecast_metrics,kappa=kappa,checks=checks))
log('COMPLETE',json.dumps(summary,ensure_ascii=False),'seconds',manifest['elapsed_seconds'])

if __name__=='__main__':main()

```

代码/实验/q2_fresh_20260912/verify_delivery.py

"""本次输出的补充核验：未来扰动仅改变负载，避免净负荷相减抵消。"""

```

from pathlib import Path
import json, csv, hashlib
import numpy as np
import fresh_engine as m
from run_fresh import load

BASE=Path(__file__).resolve().parent;ROOT=BASE.parents[2];OUT=BASE/'results'
data=next(p for p in [ROOT/'基础数据/C题/附件',ROOT/'基础数据'] if (p/'附件2.xlsx').exists())
y,p=load(data)
manifest=json.loads((OUT/'manifest.json').read_text(encoding='utf-8'))
pred=np.load(OUT/'fresh_forecasts.npz')['pred']
checks=[]
for d in [31,90,180,270]:
    altered=y.copy();altered[0,d:]+=30000
    assert np.array_equal(m.features(y,d),m.features(altered,d))
    original=m.markov(y,pred,d);mutated=m.markov(altered,pred,d)
    for key in original:np.testing.assert_array_equal(original[key],mutated[key])
    checks.append({'day_index':d,'future_load_changed_without_pv_cancellation':True,'past_only_features_and_pr
ob
abilities':True})
with (OUT/'main_intervals.csv').open(encoding='utf-8-sig',newline='') as f:rows=list(csv.DictReader(f))
assert len(rows)==48096
g=np.array([float(r['g_kwh']) for r in rows]);e=np.array([float(r['emergency_kwh']) for r in
rows]);price=np.arr
ay([float(r['price']) for r in rows])
bill=float(np.sum(price*g+5*price*e))
assert abs(bill-manifest['summary']['main']['total_cost'])<1e-5
with (OUT/'training_trace.csv').open(encoding='utf-8-sig',newline='') as f:trace=list(csv.DictReader(f))
assert len(trace)==351 and sum(int(x['models_fit']) for x in trace)==2457
assert all(x['training_end']<x['forecast_date'] for x in trace)
for name,digest in manifest['output_hashes'].items():
    assert hashlib.sha256((OUT/name).read_bytes()).hexdigest()==digest
result={'status':'passed','causal_checks':checks,'csv_recomputed_bill':bill,'training_days':351,'models_fit':2
45

```

```

7,
    'training_cutoff_before_forecast_date':True,'original_run_output_hashes_match':True}
(OUT/'independent_check.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps(result,ensure_ascii=False))

```

代码/实验/q2_fresh_20260912/write_report.py

"""仅根据本次完整运行的结果生成解法、图及复现说明。"""

```

from pathlib import Path
import json, csv, hashlib
import numpy as np

BASE=Path(__file__).resolve().parent
ROOT=BASE.parents[2]
OUT=BASE/'results'

```

TEXT=r'''# 第二问：80分位预测与动态规划——从原始附件完整重跑

本次从原始附件1、附件2开始，重新构造特征、训练预测模型、校准分位数、选择控制参数、预热电池及进行全年顺序回测。程

序没有读取任何此前预测缓存、调度轨迹或旧汇总结果。实际完成 *****FITS****个LightGBM模型拟合*******。

本次主方案总费用为 *****TOTAL****元（*****WAN****万元）。正式评价区间为2025年2月1日至12月31日，共334天、48096个十分

时段。该数值由本次新运行生成的逐时明细独立复算。

模型的完整路线为：*****历史分位数预测 → 固定80分位日前LP → 日内随机动态规划 → 实际SOC跨日接续*****。以下分成模型假

设、日前阶段、日内阶段与结果四部分，不引用参考文献。

一、模型假设与变量

1. 每段十分分钟内使用平均功率描述负载及光伏。输入列按区间终点解释：00:10对应00:00—00:10，24:00对应23:50—24:00。
2. 每日零点只利用此前信息确定全天普通购电计划，日内不修改。控制可根据已经得到的观测调整电池，不读取未来实测数据。
3. 沿用当前十分钟功率分段恒定且即时可测的执行假设。这是计算口径，不认定题面已经明确规定该测量机制；如果命令须早于当期观测，需要另行评估延迟信息模型。
4. 充电、放电效率分别为0.9，忽略题目未给参数的自放电与循环衰减。双向各0.9对应往返效率0.81，不能混用。
5. 在题目未给出额外电网容量上限时，允许应急购电补齐剩余缺口。富余可弃用，不计售电收益；实际执行不以应急购电给池充电。
6. 预测误差以三状态一阶Markov转移及每类最多7个加权样本近似。该离散模型不代表真实误差严格具有Markov性，也不完整保存全部尾部风险。
7. 1月1日零点仅初始化一次6000kWh，之后库存跨日连续。第二问不强加每日首末电量相等，也不每日重置为6000kWh。

以 d 为日期， $t=1, \dots, 144$ 为时段， $E_{\{d, t\}}$ 表示时段结束时的储电量， $E_{\{d, 0\}}$ 表示当日零点电量。一天有144个动作区间、145个库存状态。

```

$$
\Delta t = \frac{16}{\mathrm{h}}, \quad
X_{\{d, t\}} = L_{\{d, t\}} - V_{\{d, t\}}, \quad N_{\{d, t\}} = X_{\{d, t\}} \Delta t. \quad \tag{1}
$$

```

L, V, X 单位为kW； N 是净需求电量，单位kWh，允许为负。以下 g, e, c, b, w, E 分别表示普通购电、应急购电、充电、放电、弃用和储电量，均为kWh；用 b 表示放电以避免与日期 d 混淆。 p_t 单位为元/kWh，第二问每天使用附件1同一组时段价格。

二、第一阶段：80分位预测与日前购电计划

*****第一步：构造只依赖过去的预测特征。***** 每天形成25个特征：时段位置及日周期正余弦、星期、周末标记、星期周期正余弦；负载与光伏各自的昨日同刻、上周同刻、近3日均值、近7日均值及标准差、近35日同星期均值、昨日整体均值、近7日整体均值。历史不足时仅使用已经获得的样本。

每个训练日的特征也只能由该训练日以前的数据构造，不能拿训练日完整观测计算其输入特征。每天选此前最多56日训练数据，
训练起点不早于1月8日；从1月15日起正式拟合。1月2—14日用昨日同刻值启动历史预测。

****第二步：训练分位数回归。****同时预测净负荷的\$0.20, 0.50, 0.65, 0.80, 0.90\$五个分位，并分别拟合负载及光伏的中位数。
每天7个模型，共351个拟合日，合计2457次拟合。普通购电仅使用80分位，其他分位用于预测中心、尺度与误差描述，不在
正式期反选购电分位。

分位数损失写为

$$\rho_q(u) = \begin{cases} qu, & u \geq 0 \\ -(q-1)u, & u < 0 \end{cases} \quad \text{tag}\{2\}$$

历史日\$j\$的样本权重为

$$\omega_j = 2^{-(d-1-j)/28}, \quad \min_f \sum_{j \in \text{mathcal{T}_d}} \omega_j \sum_t \rho_q(X_{\{j, t\}} - f(\mathbf{x}_{\{j, t\}})). \quad \text{tag}\{3\}$$

式（3）是数据拟合项。实现用LightGBM，目标标签除以1000训练、预测后乘回1000；树深4、最多15个叶子、学习率0.05、120

轮、最小叶样本48、叶值L2正则参数5。随机种子20260911。每日重新拟合，不加载历史模型文件。

****第三步：选择预测中心并进行误差校准。****直接净负荷分位预测记为\$D_{\{d, t\}}^{\{q\}}\$。同时计算分别预测的负载与光伏中位数差：

$$M_{\{d, t\}} = \max(\widehat{L}_{\{d, t\}}^{\{0.5\}}, 0) - \max(\widehat{V}_{\{d, t\}}^{\{0.5\}}, 0). \quad \text{tag}\{4\}$$

用1月15—21日MAE比较直接预测与分开预测，1月22日冻结预测中心方式。本次得到：@@FAMILY@@。直接预测MAE为@@DIRECT_MAE@@kW，分开预测MAE为@@SEPARATE_MAE@@kW。

当分开预测胜出时，在保留直接预测分位间距的同时平移中心：

$$B_{\{d, t\}}^{\{q\}} = D_{\{d, t\}}^{\{q\}} + M_{\{d, t\}} - D_{\{d, t\}}^{\{0.5\}}. \quad \text{tag}\{5\}$$

若直接预测胜出，则\$B=D\$。中心方式冻结前的历史预测保持当时的版本，不回写为事后选中的方式。

将一天分为4个六小时块\$\text{mathcal{B}}_{\text{ell}}\$，取此前最多30日且不早于1月15日的预测误差，按分位数校准：

$$a_{\{d, \text{ell}\}}^{\{q\}} = Q_q\{X_{\{j, t\}} - B_{\{j, t\}}^{\{q\}} : j \in \text{mathcal{H}}_d, t \in \text{mathcal{B}}_{\text{ell}}\}, \\ \quad \widehat{X}_{\{d, t\}}^{\{q\}} = B_{\{d, t\}}^{\{q\}} + a_{\{d, \text{ell}\}}^{\{q\}}. \quad \text{tag}\{6\}$$

不足4个历史日时不加校准项。原始直接分位预测和最终校准分位预测分别逐时排序，消除分位交叉。这里\$\widehat{X}^{\{q\}}\$表示排序后的输出。

****第四步：说明80分位的经济依据。****暂不考虑储能，若计划购电为\$g\$、净需求为随机电量\$N\$，过量无售电收入，则单时段期望费用为

$$J(g) = pg + 5p \mathbb{E}[(N-g)^+], \quad g \geq 0. \quad \text{tag}\{7\}$$

连续分布下，内点最优条件为

$$J'(g) = 0$$

$$J'(g) = p - 5p[1 - F_N(g)] = 0 \quad \Longleftrightarrow \quad F_N(g^*) = 0.8. \quad \text{tag}{8}$$

这给出了偏保守购电的依据，但只适用于无储能的简化特例。有储能时不能机械令 $g = Q_{0.8}(N)$ 。本次按用户指定将

$$\widetilde{N}_{d,t} = \widehat{X}_{d,t}^{\{(0.8)\}} \Delta t \quad \text{tag}{9}$$

作为确定性LP输入，再联合安排购电与储能。这里固定80分位，不把它称为已由全年搜索证明的最优分位。

****第五步：建立日前LP。**** 给定该策略真实日初库存 $E_{d,0}^{\{\text{rm act}\}}$ ，求解

$$\min \sum_{t=1}^{144} p_{tg}_{d,t}, \quad \text{tag}{10}$$

满足

$$g_{d,t} + \bar{b}_{d,t} = \widetilde{N}_{d,t} + \bar{c}_{d,t} + \bar{w}_{d,t}, \quad \text{tag}{11}$$

$$\begin{aligned} \bar{E}_{d,t} &= \bar{E}_{d,t-1} + 0.9 \bar{c}_{d,t} - \frac{\bar{b}_{d,t}}{0.9}, \\ \text{quad } \bar{E}_{d,0} &= E_{d,0}^{\{\text{rm act}\}}, \quad \text{tag}{12} \end{aligned}$$

$$\begin{aligned} 1200 \leq \bar{E}_{d,t} &\leq 10800, \\ 0 \leq \bar{c}_{d,t}, \bar{b}_{d,t} &\leq \frac{5000}{6}, \\ g_{d,t}, \bar{w}_{d,t} &\geq 0. \quad \text{tag}{13} \end{aligned}$$

横线变量只是生成计划所用的辅助轨迹，不直接作为真实执行动作。该LP不含应急购电变量，也不含终端库存奖励，不强制日末恢复日初电量。

原连续LP没有互斥二元变量，因此不能写成MILP。若某辅助解同时充放电，可令 $\delta = \min(\bar{c}, \bar{b}/0.9^2)$ ，同时减少充电 δ 、放电 $0.9^2 \delta$ ，将弃用量增加 $(1 - 0.9^2) \delta$ 。这样库存递推、购电量和费用均不变，并消除重叠。实际控制使用互斥分支。

利用HiGHS求解后提取 $\mathbf{g}_d$ ，全天固定。****本次每一天LP的初始库存来自该策略自身前一日实际日末，不借用其他策略的库存。****

三、第二阶段：随机动态规划反馈控制

****第一步：估计误差状态及转移。**** 对预测误差进行标准化：

$$\begin{aligned} \sigma_{d,t} &= \max \left\{ \frac{\widehat{X}_{d,t}^{\{(0.8)\}} - \widehat{X}_{d,t}^{\{(0.2)\}}}{1.683}, 100 \right\}, \\ \text{quad } z_{j,t} &= \frac{X_{j,t} - \widehat{X}_{j,t}^{\{(0.5)\}}}{\sigma_{j,t}}. \quad \text{tag}{14} \end{aligned}$$

σ 单位kW，100kW是尺度下限。尺度换算不等于假设误差正态。按原边界约定划分

$$S_{j,t} = \begin{cases} 1, & z_{j,t} < -0.84, \\ 2, & -0.84 \leq z_{j,t} < 0.84, \\ 3, & z_{j,t} \geq 0.84. \end{cases} \quad \text{tag}{15}$$

使用此前最多30日的相邻状态计数。全局计数 C_{ij} 加1平滑，再以当前时段前后各3个时段的历史转移计数 $C_{t,ij}$ 局部修正：

$$P^{\{\text{rm all}\}}_{ij} = \frac{C_{ij} + 1}{\sum_h (C_{ih} + 1)}, \quad \text{quad}$$

$$P_{\{t, ij\}} = \frac{C_{\{t, ij\}} + 3P^{\{\text{rm all}\}}_{\{ij\}}}{\sum_h (C_{\{t, ih\}} + 3P^{\{\text{rm all}\}}_{\{ih\}})}. \quad \text{\tag{16}}$$

边界处截断邻域。 i 表示当前类别， j 表示下一时段类别。所有计数只来自当前历史。

****第二步：保留类内加权样本。**** 取同一历史窗口中 t 前后各3个时段、属于同一类别的标准化误差。若样本数不超过7，全部

等权保留；超过7则单独保留最小和最大值，把中间有序样本分5组，以组均值及组占比表示：

$$\begin{aligned} \zeta_{\{t, i, k\}} &= \frac{1}{|\mathcal{G}_{\{t, i, k\}}|} \sum_{z \in \mathcal{G}_{\{t, i, k\}}} z, \\ w_{\{t, i, k\}} &= \frac{1}{|\mathcal{G}_{\{t, i, k\}}|} M_{\{t, i\}}. \end{aligned} \quad \text{\tag{17}}$$

无样本时三个类别分别使用 $-1.4, 0, 1.4$ 作为单点回退。当前需求样本为

$$\begin{aligned} n_{\{d, t, i, k\}} &= (\widehat{X}_{\{d, t\}}^{\{0.5\}} + \sigma_{\{d, t\}} \zeta_{\{t, i, k\}}) \Delta t, \\ \text{\quad} \sum_k w_{\{t, i, k\}} &= 1. \end{aligned} \quad \text{\tag{18}}$$

每个样本分别计算缺口再加权，不先把类别平均成单点，因为

$$\sum_k w_k w_{k-a}^+ \geq \left(\sum_k w_{k-a} \right)^+. \quad \text{\tag{19}}$$

压缩保留均值与极值，但不完整保存内部尾部损失，仍属于近似。

****第三步：明确实际可行动作。**** 暂省日期，令 x 为当前开始库存， n 为当前需求， $r = n - g_t$ 。若 $r \leq 0$ ，优先用富余充电：

$$\begin{aligned} c &= \min\{-r, 5000/6, (10800 - x)/0.9\}, \\ b &= e = 0, \quad w = -r - c, \quad x' = x + 0.9c. \end{aligned} \quad \text{\tag{20}}$$

若 $r > 0$ ，不允许充电或超缺口放电，以时段末库存 x' 为变量：

$$\begin{aligned} \max\{1200, x - \min(r, 5000/6)/0.9\} &\leq x' \leq x, \\ b &= 0.9(x - x'), \quad e = r - b, \quad c = w = 0. \end{aligned} \quad \text{\tag{21}}$$

这些分支保证充放电互斥、功率上限、库存边界及无应急充电。 e 可以等于零。

****第四步：建立库存价值和Bellman递推。**** 令 $W_t(x, i)$ 表示当前库存 x 、当前误差类别 i 已知但类内幅值尚未揭示时，从

当前到日末的最小预期应急费用与终端项。终端条件为

$$\begin{aligned} W_{\{145\}}(x, i) &= -\kappa_{dx}, \\ \kappa_d &= \begin{cases} \kappa_d, & \text{12月31日} \\ 0, & \text{12月31日} \end{cases} \end{aligned} \quad \text{\tag{22}}$$

日前LP的终端项为零；日内DP是否保留库存价值由本次重新验证的 κ 确定。它们是不同的参数，不能混写。终端项不计入实际电费。

对 $t < 144$ ，先求下一时段类别的条件期望

$$F_{\{t, i\}}(x') = \sum_j P_{\{t, ij\}} W_{\{t+1\}}(x', j). \quad \text{\tag{23}}$$

最后时段直接使用 $F_{\{144, i\}}(x') = -\kappa_{dx'}$ 。定义当前样本的单步费用算子

$$\mathcal{T}_{t,i}(x,n)=\begin{cases} F_{t,i}(x+0.9c), & n-g_t \leq 0, \\ \min_{x'} \{ \text{满足式 (21)} \} \\ \{ 5p_{t[n-g_t-0.9(x-x')]} + F_{t,i}(x') \}, & n-g_t > 0, \end{cases} \quad \text{tag{24}}$$

其中 c 由式 (20) 确定。由后向前递推

$$W_t(x,i)=\sum_{k} \mathcal{T}_{t,i}(x,n_{t,i,k}). \quad \text{tag{25}}$$

式 (25) 允许当前样本揭示后动作随之变化，但选择当前动作时，未来类别必须先按式 (23) 取期望。不能让当前动作依赖尚未出现的未来样本。

当前多放电减少当前应急费用，保留库存则可能减少后续高价时段应急费用。该权衡内生成储备，不额外添加固定SOC储备公式，也不加入CVaR。

库存价值采用60kWh间隔的161个网格点：

$$E=\{1200, 1260, \dots, 10800\}. \quad \text{tag{26}}$$

网格之间线性插值；实际库存仍连续。对缺口分支检查可行区间端点及内部网格节点，即可求得线性插值近似下的单步最小值。

第五步：顺序执行并跨日接续。每十分钟读取当前真实净需求和SOC，确定误差类别，再将真实需求代入式 (20) 或 (24)，只执行当前动作：

$$\begin{aligned} E_{d,t} &= E_{d,t-1} + 0.9c_{d,t} - b_{d,t}/0.9, \\ g_{d,t} + e_{d,t} + b_{d,t} &= N_{d,t} + c_{d,t} + w_{d,t}. \end{aligned} \quad \text{tag{27}}$$

本实现每天计算价值函数，日内按观测反馈执行，不每十分钟重新拟合预测或更新转移矩阵。名称为随机动态规划反馈控制，不将它写成逐步重新求解完整随机MPC。

一月预热从6000kWh起：首日计划为零，缺口按应急购电处理；随后每天采用昨日净需求正部作为计划并贪心平衡。完成整月后得到2月初库存 E_{ESTART} kWh。正式期采用

$$E_{d+1,0} = E_{d,144}, \quad \text{tag{28}}$$

所有对照从相同一月预热终点出发，之后各自连续运行。

第六步：重新选择参数并计算账单。用1月22—31日对当前“固定80分位、LP无终端项”的架构验证 κ ，候选是 $0.9 \times \text{mean}(p)$ 的 $0.25, 0.5, 0.75, 1, 1.25$ 倍。以统一库存价格 $v_{\text{ref}} = 0.9 \times \text{mean}(p)$ 计算

$$J_{\text{val}} = C_{\text{val}} + v_{\text{ref}} (E_{\text{start}} - E_{\text{end}}). \quad \text{tag{29}}$$

候选从同一1月22日期初库存出发，参数选择不回写一月实际预热轨迹。并列时取较小 κ ，2月1日前冻结。本次选定 $\kappa = KAPPA$ 元/电池kWh，验证表见下一节。

实际总费用独立按账单计算：

$$C_{\text{total}} = \sum_{d=\text{Feb1}}^{\text{Dec31}} \sum_{t=1}^{144}$$

```
(p_tg_{d,t}+5p_te_{d,t}). \tag{30}
$$
```

不扣除虚拟库存价值，不退还未使用的计划购电费用。

四、本次新结果、验证与复现

@@RESULT_TABLE@@

主方案中，普通购电费为@@NORMAL@@元，应急购电费@@EMERGENCY@@元；应急电量@@EM_KWH@@kWh，期末库存@@EEND@@kWh。

“同计划贪心”使用与主方案完全相同的每时段普通购电量，各自维持连续SOC，用于检查日内控制贡献。“独立贪心”每天从

自身实际库存解LP，所以它与主方案的费用差同时包含次日计划变化，不能全部归为单次放电控制收益。

本次1月参数验证结果为：

@@PARAM_TABLE@@

本次六个候选的验证账单和库存调整指标完全相同，因此1月验证无法识别终端参数的优劣；按事先规定的并列规则选择最小值0

。不能把它解释为0被证明优于其他参数。最终方案的日内终端价值因此也为0，但仍通过日内未来费用的Bellman递推安排储能。

预测中位数MAE为@@MAE@@kW，RMSE为@@RMSE@@kW；80分位实际覆盖率为@@COVERAGE@@%。这些指标均由本次新预测计算。

题目四个指定日的主方案结果：

@@DAYS_TABLE@@

![累计购电费用](results/累计费用.png)

图中累计费用按本次逐日账单计算。曲线只反映所测策略，不构成所有预测参数、状态划分或控制策略的最优性证明。

所有策略的供需平衡、SOC递推、功率上限、充放电互斥、无应急充电和跨日连续均通过检查，最大物理残差

@@MAX_ERROR@@kWh

。日前特征与概率的当前/未来数据扰动检查、执行阶段未来观测扰动检查均通过。类内需求0/100各半、计划50、最低库存

的反例返回125元应急期望费用，而非均值替代的0元。各CSV账单和Excel均独立回读复核。

****边界说明：**** 当前功率即时可测、三状态一阶转移、最多7个样本和60kWh价值网格仍是近似。1月验证较短，80分位由用户固

定，未验证其他分位是劣解。题目数据曾参与先前路线设计，本次独立重训不等于获得新盲测集。报告只使用本次运行结果

，不据此宣称全局最优。

复现和文件

在仓库根目录安装requirements.txt中的依赖，执行：

```
~~~powershell
python 代码/实验/q2_fresh_20260912/run_fresh.py --output 代码/实验/q2_fresh_20260912/results_new
~~~
```

默认读取“基础数据/C题/附件”或“基础数据”下的附件1、附件2，也可以通过--data-dir明确指定。输出目录必须为空；程

序拒绝加载或覆盖已有结果。文稿脚本write_report.py读取本次results目录生成报告。

- fresh_engine.py：独立预测、LP、Markov及Bellman函数，不依赖旧实验目录。
- run_fresh.py：完整数据读取、2457次拟合、验证、回测、输出和检查。
- results/training_trace.csv：每个预测日、训练起止日与实际拟合计数。
- results/fresh_forecasts.npz：本次新预测输出，仅为交付物，重跑时不加载它。
- results/main_intervals.csv：主方案48096行逐时明细。
- results/daily_summary.csv：全部策略每日费用与首末库存。
- results/result2.xlsx：计划、充放电、应急购电、逐时明细及四个指定日。
- results/controller_selection.json：本次重新进行的参数验证。
- results/manifest.json：原始附件及源码SHA256、环境、结果、验证和运行记录。

本次完整运行退出码0，用时@@SECONDS@@秒；本次拟合数@@FITS@@，读取旧预测文件数0，读取旧结果文件数0。报告与数据供审

核，未作全局最优或自动满足竞赛提交条件的声明。

```
def main():
    manifest=json.loads((OUT/'manifest.json').read_text(encoding='utf-8'))
    assert manifest['models_trained']==2457 and manifest['old_prediction_reads']==manifest['old_result_reads']
    ==
    0
    allstats=manifest['summary'];s=allstats['main'];cs=manifest['controller_selection'];fs=manifest['forecast_
se
lection'];fm=manifest['forecast_metrics']
    names=('main': '80分位LP+动态规划', 'same_plan_greedy': '同计划贪心', 'independent_greedy': '独立闭环贪心')
    table=' | 方案 | 普通费用/元 | 应急费用/元 | 总费用/元 | 应急电量/kWh | 期末库存/kWh
|\n|---|---|---|---|---|
--:|---|\n'
    table+=' \n'.join(f" | {names[n]} | {a['normal_cost']:.2f} | {a['emergency_cost']:.2f} |
{a['total_cost']:.
2f} | {a['emergency_kWh']:.2f} | {a['E_end']:.3f} |" for n,a in allstats.items())
    pt=' | kappa | 验证账单/元 | 库存调整指标/元 | 期末库存/kWh |\n|---|---|---|---|\n'
    pt+=' \n'.join(f" | {r['kappa']:.9f} | {r['bill']:.2f} | {r['inventory_adjusted']:.2f} | {r['end']:.3f} |"
for
    r in cs['candidates'])
    with (OUT/'daily_summary.csv').open(encoding='utf-8-sig',newline='') as f:rows=list(csv.DictReader(f))
    specified=[r for r in rows if r['policy']=='main' and r['date'] in
['2025-03-20','2025-06-21','2025-09-23',
2025-12-21']]
    dt=' | 日期 | 计划费/元 | 应急费/元 | 总费/元 | 日初库存/kWh | 日末库存/kWh
|\n|---|---|---|---|---|---|
\n'
    dt+=' \n'.join(' | '+r['date']+' | '+' | '.join(f"{float(r[k]):.2f}" for k in
['normal_cost','emergency_cost',
'total_cost','start_kwh','end_kwh']))+' |' for r in specified)
    replacements=dict(FITS=2457,TOTAL=f"{s['total_cost']:.2f}",WAN=f"{s['total_cost']/10000:.6f}",FAMILY=fs['f
a
mily'],
DIRECT_MAE=f"{fs['direct_MAE_kW']:.6f}",SEPARATE_MAE=f"{fs['separate_MAE_kW']:.6f}",
ESTART=f"{s['E_start']:.3f}",EEND=f"{s['E_end']:.3f}",KAPPA=f"{cs['selected']['kappa']:.
9f
}"',
RESULT_TABLE=table,PARAM_TABLE=pt,DAYS_TABLE=dt,NORMAL=f"{s['normal_cost']:.2f}",EMERGE
NC
Y=f"{s['emergency_cost']:.2f}",EM_KWH=f"{s['emergency_kWh']:.2f}",
MAE=f"{fm['MAE_kW']:.6f}",RMSE=f"{fm['RMSE_kW']:.6f}",COVERAGE=f"{100*fm['q80_coverage']:
.
4f}",
MAX_ERROR=f"{max(max(x.values()) for x in
manifest['checks']['physical'].values()):.3e}",S
ECONDS=f"{manifest['elapsed_seconds']:.2f}")
    text=TEXT
    for key,value in replacements.items():text=text.replace('@@'+key+'@@',str(value))
    assert '@@' not in text
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.family':'Microsoft YaHei','axes.unicode_minus':False,'font.size':10})
    fig,ax=plt.subplots(figsize=(7.5,4.2))
    for (name,label),color,style in zip(names.items(),['#0072B2','#D55E00','#009E73'],['-','--',':']):
        vals=np.array([float(r['total_cost']) for r in rows if r['policy']==name])
        ax.plot(np.arange(1,335),np.cumsum(vals)/10000,label=label,color=color,linestyle=style)
    ax.set(xlabel='自2月1日起的天数',ylabel='累计购电费用 (万元)');ax.legend();fig.tight_layout()
    fig.savefig(OUT/'累计费用.png',dpi=300);fig.savefig(OUT/'累计费用.svg');plt.close(fig)
    (BASE/'解法与结果.md').write_text(text,encoding='utf-8')
    doc=text.replace('(results/累计费用.png)','(.. /代码/实验/q2_fresh_20260912/results/累计费用.png)')
    (ROOT/'建模思路/第二问_完整重跑_具体解法与结果.md').write_text(doc,encoding='utf-8')
    readme=f"# 第二问：原始附件完整重跑\n\n总费用 **{s['total_cost']:.2f}元**，本次新训练2457个LightGBM模型，
无
```

旧预测/旧结果输入。 \n\n[具体解法与结果](解法与结果.md) • [结果Excel](results/result2.xlsx) • [完整逐时明细](results/main_intervals.csv) • [运行与校验记录](results/manifest.json)\n\n核心路线：固定80分位预测、日前无终端项LP、日内Markov与类内加权样本Bellman、连续SOC。 \n\n复现须新建空输出目录： \n\n``powershell\npython

```
-m p
ip install -r 代码/实验/q2_fresh_20260912/requirements.txt\npython 代码/实验
/q2_fresh_20260912/run_fresh.py
--output 代码/实验/q2_fresh_20260912/results_new\n```\n"
(BASE/' README.md').write_text(readme,encoding='utf-8')
(BASE/' requirements.txt').write_text(' \n'.join(f"{name}=={manifest[name]}" for name in
['numpy','scipy','openpyxl','lightgbm'])+' \n',encoding='utf-8')
proof=dict(report_sha256=hashlib.sha256((BASE/' 解法与结果.md').read_bytes()).hexdigest(),
all_outputs_sha256={p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in OUT.iterdir()
if p
.is_file()}),
status='checked',fit_count=2457,model_run_exit_code=0)
(OUT/' delivery_check.json').write_text(json.dumps(proof,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({'total_cost':s['total_cost'],'kappa':cs['selected'] ['kappa'],'report':str(BASE/' 解法与结果
.md')}),ensure_ascii=False))
```

if __name__=='__main__':main()

代码/实验/q3_revision_a_20260912/check_engine.py

"""Physical, settlement, interpolation, DP equivalence and information tests."""

```
import time
import numpy as np
import q3_engine as m
```

```
def checks(y,p,load_pred,f,pred):
    # Paid cancellation and repurchase must not disappear when final volume matches.
    old=np.array([100.]);down=np.array([80.]);price=np.array([1.])
    assert m.revision_cost(down,old,price)==-10.
    assert 100+m.revision_cost(down,old,price)+m.revision_cost(old,down,price)==120.
    assert m.revision_cost(old,old,price)==0.
    # Compare all value-grid entries to the existing scalar DP, including surplus.
    rng=np.random.default_rng(20260912);H=9
    samples=rng.uniform(-400,1500,(H,3,7));weights=rng.uniform(.1,1,(H,3,7));weights/=weights.sum(axis=2,keepd
im
s=True)
P=rng.uniform(.1,1,(H-1,3,3));P/=P.sum(axis=2,keepdims=True)
model=dict(samples=samples,weights=weights,P=P)
plans=rng.uniform(0,1200,(4,H));price=rng.uniform(.3,1.5,H)
values=m.bellman_batch(price,plans,model)
error=0.
for g,v in zip(plans,values):
    expected=m.b.bellman(price,g,model,0.)
    error=max(error,float(np.max(np.abs(v-expected))))
assert error<1e-7,error
toy=dict(samples=np.tile([0.,100.],(1,3,1)),weights=np.full((1,3,2),.5),P=np.empty((0,3,3)))
assert abs(m.bellman_batch(np.array([1.]),np.array([50.]),toy)[0,0,0,0]-125)<1e-8
# Actual forecast point +1 h must land at 07:00 for the 06:00 issue.
pv=m.pv_interpolate(y,f)
np.testing.assert_allclose(pv[:,1,41],f[:,1,0])
np.testing.assert_allclose(pv[:,0,5],f[:,0,0])
# LP settlement independent recomputation already inside revision_lp.
g=m.revision_lp(np.ones(2),np.array([50.,50.]),1200.,np.array([100.,100.]))
assert np.all(g>=0)
d=31;models=[m.model_day(y,pred,d,k) for k in range(4)]
warm=m.warmup(y,p);E=warm[-1,-1,6]
t0=time.perf_counter();run=m.day_run(y,p,pred,models,d,E,return_scores=True)
seconds=time.perf_counter()-t0
# Mutation at and after 06:00 cannot affect the 06:00 purchase revision.
changed=y.copy();changed[:,d,36:]+=np.array([10000.,3000.])[:,None]
cp,_=m.conditional_forecasts(changed,load_pred,f)
np.testing.assert_array_equal(cp[:,d],pred[:,d])
```

```

for k in [0,1]:np.testing.assert_array_equal(cp[d,k],pred[d,k])
cm=[m.model_day(changed,cp,d,k) for k in range(4)]
for k in [0,1]:
    for key in models[k]:np.testing.assert_array_equal(models[k][key],cm[k][key])
cr=m.day_run(changed,p,cp,cm,d,E,return_scores=True)
np.testing.assert_array_equal(run['initial'],cr['initial'])
np.testing.assert_array_equal(run['arr'][:36],cr['arr'][:36])
np.testing.assert_array_equal(run['ledger'][run['ledger'][:0]==1],cr['ledger'][cr['ledger'][:0]==1])
# Later published forecasts cannot change earlier actions or purchase revisions.
ff=f.copy();ff[d,2:]+=2000
pp,_=m.conditional_forecasts(y,load_pred,ff)
mm=[m.model_day(y,pp,d,k) for k in range(4)]
rr=m.day_run(y,p,pp,mm,d,E)
np.testing.assert_array_equal(run['arr'][:72],rr['arr'][:72])
np.testing.assert_array_equal(run['ledger'][run['ledger'][:0]==1],rr['ledger'][rr['ledger'][:0]==1])
# Existing no-update policy must reduce exactly to fixed-plan feedback execution.
no=m.day_run(y,p,pred,models,d,E,mask=0)
vv=m.b.bellman(p,no['initial'],models[0],0.)
aa=m.b.execute((y[0]-y[1])[d],p,no['initial'],E,models[0],vv)
np.testing.assert_allclose(no['arr'],aa,atol=1e-7,rtol=0)
return dict(scalar_bellman_max_error=error,expected_shortfall_counterexample=125.,
            successive_transaction_cost=120.,hourly_alignment=True,
            current_and_future_actual_no_plan_leak=True,later_issue_no_early_leak=True,
            fixed_plan_equivalence=True,smoke_day=m.summary_day(run,p),full_update_day_seconds=seconds)

if __name__=='__main__':
    import json
    y,p,load,f=m.load_inputs();pred,pv=m.conditional_forecasts(y,load,f)
    print(json.dumps(checks(y,p,load,f,pred),ensure_ascii=False,indent=2))

```

代码/实验/q3_revision_a_20260912/deliver_q3.py

```

"""Export, independently reconcile Q3 transactions and prepare review artifacts."""
from pathlib import Path
import argparse, csv, json, sys
from copy import copy
from datetime import datetime
import numpy as np
import q3_engine as m
from run_q3 import POLICIES, dump

def csvout(path,headers,rows):
    with path.open('w',encoding='utf-8-sig',newline='') as f:
        w=csv.writer(f);w.writerow(headers);w.writerows(rows)

def interval(t,end=None):
    if end is None:end=t+1
    return f'{t*10//60:02d}:{t*10%60:02d}-{end*10//60:02d}:{end*10%60:02d}'

def reconcile(arrays,initials,ledger,p,days,y,summary):
    errs={}
    for a,name in enumerate(POLICIES):
        A=arrays[a];flat=A.reshape(-1,7);g,c,b,e,w,before,after=flat.T
        net=(y[0]-y[1])[days].ravel()/6
        physical=max(float(abs(g+e+b-net-c-w).max()),float(abs(after-before-.9*c+b/.9).max()),float(abs(before
[1
        :]-after[:1]).max()))
        assert physical<1e-6
        assert before.min()>=1200-1e-6 and after.min()>=1200-1e-6 and before.max()<=10800+1e-6 and
after.max()<=
10800+1e-6
        assert max(c.max(),b.max())<=5000/6+1e-6 and np.minimum(c,b).max()<1e-6 and np.minimum(c,e).max()<1e-6
        assert flat[:,5].min()>-1e-6
        own=ledger[ledger[:,0]==a]
        replay=initials[a].copy();cost=float(np.sum(initials[a]*p))
        for row in own:

```

```

        _, d, k, t, old, new, up, down, inc, refund, penalty=row
        i=int(d)-int(days[0]);t=int(t);k=int(k)
        assert t>=36*k and k in [1,2,3] and new>=-1e-7
        assert abs(replay[i,t]-old)<1e-7
        assert abs((new-old)-(up-down))<1e-7 and min(up,down)<1e-8
        assert abs(inc-1.5*p[t]*up)<1e-7 and abs(refund-p[t]*down)<1e-7 and abs(penalty-.5*p[t]*down)<1e-7
        cost+=inc-refund+penalty;replay[i,t]=new
    np.testing.assert_allclose(replay,A[:, :, 0], atol=1e-7, rtol=0)
    cost+=float(5*np.sum(A[:, :, 3]*p))
    error=abs(cost-summary[name]['total_cost']);assert error<1e-5, (name, error)
    errs[name]=dict(physical_max_kwh=physical, ledger_cost_error_yuan=error, ledger_rows=len(own), schedule_r
ep
    lay=True)
    return errs

def export(out, arrays, initials, ledger, p, days, y, summary):
    main=arrays[7];init=initials[7];own=ledger[ledger[:, 0]==7];D=len(days)
    head=['date', 'slot', 'interval', 'initial_kwh', 'effective_kwh', 'charge_kwh', 'discharge_kwh', 'emergency_kwh', '
s
    pill_kwh', 'start_kwh', 'end_kwh', 'price', 'net_kwh']
    rows=[[m.date(d), t+1, interval(t), float(init[i, t]), *main[i, t].tolist(), float(p[t]), float((y[0, d, t]-y[1, d, t])
/
    6)] for i, d in enumerate(days) for t in range(144)]
    csvout(out/'main_intervals.csv', head, rows)
    ledger_head=['policy', 'date', 'issue_hour', 'slot', 'interval', 'old_kwh', 'new_kwh', 'increase_kwh', 'cancel_kwh'
,
    'increase_cost', 'refund', 'penalty', 'net_adjustment_cost']
    ledger_rows=[[POLICIES[int(a)], m.date(d), int(k)*6, int(t)+1, interval(int(t)), old, new, up, down, inc, ref, pen, in
c-
    ref+pen] for a, d, k, t, old, new, up, down, inc, ref, pen in ledger]
    csvout(out/'revision_ledger.csv', ledger_head, ledger_rows)
    # Preserve the four official worksheet objects and formatting, extend sample rows.
    wb=m.openpyxl.load_workbook(m.ROOT/'基础数据/result3.xlsx')
    for title, g in [('计划购电量', init), ('调整购电量', main[:, :, 0])]:
        ws=wb[title]
        for t in range(144):ws.cell(1, t+2, interval(t))
        for i, d in enumerate(days):
            ws.cell(i+2, 1, datetime.fromisoformat(m.date(d)))
            for t, v in enumerate(g[i]):ws.cell(i+2, t+2, float(v))
            ws.cell(i+2, 146, float(g[i].sum()))
            cost=float(init[i]*p)
            if title=='调整购电量':
                ll=own[own[:, 1]==d];cost+=float((ll[:, 8]-ll[:, 9]+ll[:, 10]).sum())
            ws.cell(i+2, 147, cost)
    storage=[];events=[]
    for i, d in enumerate(days):
        for k in range(6):storage.append([m.date(d), f'{k*4:02d}:00-{(k+1)*4:02d}:00', float(main[i, k*24:(k+1)*2
4,
        1].sum()), float(main[i, k*24:(k+1)*24, 2].sum()), '00:00' if k==0 else ('24:00' if k==1 else
None), float(main[i
        , 0, 5]) if k==0 else (float(main[i, -1, 6]) if k==1 else None)])
        t=0
        while t<144:
            if main[i, t, 3]<=1e-8:t+=1;continue
            start=t
            while t<144 and main[i, t, 3]>1e-8:t+=1
            events.append([m.date(d), interval(start, t), float(main[i, start:t, 3].sum())])
    for name, rr in [('充放电量', storage), ('紧急购电量', events)]:
        ws=wb[name]
        for row in ws.iter_rows(min_row=2):
            for cell in row:cell.value=None
        for i, row in enumerate(rr, 2):
            for j, value in enumerate(row, 1):ws.cell(i, j, value)
    ws=wb.create_sheet('调整逐笔账本');ws.append(ledger_head)
    for row in ledger_rows:
        if row[0]=='mask_111':ws.append(row)
    ws=wb.create_sheet('各次计划快照');ws.append(['日期', '发布时间']+interval(t) for t in range(144))
    for i, d in enumerate(days):

```

```

        current=init[i].copy();ws.append([m.date(d),'00:00',*current.tolist()])
        for k in range(1,4):
            for row in own[(own[:,1]==d)&(own[:,2]==k)]:current[int(row[3])]=row[5]
            ws.append([m.date(d),f'{k*6:02d}:00',*current.tolist()])
ws=wb.create_sheet('十分钟明细');ws.append(head)
for row in rows:ws.append(row)
daily=list(csv.DictReader((out/'daily_summary.csv').open(encoding='utf-8-sig'))))
ws=wb.create_sheet('每日费用');ws.append(list(daily[0]))
for row in daily:
    if row['policy']=='mask_111':ws.append([v if key in ('policy','date') else float(v) for key,v in
row.items()])
specified=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
table1=[];table2=[];table3=[]
for ds in specified:
    d=(datetime.fromisoformat(ds)-datetime(2025,1,1)).days
    if d not in days:continue
    i=list(days).index(d);r=next(r for r in daily if r['policy']=='mask_111' and r['date']==ds)
    for t in [60,72,84,96,108,120]:table1.append([ds,interval(t),float(init[i,t]),float(main[i,t,0]),float
(m
ain[i,t,3])])
    table1.append([ds,'全天',float(init[i].sum()),float(main[i,:,0].sum()),float(main[i,:,3].sum())])
    table2.extend([rr for rr in storage if rr[0]==ds])
    ev=[rr for rr in events if rr[0]==ds];table3.extend(ev or [[ds,'无紧急购电',0.]])
for name,headers,rr in [('指定日表1',['日期','区间','初始购电kWh','有效普通购电kWh','紧急购电
kWh'],table1),('指定日表2',['日期','区间','充电kWh','放电kWh','时刻','库存kWh'],table2),('指定日表3',['日期','连续紧急时
段','紧急购电kWh'],table3)]:
    ws=wb.create_sheet(name);ws.append(headers)
    for row in rr:ws.append(row)
    csvout(out/(name+'.csv'),headers,rr)
ws=wb.create_sheet('说明');ws.append(['项目','口径'])
for key,value in dict(method='新版Q2负载预测+条件净误差分位+调整LP候选+DP评价和反馈执行',
    settlement='每次对上一有效计划结算;增购1.5倍;撤销量退原价再缴0.5倍违约金。此为已确
认
建模解释。',
    main='每天0、6、12、18时使用预报;候选包含不调整;不是事后最优组合',
    adjusted_sheet='保存最终有效普通购电量;全天购电费=初始计划费+所有逐笔调整净费用,不
含
紧急费',
    total_bill='每日费用表total_cost包含紧急费;不再对最终有效普通购电量重复收费',
    time='输入标签为区间终点;输出副本首列改为00:00—00:10,末列23:50—24:00;原模板保留',
    instantaneous_observation='当前十分钟功率可测且分段恒定;发布时计划不读取当期未来实
测',
    pv='小时整点线性插值至10分钟终点;发布点左锚为刚完成区间的观测近似;优化止于当日
24:00',
    state='1月1日6000kWh,完整1月共同预热后2月10800kWh;全年跨日连续',
    terminals='日前LP与日内DP终端系数均固定0;未声称第三问调参得到最优0',
    future='未来情景先取期望,下一次预报到来再修订;暂冻结未来交易的滚动近似',
    source='复用Q2已验证的历史负载预测文件;本次未重新训练;Q3购电和运行轨迹全部新算',
    status='真实计算结果供用户审核,未冻结为论文正文').items():ws.append([key,value])
for ws in wb:
    ws.freeze_panes='B2';ws.auto_filter.ref=ws.dimensions
    ws.column_dimensions['A'].width=17
wb.save(out/'result3.xlsx')
# Independent re-read of actual XLSX/CSV output.
wb=m.openpyxl.load_workbook(out/'result3.xlsx',read_only=True,data_only=True)
rr=list(wb['计划购电量'].values);assert len(rr)==D+1
np.testing.assert_allclose(np.array([r[1:145] for r in rr[1:]],float),init,rtol=0,atol=1e-8)
rr=list(wb['调整购电量'].values)
np.testing.assert_allclose(np.array([r[1:145] for r in rr[1:]],float),main[:, :, 0],rtol=0,atol=1e-8)
ordinary=sum(float(r[-1]) for r in rr[1:]);assert abs(ordinary+summary['mask_111']['emergency_cost']-summa
ry
['mask_111']['total_cost'])<1e-5
rr=list(wb['充放电量'].values);assert len(rr)==D*6+1

```

```

assert abs(sum(float(r[2]) for r in rr[1:])-float(main[:, :, 1].sum()))<1e-5
assert abs(sum(float(r[3]) for r in rr[1:])-float(main[:, :, 2].sum()))<1e-5
rr=list(wb['紧急购电量'].values)
assert abs(sum(float(r[2] or 0) for r in rr[1:])-summary['mask_111']['emergency_kwh'])<1e-5
assert wb['十分钟明细'].max_row==D*144+1
wb.close()
with (out/'main_intervals.csv').open(encoding='utf-8-sig') as f: rr=list(csv.DictReader(f))
assert len(rr)==D*144
emergency=sum(5*float(r['price'])*float(r['emergency_kwh']) for r in rr)
assert abs(emergency-summary['mask_111']['emergency_cost'])<1e-5
return dict(xlsx_readback=True, main_csv_readback=True, rows=D*144, merged_emergency_events=len(events), templ
at
e_original_unchanged=True)

def sensitivity(out, y, p, pred, arrays, days):
    rows=[]
    for ds in ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']:
        d=(datetime.fromisoformat(ds)-datetime(2025, 1, 1)).days
        if d not in days:continue
        i=list(days).index(d);E=arrays[7, i, 0, 5]
        for label, grid, atoms in [('nominal', m.GRID, 7), ('grid_30kwh', np.arange(1200, 10801, 30.), 7), ('atoms_15', m.
G
RID, 15)]:
            models=[m.model_day(y, pred, d, k, atoms) for k in range(4)]
            run=m.day_run(y, p, pred, models, d, E, grid=grid)
            if label=='nominal':np.testing.assert_allclose(run['arr'], arrays[7, i], atol=1e-7, rtol=0)
            rows.append(dict(date=ds, setting=label, **m.summary_day(run, p)))
    csvout(out/'sensitivity.csv', list(rows[0]), [list(r.values()) for r in rows])
    return rows

def plots(out, summary):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.family':'Microsoft YaHei', 'axes.unicode_minus':False, 'font.size':10})
    labels=['不更新', '6时', '12时', '6+12时', '18时', '6+18时', '12+18时', '全部', '仅更新预报']
    total=np.array([summary[n]['total_cost'] for n in POLICIES])/1e4
    emer=np.array([summary[n]['emergency_cost'] for n in POLICIES])/1e4
    fig, ax=plt.subplots(figsize=(8, 4));x=np.arange(9)
    ax.bar(x, total-emer, color='#4DBBD5', label='普通购电费用（含调整）')
    ax.bar(x, emer, bottom=total-emer, color='#E64B35', label='紧急购电费')
    ax.set_xticks(x, labels, rotation=25, ha='right');ax.set_ylabel('累计费用 / 万元')
);ax.legend(frameon=False, font
size=9, loc='upper center', bbox_to_anchor=(.5, 1.18), ncol=2)
ax.spines[['top', 'right']].set_visible(False);fig.tight_layout()
fig.savefig(out/'预报组合费用.png', dpi=300);fig.savefig(out/'预报组合费用.svg');plt.close(fig)
with (out/'daily_summary.csv').open(encoding='utf-8-sig') as f:rr=list(csv.DictReader(f))
fig, ax=plt.subplots(figsize=(7, 3.8))
for key, label, color, style in [('mask_000', '仅0时预报', '#3C5488', '-'), ('mask_111', '四次预报并调整',
, '#E64B35',
'-'), ('forecast_only', '四次预报, 仅调储能', '#00A087', '--')]:
    vals=[float(r['total_cost']) for r in rr if r['policy']==key]
    ax.plot(np.arange(1, len(vals)+1), np.cumsum(vals)/1e4, label=label, color=color, linestyle=style)
ax.set_xlabel('2月1日起累计天数');ax.set_ylabel('累计费用 / 万元');ax.legend(frameon=False, fontsize=9)
ax.spines[['top', 'right']].set_visible(False);fig.tight_layout()
fig.savefig(out/'累计费用.png', dpi=300);fig.savefig(out/'累计费用.svg');plt.close(fig)

def report(out, summary, checks, sens):
    main=summary['mask_111'];fixed=summary['mask_000'];fo=summary['forecast_only']
    q2=json.loads((m.Q2/'results/manifest.json').read_text(encoding='utf-8'))['summary']['main']
    names=['仅0时', '0、6时', '0、12时', '0、6、12时', '0、18时', '0、6、18时', '0、12、18时', '四次预报并调整', '四次
预报、购电固定']
    text=['# 第三问路线A: 真实回测结果审核', '',
        '状态: 计算与交付核验完成, 等待用户审核结果; 本文件是结果审核报告, 未替代正式Word论文章节.', '',

```

```

        '**结算前提：撤销量退还原价后缴50%违约金，增购按1.5倍，每次对上一有效计划逐笔结算。该解释已随路线A确
    认
        ，属于建模口径，不是额外取得的官方释义。**，''，
        '负载预测复用队友新版第二问的已验证前推结果：本次没有重新训练LightGBM。附件3各次PV预报、条件净误差、
LP
    调整候选、DP及所有第三问实际调度重新计算。原题目、模板及所用第二问代码/结果哈希前后保持一致；第二问源码仅
Git
    换行差异经LF归一核验。'，''，
        f''评价区间为2025年2月1日至12月31日，共{main['days']}天。主方案预先定义为四次使用预报，日内三次均可选
    择
        保持计划不变，并非事后选全年最优组合。主方案总费用 **{main['total_cost']:.2f}元**，紧急购电
**{main['emerge
    ncy_kwh']:.2f}kWh**。'，''，
        ' | 费用项 | 金额/元 | '，' |---|---:| '
        for label, key in [('0时初始计划费', 'initial_cost'), ('后续增购费（1.5倍）', 'increase_cost'), ('撤销量退款（
    扣
        减）', 'cancellation_refund'), ('撤销违约金（0.5倍）', 'cancellation_penalty'), ('紧急购电费（5倍）'
    , 'emergency_
        cost'), ('合计', 'total_cost')]: text.append(f" | {label} | {main[key]:.2f} | ")
        text+=['', f"账单恒等式：初始费+增购费 退款+违约金+紧急费。最终普通购电量为
{main['effective_kwh']:.2f}kW
        h，不能再乘原价重复记费。'，''，
            ' | 使用预报时刻 | 总费用/元 | 紧急费/元 | 紧急电量/kWh | 期末库存/kWh
    | '，' |---|---:|---:|---:| '
        for name, label in zip(POLICIES, names):
            r=summary[name]; text.append(f" | {label} | {r['total_cost']:.2f} | {r['emergency_cost']:.2f} |
{r['emer
        gency_kwh']:.2f} | {r['end_kwh']:.2f} | ")
        text+=['', '为直接回答是否增加其他预报时刻，以下在已使用另外两个日内时刻的基础上，计算增加剩余时刻后的费用
    变
        化。负数表示节省，正数表示增加费用。'，''，
            ' | 新增调整时刻 | 原先日内时刻 | 费用变化/元 | '，' |---|---:|---:| '
        for hour, base, label in [(6, 'mask_110', '12、18时'), (12, 'mask_101', '6、18时'), (18, 'mask_011', '6、12时')]:
            text.append(f" | {hour}:00 | {label} | {main['total_cost']-summary[base]['total_cost']:.2f} | ")
            text+=['', '在本次近似控制及结算口径下，12时和18时更新具有正的组合收益；在已使用12、18时的基础上增加6时，全
    年
        费用反而上升。此结论限于本次数据和参数，不能写成6时预报在所有情况下都无用。'，'']
        delta=fixed['total_cost']-main['total_cost']
        best=min(POLICIES[:8], key=lambda n:summary[n]['total_cost'])
        text+=['', f"与同样利用附件3零点预报、全天固定购电的对照相比，对照费用减去主方案费用为{delta:.2f}元，主方
    案
        相对费用变化为{-100*delta/fixed['total_cost']:.4f}%。八个组合中样本期总费最低为
        "{names[POLICIES.index(best
    )]}"; 这一排序是整段回溯描述，不能当作事前知道的最优选择。'，''，
            f"“更新预报但不改购电计划”的对照总费{fo['total_cost']:.2f}元。它每天从自身实际日初库存制定计划，
    用
        于区分更新信息作用和允许交易的作用；长期差异还包含跨日库存导致的次日计划变化，不能全部归为某一次储能控制。"
    , ''，
        f"队友新版第二问总费{q2['total_cost']:.2f}元。与第三问零点预报固定计划对照相比，差额为
{q2['total_cos
        t']-fixed['total_cost']:.2f}元：该差额同时包含外部PV预报和相应净误差校准变化，不能全归因于日内调整。'，''，
            '![各组合费用](预报组合费用.png)'，''，
            '图中普通购电费用已经合并增购、退款及违约金，紧急费用独立列出。对应数值见summary.json和逐日账单。'
    , ''，
        '![累计费用](累计费用.png)'，''，
        '累计曲线只比较已定义的可执行策略：没有事后逐日选取最便宜策略。'，''，
        '## 指定日期与交付表'，''，
            ' | 日期 | 初始费/元 | 调整净费用/元 | 紧急费/元 | 总费/元 | 日初/日末库存kWh
    | '，' |---|---:|---:|---:|
        ---:|---| '
        with (out/'daily_summary.csv').open(encoding='utf-8-sig') as f: daily=list(csv.DictReader(f))
        for ds in ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']:
            r=next(r for r in daily if r['policy']=='mask_111' and r['date']==ds)
            text.append(f" | {ds} | {float(r['initial_cost']):.2f} | {float(r['adjustment_net']):.2f} |
{float(r['e
            mergency_cost']):.2f} | {float(r['total_cost']):.2f} | {float(r['start_kwh']):.2f} /
{float(r['end_kwh']):.2f} | ")

```

```

.2f) |")
text+=[',', 'result3.xlsx保留官方四张工作表并补齐全年数据，增加逐笔账本、四次计划快照、十分钟明细、费用表及
指 定日表。输入结束时按用户已确认口径映射为00:00—00:10至23:50—24:00，原模板未改。紧急购电按连续时段合并。’
,
'',
    '## 已完成的验证与边界’,’,
    f"独立逐笔重放账本、物理约束及跨日连续通过，最大物理残差为{max(v['physical_max_kwh'] for v in
checks[
    'independent'].values()):.3e}kWh。CSV及Excel保存后回读一致。加速DP与原标量DP全网格对比、125元类内缺口反例、
1 20元撤销再买回账单、发布时刻因果扰动及固定计划退化检查通过。”,’,
    '| 日期 | 精度设置 | 总费/元 | 末库存/kWh |’,’|---|---|---:|---:|’]
    for r in sens:text.append(f" | {r['date']} | | {r['setting']} | | {r['total_cost']:.4f} | | {r['end_kwh']:.4f}
|")
    deviations=[abs(r['total_cost']-next(x['total_cost'] for x in sens if x['date']==r['date'] and
x['setting']==
    'nominal')) for r in sens]
text+=[',', f"四日精度变动下最大费用差为{max(deviations):.2f}元；网格细化会改变个别调整候选和轨迹，因此尚不
能 宣称调度对离散精度不敏感。”,’,]
text+=[',', '精度检查从主方案该日相同日初库存开始，重新优化该日；它只支持四个指定日的局部敏感性，不证明全年
网 格误差上界。’,’,
    '仍有明确近似：当前十分钟平均功率即时可测；整点PV线性插值；三状态一阶转移、类内7点压缩、60kWh价值网
格 ；有限分位候选；当前评价暂不预知下一次交易；每日规划止于24:00且终端系数固定0。没有声称全局最优。既有全年数
据 参与过方法探索，本次也不是新的盲测集。’,’,
    '## 复现’,’,
    '在项目根目录，用已安装numpy、scipy、lightgbm及openpyxl的Python运行：’,’,
    '``powershell’,’python 代码/实验/q3_revision_a_20260912/run_q3.py --output 代码/实验
/q3_revision_a_2
0260912/results_new’,
    'python 代码/实验/q3_revision_a_20260912/deliver_q3.py --output 代码/实验
/q3_revision_a_20260912/resu
lts_new’,’``’,’,
    'run_q3.py拒绝覆盖非空目录；中断后仅在代码、输入和参数完全一致时接受--resume。预测复用来源、输入和
输 出SHA256、运行命令、环境、耗时和验证记录见manifest.json与复现清单.json。’,’,]
report_dir=m.ROOT/' 论文写作/问题三';report_dir.mkdir(parents=True,exist_ok=True)
import os
content='\n'.join(text)
for image in ['预报组合费用.png', '累计费用.png']:
    content=content.replace('['+image+',', ']['+Path(os.path.relpath(out/image,report_dir)).as_posix()+')')
    (report_dir/'第三问_路线A结果审核报告.md').write_text(content,encoding='utf-8')

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--output', type=Path, default=m.BASE/'results');args=ap.parse_
ar
gs();out=args.output.resolve()
with np.load(out/'solutions.npz') as z:arrays=z['arrays'];initials=z['initials'];ledger=z['ledger'];p=z['p
ri
ce'];days=z['day_indices']
summary=json.loads((out/'summary.json').read_text(encoding='utf-8'))
y, price, load, f=m.load_inputs();np.testing.assert_array_equal(price, p)
with np.load(out/'conditional_forecasts.npz') as z:pred=z['pred']
independent=reconcile(arrays, initials, ledger, p, days, y, summary)
delivery=export(out, arrays, initials, ledger, p, days, y, summary)
sens=sensitivity(out, y, p, pred, arrays, days)
plots(out, summary)
checked=dict(independent=independent, delivery=delivery, sensitivity_days=4, visual_check='pending_manual_vie
w',
)
dump(out/'delivery_checks.json', checked);report(out, summary, checked, sens)
manifest=json.loads((out/'manifest.json').read_text(encoding='utf-8'))
for path,digest in manifest['signature']['inputs'].items():assert m.sha(path)==digest

```

```

        manifest.update(status='numerically_verified_pending_visual_and_user_review', delivery_command=[sys.executable, str(Path(__file__).relative_to(m.ROOT)), *sys.argv[1:]], delivery_source_sha256=m.sha(__file__))
        report_path=m.ROOT/'论文写作/问题三/第三问_路线A结果审核报告.md'
        manifest['review_report']={'path':str(report_path), 'sha256':m.sha(report_path)}
        manifest['output_hashes']={f.name:m.sha(f) for f in out.iterdir() if f.is_file() and f.name not in
['manifests
t.json', '复现清单.json', 'checkpoint.npz', 'daily_checkpoint.json', 'scores_checkpoint.json']}
dump(out/'manifest.json', manifest);dump(out/'复现清单.json', manifest)
print(json.dumps(checkered, ensure_ascii=False, indent=2))

```

```

if __name__ == '__main__': main()

```

代码/实验/q3_revision_a_20260912/q3_engine.py

"""Q3 route A: issue-time forecasts, paid plan revisions, stochastic DP.

Uses read-only Q2 source and its hash-verified causal load forecast artifact.

The published PV forecasts are treated as hourly point forecasts.

"""

```

from pathlib import Path
import sys
import importlib.util
import json
import hashlib
from datetime import datetime, timedelta
import numpy as np
from scipy.optimize import linprog
from scipy.sparse import lil_matrix

BASE = Path(__file__).resolve().parent
ROOT = BASE.parents[2]
Q2 = BASE.parent / 'q2_fresh_20260912'
try:
    import openpyxl
except ImportError:
    sys.path.append(str(Path.home()/'/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/Lib/site-packages'))
    import openpyxl
spec = importlib.util.spec_from_file_location('q3_q2_core', Q2/'fresh_engine.py')
b = importlib.util.module_from_spec(spec)
spec.loader.exec_module(b)
DT, ETA, CAP, EMIN, EMAX, GRID = b.DT, b.ETA, b.CAP, b.EMIN, b.EMAX, b.GRID
QUANTILES = b.QUANTILES
TOL = 2e-5

```

```

def sha(p):
    return hashlib.sha256(Path(p).read_bytes()).hexdigest()

```

```

def load_inputs():
    data = ROOT/'基础数据'
    manifest = json.loads((Q2/'results/manifest.json').read_text(encoding='utf-8'))
    for name, digest in manifest['raw_inputs'].items():
        assert sha(data/name) == digest, f'Q2 input mismatch: {name}'
    for name, digest in manifest['source_hashes'].items():
        raw=(Q2/name).read_bytes()
        # Git on Windows converted LF to CRLF; accept only this exact byte change.
        normalized=hashlib.sha256(raw.replace(b'\r\n',b'\n')).hexdigest()
        assert sha(Q2/name) == digest or normalized == digest, f'Q2 source mismatch: {name}'
    cache = Q2/'results/fresh_forecasts.npz'
    assert sha(cache) == manifest['output_hashes']['fresh_forecasts.npz']
    with np.load(cache) as f:
        load_pred = f['separate'][:, 0].copy()
    wb = openpyxl.load_workbook(data/'附件2.xlsx', read_only=True, data_only=True)
    y = []
    for name in ['小区负载', '光伏发电实际功率']:

```

```

        rows = list(wb[name].values)
        assert len(rows) == 366 and len(rows[0]) == 145
        assert [r[0] for r in rows[1:]] == [datetime(2025,1,1)+timedelta(days=d) for d in range(365)]
        y.append(np.asarray([r[1:] for r in rows[1:]], float))
wb.close()
y = np.asarray(y)
wb = openpyxl.load_workbook(data/'附件1.xlsx', read_only=True, data_only=True)
p = np.array([r[1] for r in list(wb.active.values)[1:]], float)
wb.close()
forecasts = np.full((365,4,24), np.nan)
wb = openpyxl.load_workbook(data/'附件3.xlsx', read_only=True, data_only=True)
day = None
for row in list(wb.active.values)[1:]:
    if row[0]: day = (datetime.strptime(row[0], '%Y-%m-%d')-datetime(2025,1,1)).days
    hour = int(row[1].split(':')[0]); assert hour in [0,6,12,18]
    issue = hour//6
    assert np.isnan(forecasts[day,issue]).all()
    forecasts[day,issue] = row[2:]
wb.close()
assert y.shape == (2,365,144) and p.shape == (144,)
assert np.isfinite(y).all() and np.isfinite(forecasts).all() and (forecasts>=0).all()
return y, p, load_pred, forecasts

def pv_interpolate(y, forecasts):
    pv = np.full((365,4,144), np.nan)
    for d in range(365):
        for k in range(4):
            start = k*36
            anchor = y[1,d,start-1] if start else (y[1,d-1,-1] if d else 0.)
            values = np.r_[anchor, forecasts[d,k]]
            pv[d,k,start:] = np.interp(np.arange(1,145-start)/6., np.arange(25), values)
    return pv

def conditional_forecasts(y, load_pred, forecasts):
    pv = pv_interpolate(y, forecasts)
    mu = load_pred[:,None,:]-pv
    net = y[0]-y[1]
    residual = net[:,None,:]-mu
    pred = np.full((365,4,5,144), np.nan)
    for d in range(1,365):
        ids = np.arange(max(14,d-30), d)
        for k in range(4):
            start = k*36
            pred[d,k,:,start:] = mu[d,k,start:][None,:]
            if len(ids)>=4:
                for lo in range(start,144,36):
                    hi = min(144,lo+36)
                    adjust = np.quantile(residual[ids,k,lo:hi], QUANTILES)
                    pred[d,k,:,lo:hi] += adjust[:,None]
            pred[d,k,:,start:] = np.sort(pred[d,k,:,start:],axis=0)
    return pred, pv

def model_day(y, pred, d, k, atoms=7):
    """Estimates use completed days only; initial class is a historical prior."""
    start = k*36; H = 144-start
    ids = np.arange(max(18,d-30), d)
    assert len(ids)>0
    hist = pred[ids,k,:,start:]
    scale_hist = np.maximum((hist[:,3]-hist[:,0])/1.683,100.)
    z = ((y[0]-y[1])[ids,start:]-hist[:,1])/scale_hist
    states = np.digitize(z, [-.84,.84])
    scale = np.maximum((pred[d,k,3,start:]-pred[d,k,0,start:])/1.683,100.)
    center = pred[d,k,1,start:]
    count = np.ones((3,3)); np.add.at(count, (states[:, :-1].ravel(), states[:, 1:].ravel()), 1)
    global_p = count/count.sum(axis=1,keepdims=True)
    P = np.empty((H-1,3,3)); samples = np.zeros((H,3,atoms)); weights = np.zeros_like(samples)

```

```

for t in range(H):
    lo, hi = max(0, t-3), min(H, t+4)
    for s in range(3):
        values = z[:, lo:hi][states[:, lo:hi]==s]
        if not len(values): values = np.array([[-1.4, 0., 1.4][s]])
        a, w = b.atomize(values, atoms)
        samples[t, s, :len(w)] = (center[t]+scale[t]*a)*DT
        weights[t, s, :len(w)] = w
    if t<H-1:
        lo, hi = max(0, t-3), min(H-1, t+4)
        count = 3*global_p.copy()
        np.add.at(count, (states[:, lo:hi].ravel(), states[:, lo+1:hi+1].ravel()), 1)
        P[t] = count/count.sum(axis=1, keepdims=True)
initial = np.bincount(states[:, 0], minlength=3)+[.6, 1.8, .6]
initial = initial/initial.sum()
return dict(P=P, samples=samples, weights=weights, initial=initial, center=center, scale=scale)

def revision_cost(new, old, p):
    diff = new-old
    up, down = np.maximum(diff, 0), np.maximum(-diff, 0)
    return float(1.5*p@up-.5*p@down)

def revision_lp(p, demand, E0, old):
    """No emergency in candidate LP. Includes refunds and cancellation charge."""
    H = len(p); dim = 7*H+1
    # g, c, b, w, E[H+1], up, down
    g, c, q, w, E, up, down = 0, H, 2*H, 3*H, 4*H, 5*H+1, 6*H+1
    obj = np.zeros(dim); obj[up:up+H]=1.5*p; obj[down:down+H]=-0.5*p
    bounds = [(0, None)]*dim
    A = lil_matrix((3*H, dim)); rhs = np.r_[demand, np.zeros(H), old]
    for t in range(H):
        A[t, [g+t, c+t, q+t, w+t]]=[-1, -1, 1, -1]
        A[H+t, [E+t+1, E+t, c+t, q+t]]=[-1, -1, -ETA, 1/ETA]
        A[2*H+t, [g+t, up+t, down+t]]=[-1, -1, 1]
        bounds[c+t]=bounds[q+t]=(0, CAP)
    for t in range(H+1): bounds[E+t]=(EMIN, EMAX)
    bounds[E]=(E0, E0)
    res=linprog(obj, A_eq=A.tocsr(), b_eq=rhs, bounds=bounds, method='highs')
    assert res.success, res.message
    assert abs(A@res.x-rhs).max()<TOL
    plan=np.maximum(res.x[:H], 0)
    assert abs(revision_cost(plan, old, p)-res.fun)<TOL
    return plan

def interp_batch(f, x, grid):
    """f: K,G; x: K,S,G. Linear interpolation on a regular grid."""
    loc=np.clip((x-grid[0])/(grid[1]-grid[0]), 0, len(grid)-1)
    i=np.minimum(loc.astype(int), len(grid)-2); frac=loc-i
    rows=np.arange(len(f))[:, None, None]
    return (1-frac)*f[rows, i]+frac*f[rows, i+1]

def bellman_batch(p, plans, m, grid=GRID):
    """Vectorized, identical piecewise-linear minimization to the Q2 core.

    Batch candidates share exogenous demand samples, not inventory or actions.
    """
    plans=np.atleast_2d(plans); K,H=plans.shape; G=len(grid)
    V=np.zeros((K, H+1, 3, G)); step=grid[1]-grid[0]
    width=int(np.floor(CAP/ETA/step+1e-10))+1
    ix=np.arange(G)[:, None]-np.arange(width)[None, :]
    valid=ix>=0; ix=np.maximum(ix, 0)
    for t in range(H-1, -1, -1):
        f_all=V[:, t+1] if t==H-1 else np.einsum('ij, kjg->kig', m['P'][t], V[:, t+1])
        for s in range(3):
            f=f_all[:, s]; r=m['samples'][t, s][None, :]-plans[:, t, None]

```

```

x=grid[None, None, :]
distance=np.minimum(np.maximum(r, 0), CAP)/ETA
lo=np.maximum(EMIN, x-distance[:, :, None])
knot=f+5*p[t]*ETA*grid
windows=np.where(valid[None, :, :], knot[:, ix], np.inf)
windows=np.minimum.accumulate(windows, axis=2)
size=np.floor(distance/step+1e-10).astype(int)
minima=windows[np.arange(K)[:, None, None], np.arange(G) [None, None, :], size[:, :, None]]
boundary=interp_batch(f, lo, grid)+5*p[t]*ETA*lo
deficit=5*p[t]*(r[:, :, None]-ETA*x)+np.minimum(minima, boundary)
nxt=np.minimum(EMAX, x+ETA*np.minimum(np.maximum(-r, 0), CAP)[:, :, None])
surplus=interp_batch(f, nxt, grid)
cost=np.where(r[:, :, None]>0, deficit, surplus)
V[:, t, s]=np.einsum('ksg, s->kg', cost, m['weights'])[t, s]
assert np.isfinite(V).all() and np.diff(V, axis=3).max()<TOL
return V

def day_run(y, p, pred, models, d, E0, mask=7, update_only=False, grid=GRID, return_scores=False):
    initial=b.plan_lp(p, pred[d, 0, 3]*DT, E0, 0.)
    current=initial.copy();E=float(E0);rows=[];revisions=[];scores=[]
    active=0;start=0;model=models[0]
    V=bellman_batch(p, current, model, grid)[0]
    for t in range(144):
        if t in (36, 72, 108):
            k=t//36
            if update_only or (mask & (1<<(k-1))):
                active=k;start=t;model=models[k];old=current[t:].copy()
                plans=[old];labels=['keep']
                if not update_only:
                    for qi in [1, 2, 3, 4]:
                        proposal=revision_lp(p[t:], pred[d, k, qi, t:]*DT, E, old)
                        if not any(np.allclose(proposal, g, atol=1e-8, rtol=0) for g in plans):
                            plans.append(proposal);labels.append(str(QUANTILES[qi]))
                    plans=np.asarray(plans);Vs=bellman_batch(p[t:], plans, model, grid)
                    charge=np.array([revision_cost(g, old, p[t:]) for g in plans])
                    expected=np.array([sum(model['initial'][s]*np.interp(E, grid, v[0, s]) for s in range(3)) for s
in
Vs])

                    total=charge+expected;best=int(np.argmin(total))
                    assert total[best]<=total[0]+TOL
                    current[t:]=plans[best];V=Vs[best]
                    diff=current[t:]-old
                    for j, (before, after, delta) in enumerate(zip(old, current[t:], diff), start=t):
                        if abs(delta)>1e-8:
                            up=max(delta, 0.);down=max(-delta, 0.)
                            revisions.append([k, j, before, after, up, down, 1.5*p[j]*up, p[j]*down, .5*p[j]*down])
                    if return_scores:
                        for l, c, e, v in zip(labels, charge, expected, total):scores.append([k, l, float(c), float(e), float
t(
v), l==labels[best]])
                loc=t-start
                actual=(y[0, d, t]-y[1, d, t]);s=int(np.digitize((actual-model['center'])[loc])/model['scale'])[loc], [-.84, .
84
]))
                future=V[loc+1, s] if t==143 else model['P'][loc, s]*V[loc+1]
                c, q, e, w, nxt=b.action(E, actual*DT-current[t], p[t], future, grid)
                rows.append([current[t], c, q, e, w, E, nxt]);E=nxt
            arr=np.asarray(rows);b.validate(arr, (y[0]-y[1])[d], p)
            return dict(arr=arr, initial=initial, ledger=np.asarray(revisions, float).reshape(-1, 9), scores=scores)

def summary_day(run, p):
    arr=run['arr'];ledger=run['ledger']
    initial=float(run['initial'])*p
    up=float(ledger[:, 6].sum());refund=float(ledger[:, 7].sum());penalty=float(ledger[:, 8].sum())
    emergency=float(5*p*arr[:, 3])
    return dict(initial_cost=initial, increase_cost=up, cancellation_refund=refund, cancellation_penalty=penalty,

```

```

        adjustment_net=up-refund+penalty, emergency_cost=emergency, total_cost=initial+up-refund+penalty
+e
    emergency,
        initial_kwh=float(run['initial'].sum()), effective_kwh=float(arr[:, 0].sum()),
        increase_kwh=float(ledger[:, 4].sum()), cancel_kwh=float(ledger[:, 5].sum()),
        emergency_kwh=float(arr[:, 3].sum()), start_kwh=float(arr[0, 5]), end_kwh=float(arr[-1, 6]))

def warmup(y, p):
    E=6000.; rows=[]
    net=y[0]-y[1]
    for d in range(31):
        plan=np.zeros(144) if d==0 else np.maximum(net[d-1]*DT, 0)
        a=b.execute(net[d], p, plan, E, None, None, greedy=True); rows.append(a); E=float(a[-1, 6])
    return np.asarray(rows)

def date(d):
    return (datetime(2025, 1, 1)+timedelta(days=int(d))).strftime('%Y-%m-%d')
代码/实验/q3_revision_a_20260912/run_q3.py

"""Run the approved Q3 A model, preserving all earlier Q1/Q2 files."""
from pathlib import Path
import argparse
import json
import sys
import time
import csv
from datetime import datetime
import numpy as np
import q3_engine as m
from check_engine import checks

POLICIES=['mask_000', 'mask_001', 'mask_010', 'mask_011', 'mask_100', 'mask_101', 'mask_110', 'mask_111', 'forecast_on
ly
    ']

def dump(path, value):
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding='utf-8')

def main():
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--output', type=Path, default=m.BASE/'results')
    ap.add_argument('--end-day', type=int, default=365, help='Exclusive day index, 365 means the full year')
    ap.add_argument('--resume', action='store_true')
    args=ap.parse_args(); out=args.output.resolve()
    assert out.is_relative_to(m.ROOT) and 32<=args.end_day<=365
    if out.exists() and any(out.iterdir()) and not args.resume:raise RuntimeError('Output exists; use a new
dire
    ctory or --resume for the same source')
    out.mkdir(parents=True, exist_ok=True)
    sources={str(m.BASE/n):m.sha(m.BASE/n) for n in ['q3_engine.py', 'check_engine.py', 'run_q3.py']}
    protected=[m.ROOT/'基础数据'/n for n in ['C题.pdf', '附件1.xlsx', '附件2.xlsx', '附件3.xlsx', 'result3.xlsx']]
    protected += [m.Q2/n for n in ['fresh_engine.py', 'run_fresh.py', 'results/fresh_forecasts.npz', 'results/man
if
    est.json', 'results/result2.xlsx', 'results/main.npy']]
    input_hashes={str(p):m.sha(p) for p in protected}
    signature=dict(sources=sources, inputs=input_hashes, end_day=args.end_day, policies=POLICIES,
        settlement='incremental revisions; refund old base price plus 50% penalty; 150% upward
price'
    ,
        dp_terminal=0, grid_kwh=60, atoms=7, quantiles=m.QUANTILES.tolist(),
        load_forecast='read-only hash-verified Q2 fresh_forecasts separate[:, 0]',
        reference_Q2_source_hash_rule='exact bytes or exact CRLF-to-LF normalization')
    start_time=time.perf_counter()
    if args.resume:

```

```

        assert json.loads((out/'signature.json').read_text(encoding='utf-8'))==signature,'Resume source/input
mi
    smatch'
    else:dump(out/'signature.json',signature)
    print(datetime.now().isoformat(timespec='seconds'),'loading and conditioning forecasts',flush=True)
    y,p,load,f=m.load_inputs();pred,pv=m.conditional_forecasts(y,load,f);warm=m.warmup(y,p)
    if not args.resume:
        result=checks(y,p,load,f,pred);dump(out/'engine_checks.json',result)
        np.savez_compressed(out/'conditional_forecasts.npz',pred=pred,pv=pv,load=load)
        np.save(out/'january_warmup.npy',warm)
    D=args.end_day-31
    arrays=np.zeros((9,D,144,7));initials=np.zeros((9,D,144));E=np.full(9,warm[-1,-1,6]);ledger=[];scores=[];d
ai
    ly=[];first=31
    if args.resume:
        with np.load(out/'checkpoint.npz') as z:
            arrays[:]=z['arrays'];initials[:]=z['initials'];E[:]=z['states'];first=int(z['next_day'])
            ledger=z['ledger'].tolist()
        daily=json.loads((out/'daily_checkpoint.json').read_text(encoding='utf-8'))
        scores=json.loads((out/'scores_checkpoint.json').read_text(encoding='utf-8'))
    for d in range(first,args.end_day):
        models=[m.model_day(y,pred,d,k) for k in range(4)]
        for a,name in enumerate(POLICIES):
            run=m.day_run(y,p,pred,models,d,E[a],mask=min(a,7),update_only=a==8,return_scores=a==7)
            arrays[a,d-31]=run['arr'];initials[a,d-31]=run['initial'];E[a]=run['arr'][-1,6]
            ledger.extend([[a,d,*r] for r in run['ledger'].tolist()])
            scores.extend([[d,*r] for r in run['scores']])
            daily.append(dict(policy=name,date=m.date(d),**m.summary_day(run,p)))
        if (d-31)%15==0 or d==args.end_day-1:
            print(datetime.now().isoformat(timespec='seconds'),m.date(d),'main
cumulative',round(sum(r['total_co
st'] for r in daily if r['policy']=='mask_111'),2),'elapsed',round(time.perf_counter()-start_time,1),flush
=T
rue)
        if (d-30)%30==0 or d==args.end_day-1:
            np.savez(out/'checkpoint.npz',arrays=arrays,initials=initials,states=E,next_day=d+1,ledger=np.asarray(
ra
y(ledger,float).reshape(-1,11))
            dump(out/'daily_checkpoint.json',daily);dump(out/'scores_checkpoint.json',scores)
            physical=[name:m.b.validate(arrays[a].reshape(-1,7),(y[0]-y[1])[31:args.end_day].ravel(),np.tile(p,D))
for a
            ,name in enumerate(POLICIES)}
            summary={}
            for a,name in enumerate(POLICIES):
                rr=[r for r in daily if r['policy']==name]
                summ={key:float(sum(r[key] for r in rr)) for key in rr[0] if key not in
['policy','date','start_kwh','en
d_kwh']}
                summ.update(start_kwh=float(arrays[a,0,0,5]),end_kwh=float(E[a]),emergency_days=int(np.sum(np.any(arrays
ys
[a,:, :,3]>TOL,axis=1))),
                    emergency_intervals=int(np.sum(arrays[a,:, :,3]>TOL)),days=D)
                assert abs(summ['initial_kwh']+summ['increase_kwh']-summ['cancel_kwh']-summ['effective_kwh'])<1e-5
                summary[name]=summ
            for path,digest in input_hashes.items():assert m.sha(path)==digest,'Input modified: '+path
            np.savez_compressed(out/'solutions.npz',arrays=arrays,initials=initials,ledger=np.asarray(ledger,float).re
sh
ape(-1,11),price=p,day_indices=np.arange(31,args.end_day))
            with (out/'daily_summary.csv').open('w',encoding='utf-8-sig',newline='') as f:
                writer=csv.DictWriter(f,fieldnames=list(daily[0]));writer.writeheader();writer.writerows(daily)
            dump(out/'summary.json',summary)
            with (out/'candidate_scores.csv').open('w',encoding='utf-8-sig',newline='') as f:
                w=csv.writer(f);w.writerow(['day_index','issue_index','candidate_quantile','revision_net_cost','expect
ed
_emergency_cost','planning_score','chosen']);w.writerows(scores)
            metrics={}
            for k in range(4):
                sl=slice(k*36,144);actual=(y[0]-y[1])[31:args.end_day,sl]
                error=actual-pred[31:args.end_day,k,1,sl]

```

```

        metrics[str(k*6)]=dict(MAE_kW=float(abs(error).mean()), RMSE_kW=float(np.sqrt((error**2).mean())),
                                q80_coverage=float(np.mean(actual<=pred[31:args.end_day,k,3,sl])))
    dump(out/'forecast_metrics.json', metrics)
    manifest=dict(status='computed_pending_delivery_checks_and_user_review', created=datetime.now().isoformat(),
                  python=sys.version, numpy=np.__version__, scipy=m.b.scipy.__version__, lightgbm=m.b.lgb.__versi
on
    __, openpyxl=m.openpyxl.__version__,
                  elapsed_seconds=time.perf_counter()-start_time, signature=signature, physical_checks=physical,
ch
                  reused_Q2_forecast_files=l, new_forecast_model_fits=0, new_Q3_dispatch=True, protected_files_un
anged=True,
                  command=[sys.executable, str(Path(__file__).relative_to(m.ROOT)), *sys.argv[1:]],
                  output_hashes={p.name:m.sha(p) for p in out.iterdir() if p.is_file() and p.name not in
['manif
est.json', 'checkpoint.npz', 'daily_checkpoint.json', 'scores_checkpoint.json']})
    dump(out/'manifest.json', manifest)
    print(' COMPLETE', json.dumps(summary, ensure_ascii=False), flush=True)

```

```

TOL=m.TOL
if __name__=='__main__':main()

```

代码/实验/q4_independent_codex_20260912/audit.py

```

"""Independent accounting/physics/Excel verification, no solver or cost function calls."""
from pathlib import Path
from datetime import datetime, timedelta
import csv
import hashlib
import json
import sys
import numpy as np
BASE=Path(__file__).resolve().parent; ROOT=BASE.parents[2]; OUT=BASE/'results'; DATA=ROOT/'基础数据'
sys.path.append(str(Path.home()/'/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/Lib/site-pack
ag
es'))
import openpyxl

def sha(p): return hashlib.sha256(p.read_bytes()).hexdigest()

def style(cell):
    # openpyxl serializes an untouched blank cell as None and a populated
    # default cell as nine zero style indices. These are visually identical.
    return tuple(cell._style) if cell._style is not None else (0,)*9

def main():
    records={}
    def check(name, ok, detail):
        records[name]=dict(passed=bool(ok), detail=detail)
        print(f'{"PASS" if ok else "FAIL"} {name}: {detail}', flush=True)
    raw=[]
    for name, sheets in [(('附件2.xlsx', ['小区负载', '光伏发电实际功率']), ('附件4.xlsx', ['Sheet1']))]:
        wb=openpyxl.load_workbook(DATA/name, read_only=True, data_only=True)
        for sn in sheets: raw.append(np.array([r[1:] for r in list(wb[sn].values)[1:]), float))
        wb.close()
    net=(raw[0]-raw[1])/6; prices=raw[2]
    annual=json.loads((OUT/'annual.json').read_text(encoding='utf-8'))['strategies']
    with np.load(OUT/'trajectories.npz') as z: paths={n:z[n] for n in z.files}
    with np.load(OUT/'transactions.npz') as z: tx={n:z[n] for n in z.files}
    with np.load(OUT/'plan_snapshots.npz') as z: snaps={n:z[n] for n in z.files}
    daily={}
    with (OUT/'daily_costs.csv').open(encoding='utf-8-sig') as f:
        for r in csv.DictReader(f):
            key=(r['strategy'], int(r['day'])); assert key not in daily; daily[key]=r
    warm=np.load(OUT/'warmup.npy')
    check('warmup_chain', warm.shape==(31, 144, 11) and warm[0, 0, 6]==6000 and
np.array_equal(warm.reshape(-1, 11)[1:
, 6], warm.reshape(-1, 11)[:1, 7]), float(warm[-1, -1, 7]))
    for name, r in paths.items():

```

```

        check(name+'_shape_data', r.shape==(334, 144, 11) and np.allclose(r[:, :, 8], net[31:], atol=1e-10, rtol=0)
and
        np.array_equal(r[:, :, 9], prices[31:]), list(r.shape))
        balance=float(np.max(np.abs(r[:, :, 1]+r[:, :, 4]+r[:, :, 3]-net[31:]-r[:, :, 2]-r[:, :, 5])))
        state=float(np.max(np.abs(r[:, :, 7]-r[:, :, 6]-.9*r[:, :, 2]+r[:, :, 3]/.9)))
        check(name+'_physical', max(balance, state)<1e-6, dict(balance=balance, state=state))
        boundary=max(0, 1200-r[:, :, 6:8].min(), r[:, :, 6:8].max()-10800, r[:, :, 2:4].max()-5000/6, -r[:, :, 6].min())
        overlap=max(float(np.minimum(r[:, :, 2], r[:, :, 3]).max()), float(np.minimum(r[:, :, 2], r[:, :, 4]).max()))
        check(name+'_limits', boundary<1e-6 and overlap<1e-6, dict(boundary=float(boundary), overlap=overlap))
        flat=r.reshape(-1, 11)
        check(name+'_continuity', r[0, 0, 6]==warm[-1, -1, 7] and
np.array_equal(flat[1:, 6], flat[:-1, 7]), float(r[0, 0,
        6]))
        transactions=tx[name]; replay=r[:, :, 0].copy(); extra=np.zeros(334); last=(-1, -1, -1); valid=True
        dailyextra=np.zeros((334, 3))
        for d, k, t, old, new, inc, cancel, pred in transactions:
            d, k, t=int(d), int(k), int(t); i=d-31
            valid &= (d, k, t)>=last and 1<=k<=3 and 36*k<=t<144 and abs(old-replay[i, t])<1e-8
            valid &= abs(inc-max(new-old, 0))<1e-10 and abs(cancel-max(old-new, 0))<1e-10
            replay[i, t]=new; p=prices[d, t]
            dailyextra[i]+=[1.5*p*inc, p*cancel, .5*p*cancel]; extra[i]+=p*(1.5*inc-.5*cancel); last=(d, k, t)
        check(name+'_transaction_replay', valid and np.max(np.abs(replay-r[:, :, 1]))<1e-8, int(len(transactions)))
        ini=(r[:, :, 0]*prices[31:]).sum(1); emergency=(r[:, :, 4]*prices[31:]*5).sum(1)
        calculated=ini+extra+emergency; exported=np.array([float(daily[name, d]['total_cost']) for d in
range(31,
        365)])
        err=max(float(np.max(np.abs(calculated-exported))), abs(float(calculated.sum())-annual[name]['total_cos
t'
        ]))
        check(name+'_accounting', err<1e-5, dict(total=float(calculated.sum()), max_error=err))
        spliterr=0.
        for i, d in enumerate(range(31, 365)):
            for key, v in zip(['initial_cost', 'increase_cost', 'refund', 'penalty', 'emergency_cost'], [ini[i], *dai
ly
            extra[i], emergency[i]]):
                spliterr=max(spliterr, abs(v-float(daily[name, d][key])))
            check(name+'_bill_components', spliterr<1e-5, spliterr)
            # Reconstruct every published snapshot, including inactive update times.
            snapshot_ok=True
            for i, d in enumerate(range(31, 365)):
                current=r[:, :, 0].copy()
                for k in range(4):
                    for ev in transactions[(transactions[:, 0]==d)&(transactions[:, 1]==k)]:
current[int(ev[2])]=ev[4]
                    snapshot_ok &= np.array_equal(current, snaps[name][i, k])
                check(name+'_snapshots', snapshot_ok, 'four snapshots per day replayed')
                if name in ['A42', 'B42', 'A43_000', 'A43_update_only']:
                    check(name+'_frozen', len(transactions)==0 and np.array_equal(r[:, :, 0], r[:, :, 1]), 'ordinary plan
fixed
        ')
        # LP outcomes are checked independently from candidate output.
        with (OUT/'candidate_scores.csv').open(encoding='utf-8-sig') as f: candidates=list(csv.DictReader(f))
        check('lp_status', all(int(r['status'])==0 and float(r['residual'])<1e-6 for r in
candidates), len(candidates)
        )
        for tag, name in [('2', 'A42'), ('3', 'A43_111')]:
            wb=openpyxl.load_workbook(OUT/f'result4-{tag}.xlsx', data_only=True)
            tpl=openpyxl.load_workbook(DATA/f'result4-{tag}.xlsx', data_only=True); r=paths[name]
            ok=wb.sheetnames==tpl.sheetnames; err=0.; style_errors=0
            for sn, col in [('计划购电量', 0), ('调整购电量', 1)]:
                if sn not in wb.sheetnames: continue
                ws=wb[sn]; ts=tpl[sn]
                ok &= ws.max_row==335 and ws.max_column==147
                ok &= ws.cell(1, 2).value=='00:00-00:10' and ws.cell(1, 145).value=='23:50-24:00'
                for i in range(334):
                    ok &= ws.cell(i+2, 1).value==datetime(2025, 2, 1)+timedelta(days=i)
                    got=np.array([ws.cell(i+2, c).value for c in range(2, 146)], float)
                    err=max(err, float(np.max(np.abs(got-r[i, :, col]))), abs(ws.cell(i+2, 146).value-r[i, :, col].sum()))

```

```

        dc=daily[name,i+31]; expected=float(dc['initial_cost'])+(float(dc['adjustment_net']) if col
else
    0)
        err=max(err,abs(ws.cell(i+2,147).value-expected))
        for c in range(1,148): style_errors+=int(style(ws.cell(i+2,c))!=style(ts.cell(i+2,c)))
        style_errors+=int(ws.row_dimensions[i+2].height!=ts.row_dimensions[i+2].height)
ws=wb['充放量']; ts=tpl['充放量']; ok &= ws.max_row==2005
for i in range(334):
    for b in range(6):
        row=2+i*6+b; err=max(err,abs(ws.cell(row,3).value-r[i,b*24:(b+1)*24,2].sum()),abs(ws.cell(row,
4)
.value-r[i,b*24:(b+1)*24,3].sum()))
        for c in range(1,7): style_errors+=int(style(ws.cell(row,c))!=style(ts.cell(b+2,c)))
        style_errors+=int(ws.row_dimensions[row].height!=ts.row_dimensions[b+2].height)
        err=max(err,abs(ws.cell(2+i*6,6).value-r[i,0,6]),abs(ws.cell(3+i*6,6).value-r[i,-1,7]))
ws=wb['紧急购电量']; ts=tpl['紧急购电量']; seen={}; current=None; printed=0.
for cells in list(ws.values)[1:]:
    if cells[0] is not None: current=cells[0].strftime('%Y-%m-%d'); seen[current]=[]
    seen[current].append(cells[1:]); printed+=float(cells[2])
row=2
for block in seen.values():
    for j in range(len(block)):
        pattern=0 if j==0 else 2 if j==len(block)-1 else 1
        for c in range(1,4): style_errors+=int(style(ws.cell(row,c))!=style(ts.cell(pattern+2,c)))
        style_errors+=int(ws.row_dimensions[row].height!=ts.row_dimensions[pattern+2].height)
        row+=1
ok &= len(seen)==334
for i in range(334):
    ds=(datetime(2025,2,1)+timedelta(days=i)).strftime('%Y-%m-%d'); reconstructed=np.zeros(144)
    for period,q in seen[ds]:
        if period=='无': ok &= abs(q)<1e-8; continue
        a,b=period.split('-'); aa=sum(x*y for x,y in zip(map(int,a.split(':')), [6,.1])); bb=sum(x*y
for
    x,y in zip(map(int,b.split(':')), [6,.1]))
        aa,bb=int(round(aa)),int(round(bb)); ok &= np.all(r[i,aa:bb,4]>1e-8) and not
reconstructed[aa:bb
].any()
        reconstructed[aa:bb]=1; err=max(err,abs(q-r[i,aa:bb,4].sum()))
        ok &= np.array_equal(reconstructed>0,r[i,:,4]>1e-8)
        err=max(err,abs(printed-r[:, :, 4].sum()))
        for sn in wb.sheetnames:
            for col,dim in tpl[sn].column_dimensions.items():
style_errors+=int(wb[sn].column_dimensions[col].width
dth!=dim.width)
            ok &= str(wb[sn].page_setup)==str(tpl[sn].page_setup)
            ok &= not any(c.data_type=='e' or c.value==' ' for row in wb[sn] for c in row)
            check(name+'_excel_values',ok and err<1e-5,dict(max_error=err,emergency_kwh=printed))
            check(name+'_excel_styles',style_errors==0,style_errors)
            wb.close(); tpl.close()
            with (OUT/f'{name}_intervals.csv').open(encoding='utf-8-sig') as f: csvrows=list(csv.DictReader(f))
            keys=[(rr['date'],rr['slot']) for rr in csvrows]
            check(name+'_csv_keys',len(keys)==48096 and len(set(keys))==48096,len(keys))
            sig=json.loads((OUT/'signature.json').read_text(encoding='utf-8'))
            changed=[p for p,h in sig['inputs'].items() if sha(ROOT/p)!=h]
            check('protected_input_hashes',not changed,changed)
            changed=[p for p,h in sig['sources'].items() if sha(BASE/p)!=h]
            check('solver_source_hashes',not changed,changed)
            state=json.loads((OUT/'checkpoint.json').read_text(encoding='utf-8'))
            bad_days=[d for d,h in state['day_hashes'].items() if sha(OUT/'days'/f'{int(d):03d}.npz')!=h]
            check('checkpoint_completeness',state['next_day']==365 and len(state['day_hashes'])==334 and not
bad_days,bad
            d_days)
            payload=dict(passed=sum(r['passed'] for r in records.values()),total=len(records),checks=records)
            (OUT/'audit.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf-8')
            # Output-only refresh; the auditing calculations above call no solver/cost helper.
            import deliver
            deliver.report(); deliver.manifest()
            print(f'Audit {payload["passed"]}/{payload["total"]}')
            return payload['passed']==payload['total']

```

```

if __name__ == '__main__': sys.exit(0 if main() else 1)
代码/实验/q4_independent_codex_20260912/deliver.py

"""Standalone tables/official-template export from independent run artifacts."""
import argparse
import csv
import json
import sys
from copy import copy
from pathlib import Path
from datetime import datetime
import numpy as np
import model as m

SPEC=['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
OUT=m.BASE/'results'

def write_csv(path, rows):
    with path.open('w', encoding='utf-8-sig', newline='') as f:
        wr=csv.DictWriter(f, fieldnames=list(rows[0])); wr.writeheader(); wr.writerows(rows)

def events(e):
    mask=e>1e-8; starts=np.flatnonzero(mask&~np.r_[False,mask[:-1]])
    ends=np.flatnonzero(mask&~np.r_[mask[1:],False])+1
    return [(int(a),int(b),float(e[a:b].sum())) for a,b in zip(starts,ends)]

def repeat_template_row(ws, target, style, height, values):
    for c,v in enumerate(values,1):
        cell=ws.cell(target,c,v); cell._style=copy(style[c-1])
    ws.row_dimensions[target].height=height

def template(tag, trajectory, daily):
    wb=m.openpyxl.load_workbook(m.DATA/f'result4-{tag}.xlsx')
    for sheet,col in [('计划购电量',0),('调整购电量',1)]:
        if sheet not in wb.sheetnames: continue
        ws=wb[sheet]
        for t in range(144): ws.cell(1,t+2,m.interval(t))
        for i,r in enumerate(trajectory):
            dt=datetime.fromisoformat(m.date(i+31)); d=daily[m.date(i+31)]
            amount=r[:,col]; cost=d['initial_cost']+(d['adjustment_net'] if col==1 else 0)
            vals=[dt,*map(float,amount),float(amount.sum()),cost]
            for j,v in enumerate(vals,1): ws.cell(i+2,j,v)
        ws=wb['充放电量']
        styles=[[copy(ws.cell(r,c)._style) for c in range(1,7)] for r in range(2,8)]
        heights=[ws.row_dimensions[r].height for r in range(2,8)]
        ws.delete_rows(2,ws.max_row-1)
        for i,day in enumerate(trajectory):
            for block in range(6):
                vals=[datetime.fromisoformat(m.date(i+31)) if block==0 else None,
                    m.interval(block*24,(block+1)*24),float(day[block*24:(block+1)*24,2].sum()),
                    float(day[block*24:(block+1)*24,3].sum()),
                    '00:00' if block==0 else '24:00' if block==1 else None,
                    float(day[0,6]) if block==0 else float(day[-1,7]) if block==1 else None]
                repeat_template_row(ws,2+i*6+block,styles[block],heights[block],vals)
        ws=wb['紧急购电量']
        styles=[[copy(ws.cell(r,c)._style) for c in range(1,4)] for r in range(2,5)]
        heights=[ws.row_dimensions[r].height for r in range(2,5)]
        ws.delete_rows(2,ws.max_row-1); row=2
        for i,day in enumerate(trajectory):
            ev=events(day[:,4]); ev=ev or [(None,None,0.)]
            for j,(a,b,q) in enumerate(ev):
                pattern=0 if j==0 else 2 if j==len(ev)-1 else 1
                vals=[datetime.fromisoformat(m.date(i+31)) if j==0 else None,m.interval(a,b) if a is not None
            else ,
            无',q]
            repeat_template_row(ws,row,styles[pattern],heights[pattern],vals); row+=1
    wb.save(OUT/f'result4-{tag}.xlsx'); wb.close()

```

```

def sensitivity():
    with np.load(OUT/'features.npz') as z: f={k:z[k] for k in z.files}
    with np.load(OUT/'trajectories.npz') as z: main=z['A43_111']
    rows=[]
    for ds in SPEC:
        d=(datetime.fromisoformat(ds)-datetime(2025,1,1)).days; energy=float(main[d-31,0,6]); base=None
        for step,atoms in [(60,7),(30,7),(60,15)]:
            r=m.simulate(f,d,energy,grid_step=step,atoms=atoms)
            costs=m.costs(r)
            if base is None: base=costs['total_cost']
            rows.append(dict(date=ds,grid_step=step,atoms=atoms,**costs,delta_cost=costs['total_cost']-base))
            print(ds,step,atoms,round(costs['total_cost'],6),flush=True)
    write_csv(OUT/'sensitivity.csv',rows)

def tables():
    with np.load(OUT/'trajectories.npz') as z: trajectories={n:z[n] for n in ['A42','A43_111']}
    with (OUT/'daily_costs.csv').open(encoding='utf-8-sig') as f:
        daily={}
        for r in csv.DictReader(f): daily.setdefault(r['strategy'],{})[r['date']]={k:float(v) for k,v in
r.items
    () if k not in ['strategy','date']}
        tab1=[];tab2=[];tab3=[]
        for tag,name in [('2','A42'),('3','A43_111')]:
            template(tag,trajectories[name],daily[name])
            for ds in SPEC:
                i=(datetime.fromisoformat(ds)-datetime(2025,2,1)).days; day=trajectories[name][i]
                for hour in [10,12,14,16,18,20]:
                    t=hour*6
                    tab1.append(dict(strategy=name,date=ds,interval=m.interval(t),initial_kwh=day[t,0],ordinary_kwh=
day[t,1],
                                initial_daily_kwh=day[:,0].sum(),ordinary_daily_kwh=day[:,1].sum(),total_cost=daily[name][
ds
                                ]['total_cost']))
                    for block in range(6):
                        tab2.append(dict(strategy=name,date=ds,interval=m.interval(block*24,(block+1)*24),
                            charge_kwh=day[block*24:(block+1)*24,2].sum(),discharge_kwh=day[block*24:(block+1)*24,3].sum
um
                            ()),
                                E_start=day[0,6],E_end=day[-1,7]))
                    for a,b,q in events(day[:,4]) or [(None,None,0.)]:
                        tab3.append(dict(strategy=name,date=ds,interval=m.interval(a,b) if a is not None else '无'
,emerg
                            ency_kwh=q))
                    for n,rows in [(1,tab1),(2,tab2),(3,tab3)]: write_csv(OUT/f'specified_table{n}.csv',rows)
                    # Price prediction diagnostics and information cutoffs are generated from new features.
                    with np.load(OUT/'features.npz') as z:
                        metrics=[]; trace=[]
                        for k in range(4):
                            diff=z['ph'][31:,k,36*k:]-z['price'][31:,36*k:]
                            metrics.append(dict(issue_hour=6*k,mae=float(np.abs(diff).mean()),rmse=float(np.sqrt((diff**2).mea
n(
                            )))))
                        for d in range(31,365): trace.append(dict(date=m.date(d),issue_hour=6*k,first_delivery_slot=36*k,
                            last_observed_slot=36*k-1,history_first_day=max(18,d-30),history_last_day=d-1))
                        write_csv(OUT/'price_metrics.csv',metrics); write_csv(OUT/'information_cutoffs.csv',trace)

def figures():
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.sans-serif':['Microsoft YaHei'],'axes.unicode_minus':False,'font.size':10,'save
fi
    g.dpi':300})
    figdir=m.BASE/'figures'; figdir.mkdir(exist_ok=True)
    annual=json.loads((OUT/'annual.json').read_text(encoding='utf-8'))['strategies']
    with (OUT/'daily_costs.csv').open(encoding='utf-8-sig') as f: daily=list(csv.DictReader(f))
    names=['A42','B42','A43_111','B43']; palette=['#0072B2','#56B4E9','#D55E00','#E69F00']
    fig,ax=plt.subplots(figsize=(7.5,4.3),layout='constrained')

```

```

for n,c,ls in zip(names,palette,['-','-','-','-']):
    vals=np.array([float(r['total_cost']) for r in daily if r['strategy']==n])
    ax.plot(np.arange(1,335),np.cumsum(vals)/1e6,color=c,linestyle=ls,label=n)
ax.set(xlabel='正式期天数',ylabel='累计购电费用 / 百万元'); ax.legend(); ax.grid(alpha=.2)
fig.savefig(figdir+'cumulative_cost.png'); fig.savefig(figdir+'cumulative_cost.svg'); plt.close(fig)
masks=[f'A43_{i:03b}' for i in range(8)]
fig,ax=plt.subplots(figsize=(7.5,4.3),layout='constrained')
ax.bar([n[-3:] for n in masks],[annual[n]['total_cost']/1e6 for n in masks],color='#D55E00' if
n.endswith('
111') else '#0072B2' for n in masks])
ax.set(xlabel='06/12/18 时更新组合（最低位为06时）',ylabel='总购电费用 / 百万元');
ax.grid(axis='y',alpha=.2)
)
fig.savefig(figdir+'mask_costs.png'); fig.savefig(figdir+'mask_costs.svg'); plt.close(fig)
with np.load(OUT+'trajectories.npz') as z: a=z['A43_111']
peak=max(a[(datetime.fromisoformat(ds)-datetime(2025,2,1)).days,:4].max() for ds in SPEC)
fig,axes=plt.subplots(2,2,figsize=(9,6),layout='constrained')
for ds,ax in zip(SPEC,axes.ravel()):
    i=(datetime.fromisoformat(ds)-datetime(2025,2,1)).days; day=a[i]
    ax.plot(np.arange(145)/6,np.r_[day[0,6],day[:,7]],color='#0072B2',label=ds)
    ax2=ax.twinx(); plotted=np.where(day[:,4]>1e-8,day[:,4],0)
    ax2.bar((np.arange(144)+.5)/6,plotted,width=1/6,color='#D55E00',alpha=.4)
    ax2.set_ylim(0,max(1,float(peak)*1.1))
    ax.set(xlabel='时刻 / h',ylabel='储电量 / kWh',xlim=(0,24),ylim=(1000,11000)); ax2.set_ylabel('紧急购
电
/ kWh')
    ax.legend(fontsize=9,loc='upper right'); ax.grid(alpha=.2)
fig.savefig(figdir+'specified_soc.png'); fig.savefig(figdir+'specified_soc.svg'); plt.close(fig)

def report():
    a=json.loads((OUT+'annual.json').read_text(encoding='utf-8'));
    checks=json.loads((OUT+'audit.json').read_text
    t(encoding='utf-8')) if (OUT+'audit.json').exists() else None
    unit=json.loads((m.BASE+'tests/unit_checks.json').read_text(encoding='utf-8'))
    s=a['strategies']; main=s['A43_111']; fixed=s['A42']; upd=s['A43_update_only']
    lines=[ '# 第四问 A: 独立实现计算报告',
    '依据已确认的《第四问A 编程实施方案》独立编写; 本轮未重新读取、复制、导入或执行 DS 的第四问代码。',
    '只复用经哈希核验的 Q2/Q3 预测缓存; 价格预测、联合误差模型、LP、DP、所有第四问决策和账本均重新计算。新
预
测模型训练次数为 0。',
    '本目录结果供用户审核, 尚未冻结为论文结论。',
    f'正式评价: {a["days"]} 天, 每策略 {a["days"]*144} 个区间; 1 月从 6000 kWh 连续暖启动。',
    '原始 00:10 列按已确认约定代表 00:00--00:10, 输出模板对应改为开始--结束时段。',
    '| 策略 | 初始计划费/元 | 调整净费/元 | 紧急费/元 | 总费/元 | 紧急电量/kWh | 期末库存/kWh |',
    '|---|---|---|---|---|---|---|']
    for n,v in s.items():
        lines.append(f'| {n} | {v["initial_cost"]:.2f} | {v["adjustment_net"]:.2f} |
{v["emergency_cost"]:.2f}
| | {v["total_cost"]:.2f} | {v["emergency_kwh"]:.2f} | {v["E_end"]:.2f} |')
        lines+=['',f'4-2 主结果为 {fixed["total_cost"]:.2f} 元; 4-3 主结果为 {main["total_cost"]:.2f} 元。',
        f'在同样更新预测的条件下, 开放调整相对 update_only 减少费用
{upd["total_cost"]-main["total_cost"]:.2f}
元 ( {100*(upd["total_cost"]-main["total_cost"])/upd["total_cost"]:.3f}% ) 。',
        f'八种组合中本次最低为 {min((n for n in s if n.startswith("A43_") and n[-3:].isdigit()),key=lambda
n:s[n
]["total_cost"])}. 主结果事先指定 111, 未按全年结果回选。',
        f'单元与因果检查 {unit["passed"]}/{unit["total"]} 通过。',
        f'独立全年与交付审核: {checks["passed"]}/{checks["total"]} 通过。' if checks else '独立全年与交付审核
:
待执行。',
        '费用使用交付区间真实价; 调整逐笔计算增购1.5倍、退款1倍与违约0.5倍, 紧急购电5倍。调整表全天购电费含初
始
计划费与调整净费, 不含紧急费; 不得再加一次计划表费用。',
        '所有 LP 为预测候选生成器, 实际电池动作由反馈 DP 执行; 无强制日未回到6000条件。',
        '局限: 80分位 LP、60 kWh 网络、7个历史代表及零终端价值均为固定近似; 当前窗口不优化未来尚未到达的发布和
调
整机会。A/B 同时改变价格结构与状态近似, 不能把全部差值归因于相关性。各策略库存不同, 费用比较应结合期末库存。',
        '四日敏感性明细见 results/sensitivity.csv, 只反映四日局部变化, 不能据此断言全年精度。',

```

```

    '图1: 主策略累计费用 (figures/cumulative_cost.png)。图2: 八组合总费用 (figures/mask_costs.png)。图3
:
    指定日期SOC与紧急购电 (figures/specified_soc.png)。',
    '指定日期表1/2/3分别见 results/specified_table1.csv、specified_table2.csv、specified_table3.csv。',
    '静态重计价对照见 results/static_repricing.csv; 它保留 Q2/Q3 原有数量, 不属于第四问重新优化。',' ',
    '复现顺序: python -X utf8 test_model.py → python -X utf8 run.py --jobs 9 → python -X utf8
deliver.py -
    -sensitivity → python -X utf8 audit.py。',
    '全新运行要求 results 不存在或为空; 中断时使用 --resume, 仅接受输入/源码/参数/策略签名一致的完整检查点。'
    ']
    warm=np.load(OUT/'warmup.npy')
    lines+=['',f'1 月暖启动实际费用 {np.sum(warm[:, :, 9]*(warm[:, :, 0]+5*warm[:, :, 4])):.2f} 元, 单独记录, 不计
入
    上述 2—12 月费用。']
    recovery=m.BASE/'tests/recovery_checks.json'
    if recovery.exists():
        rr=json.loads(recovery.read_text(encoding='utf-8'))
        lines.append(f'运行恢复与输出保护检查 {rr["passed"]}/{rr["total"]} 通过; 实际续跑前两天轨迹与连续运行
逐
        数组一致。')
        q3=json.loads((m.Q3/'summary.json').read_text(encoding='utf-8'))
        rank3=sorted([n for n in q3 if n.startswith('mask_')],key=lambda n:q3[n]['total_cost'])
        rank4=sorted([n for n in s if n.startswith('A43_') and n[-3:].isdigit()],key=lambda n:s[n]['total_cost'])
        lines+=['', '第三问组合排序: '+', '.join(n[-3:] for n in rank3)+'。',
        '第四问组合排序: '+', '.join(n[-3:] for n in rank4)+'。仅比较排序, 两问费用结算价不同。',
        'adjust_only 为扩展诊断: 不更新预测时, 将原模型的状态先验沿转移矩阵推进到当前窗口, 再评价剩余计划
;
        它不是必跑的4-3八组合之一。']
        lines.append('SOC图中紧急购电小于等于1e-8 kWh的浮点噪声仅在绘图时隐藏, 四幅图采用相同紧急电量纵轴; 原始数
量
        与账本未舍弃。')
        (m.BASE/'独立计算报告.md').write_text('\n'.join(lines)+'\n',encoding='utf-8')

def manifest():
    outputs={str(p.relative_to(m.BASE)):m.digest(p) for folder in [OUT,m.BASE/'figures',m.BASE/'tests'] for p
in
        folder.glob('*') if p.is_file() and p.name not in ['manifest.json','复现清单.json']}
    outputs['独立计算报告.md']=m.digest(m.BASE/'独立计算报告.md')
    data=dict(inputs=m.provenance(), sources={p.name:m.digest(p) for p in m.BASE.glob('*.*')}, outputs=outputs,
        parameters=m.PARAMS, reused_forecasts=True, new_model_fits=0, independent_from_prior_q4=True,
        seed=20260912, seed_purpose='randomized mathematical tests; solver/model compression
deterministic'
    ,
        run=json.loads((OUT/'run_record.json').read_text(encoding='utf-8')), status='computed_pending_use
r_
        review',
        reproduction='python -X utf8 test_model.py; python -X utf8 run.py --jobs 9; python -X utf8
deliver
        .py --sensitivity; python -X utf8 audit.py')
    for n in ['manifest.json','复现清单.json']: (OUT/n).write_text(json.dumps(data, ensure_ascii=False, indent=2)
    ,
        encoding='utf-8')

def main():
    parser=argparse.ArgumentParser(); parser.add_argument('--sensitivity', action='store_true');
args=parser.parse
    e_args()
    assert json.loads((OUT/'annual.json').read_text(encoding='utf-8'))['days']==334
    if args.sensitivity: sensitivity()
    tables(); figures(); report(); manifest()
    print('Exported official templates, figures, tables, report and provenance manifest.')

if __name__=='__main__': main()

```

代码/实验/q4_independent_codex_20260912/inspect_inputs.py

```

"""Read only the official files and Q2/Q3 data interfaces; no Q4 imports."""
import sys
from pathlib import Path

```

```

sys.path.append(str(Path.home()/'/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/Lib/site-pack
ag
es'))
import openpyxl
import numpy as np
from pypdf import PdfReader
root = Path(__file__).resolve().parents[3]
for name in ['附件2.xlsx', '附件3.xlsx', '附件4.xlsx', 'result4-2.xlsx', 'result4-3.xlsx']:
    wb = openpyxl.load_workbook(root/'基础数据'/name, read_only=True, data_only=True)
    print(name)
    for ws in wb:
        print(ws.title, ws.max_row, ws.max_column)
        rows = list(ws.values)
        print([list(r[:8]) for r in rows[:8]])
        print('last:', list(rows[-1][:3]), list(rows[0][-3:]))
    wb.close()
for folder, names in [('q2_fresh_20260912', ['fresh_forecasts.npz', 'main.npy']), ('q3_revision_a_20260912', ['cond
it
ional_forecasts.npz', 'solutions.npz'])]:
    for name in names:
        obj=np.load(root/'代码/实验'/folder/'results'/name)
        print(folder,name, {k:obj[k].shape for k in obj.files} if hasattr(obj,'files') else obj.shape)
for page in PdfReader(root/'基础数据/C题.pdf').pages:
    print(page.extract_text())

```

代码/实验/q4_independent_codex_20260912/model.py

"""Independent implementation of the approved Q4-A mathematical contract.

No imports from earlier Q4/Q2/Q3 implementations. Only forecast data are reused.

All powers are converted to grid-side kWh at the data boundary.

"""

```

from pathlib import Path
from dataclasses import dataclass
from functools import lru_cache
from datetime import datetime, timedelta
import hashlib
import json
import sys
import numpy as np
from scipy.optimize import linprog
from scipy.sparse import lil_matrix

BASE = Path(__file__).resolve().parent
ROOT = BASE.parents[2]
DATA = ROOT/'基础数据'
Q2 = BASE.parent/'q2_fresh_20260912/results'
Q3 = BASE.parent/'q3_revision_a_20260912/results'
sys.path.append(str(Path.home()/'/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/Lib/site-pack
ag
es'))
import openpyxl

LOW, HIGH, RATE, ETA = 1200., 10800., 5000./6., .9
PARAMS = dict(dt=1/6, eta=ETA, low=LOW, high=HIGH, rate=RATE,
              grid_step=60, atoms=7, window=30, history_start=18,
              state_cut=.84, scale_floor=.01, terminal=0, tie_tol=1e-8)
FIELDS = ['initial', 'ordinary', 'charge', 'discharge', 'emergency', 'spill', 'before', 'after', 'net_kwh', 'price', 'is
su
e']

def digest(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def date(day):
    return (datetime(2025, 1, 1)+timedelta(days=int(day))).strftime('%Y-%m-%d')

def interval(t, end=None):
    end = t+1 if end is None else end
    return f'{t//6:02d}:{t%6*10:02d}-{end//6:02d}:{end%6*10:02d}'

```

```

def input_paths():
    return [DATA/n for n in ['C题.pdf', '附件1.xlsx', '附件2.xlsx', '附件3.xlsx', '附件
4.xlsx', 'result4-2.xlsx', 'res
ult4-3.xlsx']] + [
    Q2/'fresh_forecasts.npz', Q2/'manifest.json', Q2/'main.npy',
    Q3/'conditional_forecasts.npz', Q3/'manifest.json', Q3/'solutions.npz', Q3/'summary.json',
    Q2/'main_intervals.csv', Q3/'main_intervals.csv', Q3/'revision_ledger.csv']

def provenance():
    m2=json.loads((Q2/'manifest.json').read_text(encoding='utf-8'))
    m3=json.loads((Q3/'manifest.json').read_text(encoding='utf-8'))
    for name, h in m2['raw_inputs'].items():
        assert digest(DATA/name)==h, ('Q2 raw input mismatch', name)
    for folder, manifest, names in [(Q2, m2, ['fresh_forecasts.npz', 'main.npy', 'main_intervals.csv']),
                                     (Q3, m3, ['conditional_forecasts.npz', 'solutions.npz', 'main_intervals.csv', 're
vi
sion_ledger.csv'])]:
        for n in names:
            assert digest(folder/n)==manifest['output_hashes'][n], ('upstream output mismatch', n)
    upstream={Path(k).name:v for k, v in m3['signature']['inputs'].items()}
    for n in ['附件2.xlsx', '附件3.xlsx']:
        assert digest(DATA/n)==upstream[n], ('Q3 source mismatch', n)
    assert digest(Q2/'fresh_forecasts.npz')==upstream['fresh_forecasts.npz']
    return {str(p.relative_to(ROOT)):digest(p) for p in input_paths()}

def load_data():
    values=[]; labels=None
    for file, sheets in [(('附件2.xlsx', ['小区负载', '光伏发电实际功率']), ('附件4.xlsx', ['Sheet1']))]:
        wb=openpyxl.load_workbook(DATA/file, read_only=True, data_only=True)
        for sn in sheets:
            rows=list(wb[sn].values)
            assert len(rows)==366 and all(len(r)==145 for r in rows)
            assert [r[0] for r in rows[1:]]==[datetime(2025, 1, 1)+timedelta(days=d) for d in range(365)]
            if labels is None: labels=rows[0][1:]
            assert rows[0][1:]==labels
            values.append(np.array([r[1:] for r in rows[1:]], float))
        wb.close()
    assert str(labels[0])=='00:10:00' and labels[-1]=='0:00+1'
    assert all(np.isfinite(v).all() for v in values) and (values[2]>0).all()
    with np.load(Q2/'fresh_forecasts.npz') as z: q42=z['pred'].copy()[ :, None]
    with np.load(Q3/'conditional_forecasts.npz') as z: q43=z['pred'].copy()
    assert q42.shape==(365, 1, 5, 144) and q43.shape==(365, 4, 5, 144)
    assert np.isfinite(q42[18:]).all()
    for k in range(4): assert np.isfinite(q43[18:, k, :, 36*k:]).all()
    return dict(load=values[0], pv=values[1], net=values[0]-values[1], price=values[2], q42=q42, q43=q43)

def price_features(actual):
    """Every cell is produced using only the history available at its issue time."""
    ph=np.full((365, 4, 144), np.nan); sd=ph.copy()
    for d in range(1, 365):
        ph[d, 0]=actual[max(0, d-7):d].mean(axis=0)
        for k in range(1, 4):
            u=36*k
            bias=(actual[d, u-6:u]-ph[d, 0, u-6:u]).mean()
            ph[d, k, u:]=np.maximum(0, ph[d, 0, u:]+bias*np.exp(-np.arange(1, 145-u)/6))
    residual=actual[:, None, :]-ph
    for d in range(1, 365):
        for k in range(4):
            u=36*k
            for t in range(u, 144):
                block=residual[max(1, d-30):d, k, max(u, t-3):min(144, t+4)]
                sd[d, k, t]=max(.01, float(np.sqrt(np.mean(block**2)))) if block.size else .01
    return ph, sd, residual/sd

def features(raw):
    ph, sd, zp=price_features(raw['price'])
    result=dict(raw, ph=ph, sd=sd, zp=zp)
    for src in ['42', '43']:

```

```

        q=raw['q'+src]
        center=q[:, :, 1]; scale=np.maximum((q[:, :, 3]-q[:, :, 0])/1.683, 100)
        result['center'+src]=center; result['scale'+src]=scale
        result['zn'+src]=(raw['net'][:, None, :]-center)/scale
    return result

def category(z):
    return np.searchsorted([-0.84, 0.84], z, side='right')

def compress(points, limit):
    """Stable earliest-ID tie order; original point mass is preserved."""
    unique, first, counts=np.unique(points, axis=0, return_index=True, return_counts=True)
    order=np.argsort(first); unique=unique[order]; counts=counts[order]; first=first[order]
    if len(unique)<=limit:
        return unique, counts/counts.sum(), first
    chosen=[]
    for axis in range(points.shape[1]):
        for index in [int(np.argmax(unique[:, axis])), int(np.argmax(unique[:, axis]))]:
            if index not in chosen: chosen.append(index)
    distance=((unique[:, None, :]-unique[chosen][None, :, :])**2).sum(axis=2).min(axis=1)
    while len(chosen)<limit:
        index=int(np.argmax(distance)); chosen.append(index)
        distance=np.minimum(distance, ((unique-unique[index])**2).sum(axis=1))
    assignment=((unique[:, None, :]-unique[chosen][None, :, :])**2).sum(axis=2).argmin(axis=1)
    mass=np.bincount(assignment, weights=counts, minlength=len(chosen))
    return unique[chosen], mass/mass.sum(), first[chosen]

@dataclass
class Scenario:
    start:int
    transition:np.ndarray
    prior:np.ndarray
    demand:np.ndarray
    price:np.ndarray
    mass:np.ndarray
    mean_price:np.ndarray
    center:np.ndarray
    scale:np.ndarray
    price_center:np.ndarray
    price_scale:np.ndarray
    joint:bool
    fallback:tuple
    representatives:np.ndarray

def scenario(f, d, k, src, joint=True, atoms=7):
    u=36*k; h=144-u; states=9 if joint else 3
    znet=f['zn'+src][max(18, d-30):d, k, u:]
    zprice=f['zp'][max(18, d-30):d, k, u:]
    cats=category(znet)*3+category(zprice) if joint else category(znet)
    points=np.stack([znet, zprice], axis=-1) if joint else znet[..., None]
    flat=points.reshape(-1, points.shape[-1]); codes=cats.ravel()
    nu=np.bincount(codes, minlength=states)/codes.size
    def counts(a, b):
        return np.bincount((a*states+b).ravel(), minlength=states**2).reshape(states, states)
    allcounts=counts(cats[:, :-1], cats[:, 1:])
    allp=(allcounts+nu)/(allcounts.sum(axis=1)[:, None]+1)
    trans=np.empty((h-1, states, states))
    for t in range(h-1):
        lo=max(0, t-3); hi=min(h-1, t+4)
        c=counts(cats[:, lo:hi], cats[:, lo+1:hi+1])
        trans[t]=(c+3*allp)/(c.sum(axis=1)[:, None]+3)
    prior=(np.bincount(cats[:, 0], minlength=states)+3*nu)/(len(cats)+3)
    n=np.zeros((h, states, atoms)); p=n.copy(); mass=n.copy(); ids=np.full(n.shape, -1, int)
    fallback_state=fallback_all=0
    for t in range(h):
        lo=max(0, t-3); hi=min(h, t+4)
        local=np.arange(codes.size).reshape(cats.shape)[:, lo:hi].ravel()
        for s in range(states):
            picked=local[codes[local]==s]

```

```

        if not len(picked):
            fallback_state+=1; picked=np.flatnonzero(codes==s)
        if not len(picked):
            fallback_all+=1; picked=np.arange(codes.size)
        rep,w,first=compress(flat[picked],atoms); m=len(w)
        n[t,s,:m]=(f['center'+src][d,k,u+t]+f['scale'+src][d,k,u+t]*rep[:,0])/6
        p[t,s,:m]=np.maximum(0,f['ph'][d,k,u+t]+f['sd'][d,k,u+t]*rep[:,1]) if joint else f['ph'][d,k,u+t]
        mass[t,s,:m]=w; ids[t,s,:m]=picked[first]
    prob=prior.copy(); mean=[]
    for t in range(h):
        mean.append(np.sum(prob[:,None]*mass[t]*p[t]))
        if t<h-1: prob=prob@trans[t]
    assert np.allclose(mass.sum(2),1) and np.allclose(trans.sum(2),1)
    return Scenario(u,trans,prior,n,p,mass,np.array(mean),f['center'+src][d,k,u:],
        f['scale'+src][d,k,u:],f['ph'][d,k,u:],f['sd'][d,k,u:],joint,
        (fallback_state,fallback_all),ids)

@lru_cache(maxsize=8)
def lp_matrix(h,revision):
    # Interleaved variables per slot: g,c,b,w,E_after[,increase,cancel].
    width=7 if revision else 5
    a=lil_matrix(((3 if revision else 2)*h,h*width))
    bounds=[]
    for t in range(h):
        j=t*width
        a[2*t,j:j+4]=[1,-1,1,-1]
        a[2*t+1,j+1]=-ETA; a[2*t+1,j+2]=1/ETA; a[2*t+1,j+4]=1
        if t: a[2*t+1,j-width+4]=-1
        bounds.extend([(0,None),(0,RATE),(0,RATE),(0,None),(LOW,HIGH)])
        if revision:
            a[2*h+t,j]=1; a[2*h+t,j+5]=-1; a[2*h+t,j+6]=1
            bounds.extend([(0,None),(0,None)])
    return a.tocsr(),bounds,width

def plan(price,net_kwh,energy,old=None):
    h=len(price); revision=old is not None
    a,bounds,width=lp_matrix(h,revision)
    b=np.zeros(a.shape[0]); b[:2*h:2]=net_kwh; b[1]=energy
    c=np.zeros(h*width)
    if revision:
        b[2*h:]=old; c[5::width]=1.5*price; c[6::width]=-0.5*price
    else: c[:width]=price
    assert np.isfinite(b).all() and np.isfinite(c).all()
    fit=linprog(c,A_eq=a,b_eq=b,bounds=bounds,method='highs')
    if not fit.success: raise RuntimeError(f'LP status {fit.status}: {fit.message}')
    residual=float(np.abs(a@fit.x-b).max())
    assert residual<1e-6
    return np.maximum(0,fit.x[:width]), dict(status=int(fit.status),residual=residual,objective=float(fit.fun))

)

def action(energy,gap,price,future,grid):
    """Scalar reference action; continuous energy, endpoints + interior knots."""
    if gap<=0:
        c=min(-gap,RATE,(HIGH-energy)/ETA); en=energy+ETA*c
        return np.array([c,0,0,-gap-c,en]),float(np.interp(en,grid,future))
    lower=max(LOW,energy-min(gap,RATE)/ETA)
    nodes=np.r_[lower,grid[(grid>lower)&(grid<energy)],energy]
    bills=5*price*(gap-ETA*(energy-nodes))+np.interp(nodes,grid,future)
    index=int(np.flatnonzero(bills<=bills.min()+1e-8)[0])
    en=nodes[index]; b=ETA*(energy-en)
    return np.array([0,b,max(0,gap-b),0,en]),float(bills[index])

def interpolate_rows(values,x,grid):
    """values (S,M), x (S,L,M); equally spaced grid."""
    pos=np.clip((x-grid[0])/(grid[1]-grid[0]),0,len(grid)-1)
    ix=np.minimum(pos.astype(int),len(grid)-2); frac=pos-ix
    s=np.arange(len(values))[:,None,None]
    return values[s,ix]*(1-frac)+values[s,ix+1]*frac

```

```

def bellman(g, mdl, grid, scalar=False):
    h, s, l=mdl.mass.shape; assert np.asarray(g).shape==(h,)
    w=np.zeros((h+1, s, len(grid)))
    for t in range(h-1, -1, -1):
        future=mdl.transition[t]*w[t+1] if t<h-1 else w[t+1]
        if scalar:
            for state in range(s):
                for j, e in enumerate(grid):
                    w[t, state, j]=sum(mdl.mass[t, state, a]*action(e, mdl.demand[t, state, a]-g[t],
                        mdl.price[t, state, a], future[state], grid)[1] for a in range(l) if
mdl.mass[t, state, a]>0)
                continue
            gap=mdl.demand[t]-g[t]; pp=mdl.price[t]; e=grid[None, None, :]
            lower=np.maximum(LOW, e-np.minimum(np.maximum(gap, 0), RATE)[:, :, None]/ETA)
            at_lower=interpolate_rows(future, lower, grid)+5*pp[:, :, None]*(gap[:, :, None]-ETA*(e-lower))
            best=5*pp[:, :, None]*gap[:, :, None]+future[:, None, :]
            best=np.minimum(best, at_lower)
            # All internal knots are grid[j-offset]. This shifted reduction avoids M^2 storage.
            reach=np.minimum(np.maximum(gap, 0), RATE)/ETA
            for offset in range(1, min(len(grid), int(np.ceil(RATE/ETA/(grid[1]-grid[0]))+1))):
                delta=offset*(grid[1]-grid[0])
                trial=future[:, None, :-offset]+5*pp[:, :, None]*(gap[:, :, None]-ETA*delta)
                valid=reach>=delta
                best[:, :, offset:]=np.minimum(best[:, :, offset:], np.where(valid[:, :, None], trial, np.inf))
            en=np.minimum(HIGH, e+ETA*np.minimum(np.maximum(-gap, 0), RATE)[:, :, None])
            surplus=interpolate_rows(future, en, grid)
            costs=np.where((gap<=0)[:, :, None], surplus, best)
            w[t]=np.sum(mdl.mass[t, :, :, None]*costs, axis=1)
    return w

def transact(old, new, p):
    return np.sum(p*(1.5*np.maximum(new-old, 0)-.5*np.maximum(old-new, 0)))

def simulate(f, day, energy, src='43', joint=True, version_mask=7, adjust_mask=7, grid_step=60, atoms=7, cache=None):
    grid=np.arange(LOW, HIGH+grid_step/2, grid_step)
    models={} if cache is None else cache
    def get(k):
        key=(src, joint, k, atoms)
        if key not in models: models[key]=scenario(f, day, k, src, joint, atoms)
        return models[key]
    original, initial_lp=plan(f['ph'][day, 0], f['q'+src][day, 0, 3]/6, energy)
    current=original.copy(); mdl=get(0); w=bellman(current, mdl, grid)
    rows=np.zeros((144, len(FIELDS))); snapshots=[]; changed=[]; events=[]; scores=[]
    active=0
    for t in range(144):
        if t and t%36==0:
            k=t//36; bit=1<<(k-1)
            if version_mask&bit:
                active=k; mdl=get(k)
            if adjust_mask&bit:
                # In adj-only diagnostics, crop the old model at the current time.
                local=mdl if mdl.start==t else crop(mdl, t)
                old=current[t:].copy(); proposals=[old]; labels=['keep'];
    infos=[dict(status=0, residual=0, object
        ive=0)]
        for qindex in [1, 2, 3, 4]:
            proposed, info=plan(f['ph'][day, active, t:], f['q'+src][day, active, qindex, t:]/6, energy, old)
            if not any(np.max(np.abs(proposed-x))<=1e-8 for x in proposals):
                proposals.append(proposed); labels.append(str([.2, .5, .65, .8, .9][qindex]));
    infos.append(
        info)
        best=np.inf; selected=0; selected_w=None
        for i, p in enumerate(proposals):
            value=bellman(p, local, grid)
            score=float(sum(local.prior[s]*np.interp(energy, grid, value[0, s]) for s in
range(len(local.pr
ior))))+transact(old, p, local.mean_price))
            scores.append(dict(issue=k, candidate=labels[i], score=score, **infos[i]))
            if score<best-1e-8: best=score; selected=i; selected_w=value

```

```

        new=proposals[selected]; delta=new-old
        for h in np.flatnonzero(delta!=0):
            events.append([k, t+int(h), old[h], new[h], max(delta[h], 0), max(-delta[h], 0), f['ph']][day, activ
e,
t+h]])
            current[t:]=new; mdl=local; w=selected_w
            elif version_mask&bit:
                w=bellman(current[t:], mdl, grid)
            if t%36==0:
                snapshots.append(current.copy()); changed.append(bool(t and
np.max(np.abs(snapshots[-1]-snapshots[-2
]))>1e-8))
                h=t-mdl.start
                # Only here does the controller observe this interval's real price and demand.
                n=f['net'][day, t]/6; p=f['price'][day, t]
                zn=(n*6-mdl.center[h])/mdl.scale[h]; zp=(p-mdl.price_center[h])/mdl.price_scale[h]
                state=int(category(zn)*3+category(zp)) if joint else int(category(zn))
                future=mdl.transition[h, state]*w[h+1] if h<len(mdl.transition) else np.zeros(len(grid))
                control, _=action(energy, n-current[t], p, future, grid)
                c, b, em, spill, en=control
                rows[t]=[original[t], current[t], c, b, em, spill, energy, en, n, p, active]
                energy=en
            events=np.array(events, float).reshape(-1, 7)
            return dict(rows=rows, events=events, snapshots=np.array(snapshots), changed=np.array(changed), scores=scores,
in
        itial_lp=initial_lp)

def crop(mdl, start):
    """Remaining horizon with prior propagated under the unchanged information model."""
    a=start-mdl.start; prior=mdl.prior.copy()
    for t in range(a): prior=prior@mdl.transition[t]
    return Scenario(start, mdl.transition[a:], prior, mdl.demand[a:], mdl.price[a:], mdl.mass[a:],
        mdl.mean_price[a:], mdl.center[a:], mdl.scale[a:], mdl.price_center[a:], mdl.price_scale[a:],
        mdl.joint, mdl.fallback, mdl.representatives[a:])

def costs(result):
    r=result['rows']; ev=result['events']; p=r[:, 9]
    initial=float(p[r[:, 0]]); emergency=float(5*p[r[:, 4]])
    inc=refund=penalty=0.
    if len(ev):
        price=p[ev[:, 1].astype(int)]
        inc=float(1.5*price@ev[:, 4]); refund=float(price@ev[:, 5]); penalty=.5*refund
    return dict(initial_cost=initial, increase_cost=inc, refund=refund, penalty=penalty,
        adjustment_net=inc-refund+penalty, emergency_cost=emergency, total_cost=initial+inc-refund+penalty+emerg
en
cy,
        initial_kwh=float(r[:, 0].sum()), effective_kwh=float(r[:, 1].sum()),
        increase_kwh=float(ev[:, 4].sum()), cancel_kwh=float(ev[:, 5].sum()),
        emergency_kwh=float(r[:, 4].sum()), charge_kwh=float(r[:, 2].sum()), discharge_kwh=float(r[:, 3].sum()),
        spill_kwh=float(r[:, 5].sum()), E_start=float(r[0, 6]), E_end=float(r[-1, 7]))

def warmup(f):
    energy=6000.; allrows=[]
    grid=np.array([LOW, HIGH])
    for d in range(31):
        g=np.maximum(f['net'][d-1]/6, 0) if d else np.zeros(144)
        for t in range(144):
            n=f['net'][d, t]/6; p=f['price'][d, t]
            ctl, _=action(energy, n-g[t], p, np.zeros(2), grid)
            c, b, em, spill, en=ctl
            allrows.append([g[t], g[t], c, b, em, spill, energy, en, n, p, 0]); energy=en
    return np.array(allrows).reshape(31, 144, len(FIELDS))

```

代码/实验/q4_independent_codex_20260912/run.py

```

"""Sequential days, parallel independent strategy groups, atomic day checkpoints."""
import os
for key in ['OPENBLAS_NUM_THREADS', 'OMP_NUM_THREADS', 'MKL_NUM_THREADS']:
    os.environ[key]='1'
import argparse

```

```

import csv
import json
import time
import platform
from concurrent.futures import ProcessPoolExecutor
from datetime import datetime
import numpy as np
import scipy
import model as m
from test_model import physical

STRATEGIES={'A42': ('42', True, 0, 0), 'B42': ('42', False, 0, 0), 'B43': ('43', False, 7, 7),
             **{f'A43_{mask:03b}': ('43', True, mask, mask) for mask in range(8)},
             'A43_update_only': ('43', True, 7, 0), 'A43_adjust_only': ('43', True, 0, 7)}
GROUPS=[['A42'], ['B42'], ['B43'], ['A43_000', 'A43_001'], ['A43_010', 'A43_011'],
         ['A43_100', 'A43_101'], ['A43_110', 'A43_111', 'A43_update_only'], ['A43_adjust_only']]

def signature():
    return dict(inputs=m.provenance(), sources={n:m.digest(m.BASE/n) for n in ['model.py', 'run.py']},
               params=m.PARAMS, strategies={k:list(v) for k,v in STRATEGIES.items()}, period=[31, 365])

def dump(path, obj):
    temp=path.with_suffix(path.suffix+'.tmp')
    temp.write_text(json.dumps(obj, ensure_ascii=False, indent=2), encoding='utf-8'); os.replace(temp, path)

def init_worker(cache):
    global FEATURES
    with np.load(cache) as z: FEATURES={k:z[k] for k in z.files}

def worker(task):
    day, names, energies=task; cache={}; out={}
    for name in names:
        src, joint, vm, am=STRATEGIES[name]
        result=m.simulate(FEATURES, day, energies[name], src, joint, vm, am, cache=cache)
        residual=physical(result['rows'])
        assert max(residual.values())<1e-6, (name, day, residual)
        result['daily']=dict(strategy=name, day=day, date=m.date(day), **m.costs(result))
        result['physical']=residual
        out[name]=result
    diag=[]
    for (src, joint, k, atoms), mdl in cache.items():
        diag.append(dict(day=day, source=src, joint=joint, issue=k, atoms=atoms,
                        fallback_state=mdl.fallback[0], fallback_all=mdl.fallback[1]))
    return out, diag

def pack_day(path, results, diagnostics):
    data={}; info={}
    for name, r in results.items():
        for field in ['rows', 'events', 'snapshots', 'changed']: data[name+'_'+field]=r[field]
        info[name]={k:r[k] for k in ['daily', 'physical', 'scores', 'initial_lp']}
    data['metadata']=np.array(json.dumps(dict(strategies=info, diagnostics=diagnostics)))
    temp=path.with_suffix('.tmp')
    with temp.open('wb') as f: np.savez_compressed(f, **data)
    os.replace(temp, path)

def csv_write(path, rows):
    rows=list(rows)
    if not rows: return
    with path.open('w', encoding='utf-8-sig', newline='') as f:
        writer=csv.DictWriter(f, fieldnames=list(rows[0])); writer.writeheader(); writer.writerows(rows)

def collect(out, end):
    daily=[]; scores=[]; diag={}; trajectories={n:[] for n in STRATEGIES}; all_ev={n:[] for n in STRATEGIES}
    snapshots={n:[] for n in STRATEGIES}; maxphys={n:{} for n in STRATEGIES}
    for d in range(31, end):
        with np.load(out/'days'/f'{d:03d}.npz') as z:
            meta=json.loads(str(z['metadata']))
            for r in meta['diagnostics']: diag[(d, r['source'], r['joint'], r['issue'])]=r
            for name in STRATEGIES:
```

```

        data=meta['strategies'][name]; daily.append(data['daily'])
        scores.append(dict(strategy=name, day=d, issue=0, candidate='initial', score=None, **data['initial_
lp    ']))

        scores.extend(dict(strategy=name, day=d, **r) for r in data['scores'])
        for k, v in data['physical'].items(): maxphys[name][k]=max(v, maxphys[name].get(k, 0))
        trajectories[name].append(z[name+'__rows']); snapshots[name].append(z[name+'__snapshots'])
        ev=z[name+'__events']
        all_ev[name].append(np.column_stack([np.full(len(ev), d), ev]))
    arrays={n:np.array(v) for n, v in trajectories.items()}
    np.savez_compressed(out/'trajectories.npz', **arrays)
    np.savez_compressed(out/'plan_snapshots.npz', **{n:np.array(v) for n, v in snapshots.items()})
    np.savez_compressed(out/'transactions.npz', **{n:np.concatenate(v) for n, v in all_ev.items()})
    csv_write(out/'daily_costs.csv', daily); csv_write(out/'candidate_scores.csv', scores)
    csv_write(out/'scenario_diagnostics.csv', diag.values())
    annual={}
    for name in STRATEGIES:
        rows=[r for r in daily if r['strategy']==name]
        annual[name]={k:sum(r[k] for r in rows) for k in rows[0] if k not in
['strategy', 'day', 'date', 'E_start',
    'E_end']}
        annual[name].update(E_start=rows[0]['E_start'], E_end=rows[-1]['E_end'], emergency_days=sum(r['emergency
_k    wh']>1e-8 for r in rows))
        r=arrays[name]
        assert np.array_equal(r[1:, 0, 6], r[: -1, -1, 7])
        dump(out/'annual.json', dict(days=end-31, strategies=annual, physical=maxphys, columns=m.FIELDS))
        for name in ['A42', 'A43_111']:
            a=arrays[name]; rows=[]; ledger=[]
            for dd, rday in enumerate(a):
                for t, row in enumerate(rday):
                    rows.append(dict(date=m.date(dd+31), slot=t, interval=m.interval(t), **dict(zip(m.FIELDS, map(floa
t,
                    row))), net_kw=float(row[8]*6)))
            for d, k, t, old, new, inc, cancel, forecast in np.concatenate(all_ev[name]):
                price=arrays[name][int(d)-31, int(t), 9]
                ledger.append(dict(date=m.date(d), issue_hour=int(k)*6, slot=int(t), old=old, new=new, increase=inc, can
ce    l=cancel,
                                forecast_price=forecast, actual_price=price, increase_cost=1.5*price*inc, refund=p
ri    ce*cancel,
                                penalty=.5*price*cancel, adjustment_net=price*(1.5*inc-.5*cancel)))
            csv_write(out/f' {name}_intervals.csv', rows); csv_write(out/f' {name}_ledger.csv', ledger)
        return annual

def static_repricing(out, price):
    results=[]
    for src in ['2', '3']:
        folder=m.Q2 if src=='2' else m.Q3
        with (folder/'main_intervals.csv').open(encoding='utf-8-sig') as f: rows=list(csv.DictReader(f))
        p=price[31:].ravel(); assert len(rows)==334*144
        g=np.array([float(r['g_kwh'] if src=='2' else 'initial_kwh') for r in rows])
        em=np.array([float(r['emergency_kwh']) for r in rows]); adj=0.
        for i, r in enumerate(rows):
            assert r['date']==m.date(31+i//144) and int(r['slot'])==i%144+1
            ordinary=float(r['g_kwh'] if src=='2' else 'effective_kwh')
            balance=ordinary+float(r['emergency_kwh'])+float(r['discharge_kwh'])-float(r['net_kwh'])-float(r['
ch    arge_kwh'])-float(r['waste_kwh'] if src=='2' else 'spill_kwh')
            assert abs(balance)<1e-6
        if src=='3':
            with (folder/'revision_ledger.csv').open(encoding='utf-8-sig') as f:
                for r in csv.DictReader(f):
                    if r['policy']!='mask_111': continue
                    d=(datetime.fromisoformat(r['date'])-datetime(2025, 1, 1)).days
                    adj+=price[d, int(r['slot'])-1]*(1.5*float(r['increase_kwh'])-.5*float(r['cancel_kwh']))
            results.append(dict(strategy='static_Q'+src, initial_cost=float(p@g), adjustment_net=adj,
                                emergency_cost=float(5*p@em), total_cost=float(p@g+adj+5*p@em),

```

```

        E_end=float(rows[-1]['end_kwh']))
    csv_write(out/'static_repricing.csv',results)

def main():
    parser=argparse.ArgumentParser(); parser.add_argument('--resume',action='store_true')
    parser.add_argument('--days',type=int,default=334); parser.add_argument('--jobs',type=int,default=9)
    parser.add_argument('--output',default='results'); args=parser.parse_args()
    assert 1<=args.days<=334 and 1<=args.jobs<=16
    out=(m.BASE/args.output).resolve(); assert out.is_relative_to(m.BASE)
    start=time.perf_counter(); sign=signature(); begin=31
    if args.resume:
        assert json.loads((out/'signature.json').read_text(encoding='utf-8'))==sign,'signature mismatch'
        state=json.loads((out/'checkpoint.json').read_text(encoding='utf-8')); begin=state['next_day'];
    energies
    =state['energies']
    assert 31<=begin<=365 and set(energies)==set(STRATEGIES)
    assert all(m.LOW-1e-6<=v<=m.HIGH+1e-6 for v in energies.values())
    for d in range(31,begin):
        path=out/'days'/f'{d:03d}.npz'; assert m.digest(path)==state['day_hashes'][str(d)]
    if begin>31:
        with np.load(out/'days'/f'{begin-1:03d}.npz') as z:
            assert all(energies[n]==float(z[n+'__rows'][-1,7]) for n in STRATEGIES)
        # A complete day written just before a checkpoint interruption may exist; never overwrite silently.
        assert not (out/'days'/f'{begin:03d}.npz').exists(),'orphan day: inspect before recovery'
        init_worker(out/'features.npz')
    else:
        if out.exists() and any(out.iterdir()): raise RuntimeError('output not empty: choose a new directory
or
--resume')
        out.mkdir(parents=True,exist_ok=True); (out/'days').mkdir()
        f=m.features(m.load_data()); np.savez_compressed(out/'features.npz',**f)
        warm=m.warmup(f); np.save(out/'warmup.npy',warm)
        energies={n:float(warm[-1,-1,7]) for n in STRATEGIES}
        state=dict(next_day=31,energies=energies,day_hashes={})
        dump(out/'signature.json',sign); dump(out/'checkpoint.json',state)
        init_worker(out/'features.npz')
    end=31+args.days; assert begin<=end
    started=datetime.now().isoformat()
    print(f'Independent run: {len(STRATEGIES)} strategies, days {begin}..{end-1},
workers={args.jobs}',flush=True
e)
with ProcessPoolExecutor(max_workers=args.jobs,initializer=init_worker,initargs=(out/'features.npz',)) as
po
ol:
    for d in range(begin,end):
        futures=[pool.submit(worker,(d,g,energies)) for g in GROUPS]; results={}; diag=[]
        for future in futures:
            r,extra=future.result(); results.update(r); diag.extend(extra)
        pack_day(out/'days'/f'{d:03d}.npz',results,diag)
        energies={n:float(r['rows'][-1,7]) for n,r in results.items()}
        state.update(next_day=d+1,energies=energies);
state['day_hashes'][str(d)]=m.digest(out/'days'/f'{d:0
3d}.npz')
        dump(out/'checkpoint.json',state)
        if (d-begin)%10==0 or d==end-1:
            elapsed=time.perf_counter()-start
            print(f'{m.date(d)} {d-30}/{args.days} elapsed={elapsed:.1f}s',flush=True)
        annual=collect(out,end)
        if end==365: static_repricing(out,FEATURES['price'])
        dump(out/'run_record.json',dict(started=started,ended=datetime.now().isoformat(),elapsed=time.perf_counter
))
    -start,
        command=os.sys.argv,exit_code=0,python=os.sys.version,numpy=np.__version__,scipy=scipy.__version__,
        platform=platform.platform(),resumed=args.resume,days_computed=end-begin,random_seed=None,model_fits=0)
    )
    assert m.provenance()==sign['inputs'],'protected inputs changed'
    for n,r in annual.items(): print(f'{n}: {r["total_cost"]:.2f} yuan; emergency={r["emergency_kwh"]:.2f}
kWh
',flush=True)

```

```
if __name__ == '__main__': main()
```

代码/实验/q4_independent_codex_20260912/test_model.py

```
"""Executable mathematical tests, written independently of earlier Q4 tests."""
```

```
import json
```

```
import time
```

```
import numpy as np
```

```
import model as m
```

```
def physical(rows):
```

```
    r=rows.reshape(-1, len(m.FIELDS)); g, c, b, e, w, E, En, n=r[:, 1], r[:, 2], r[:, 3], r[:, 4], r[:, 5], r[:, 6], r[:, 7], r[:, 8]
```

```
    return dict(balance=float(np.abs(g+e+b-n-c-w).max()),
```

```
                recurrence=float(np.abs(En-E-.9*c+b/.9).max()),
```

```
                boundary=float(max(0, m.LOW-E.min(), m.LOW-En.min(), E.max()-m.HIGH, En.max()-m.HIGH)),
```

```
                power=float(max(0, c.max()-m.RATE, b.max()-m.RATE)),
```

```
                nonnegative=float(max(0, -min(g.min(), c.min(), b.min(), e.min(), w.min()))),
```

```
                overlap=float(np.minimum(c, b).max()), emergency_charging=float(np.minimum(c, e).max()))
```

```
def synthetic(demand, price, mass, transition=None, prior=None):
```

```
    h, s, l=demand.shape
```

```
    if transition is None: transition=np.full((h-1, s, s), 1/s)
```

```
    if prior is None: prior=np.full(s, 1/s)
```

```
    return m.Scenario(0, transition, prior, demand, price, mass, np.zeros(h), np.zeros(h), np.ones(h),
                      np.zeros(h), np.ones(h), True, (0, 0), np.zeros((h, s, l), int))
```

```
def main():
```

```
    started=time.perf_counter(); reports={}
```

```
    def check(label, ok, detail):
```

```
        reports[label]=dict(passed=bool(ok), detail=detail)
```

```
        print(f'{"PASS" if ok else "FAIL"} {label}: {detail}', flush=True)
```

```
        if not ok: raise AssertionError(label)
```

```
    provenance=m.provenance(); raw=m.load_data(); f=m.features(raw)
```

```
    check('input_contract', raw['net'].shape==(365, 144) and
```

```
raw['price'].min()>0, dict(price_min=float(raw['price']
    ).min()), protected_files=len(provenance)))
```

```
    # Hand-derived financial and optimal action cases.
```

```
    bill=200+m.transact(np.array([100.]), np.array([80.]), np.array([2.]))+m.transact(np.array([80.]), np.array([
```

```
10
```

```
    0.]), np.array([2.]))
```

```
    check('round_trip_settlement', abs(bill-240)<1e-10, bill)
```

```
    mdl=synthetic(np.array([[0., 100.]]), np.array([[1., 3.]]), np.array([[.5, .5]]))
```

```
    grid=np.arange(m.LOW, m.HIGH+1, 60.)
```

```
    val=m.bellman(np.array([50.]), mdl, grid)[0, 0, 0]
```

```
    check('joint_price_demand', abs(val-375)<1e-9, float(val))
```

```
    low, _=m.action(2000, 100, 1., -2*grid, grid); high, _=m.action(2000, 100, 1., -2*grid, grid)
```

```
    check('observed_price_response', abs(low[2]-100)<1e-8 and
```

```
abs(high[1]-100)<1e-8, dict(low=low.tolist(), high=hi
```

```
gh.tolist()))
```

```
    # Randomized Bellman comparisons against scalar exhaustive knot enumeration.
```

```
    rng=np.random.default_rng(20260912)
```

```
    n=rng.uniform(-600, 1600, (4, 3, 4)); p=rng.uniform(0, 1.7, n.shape); wt=rng.uniform(.1, 1, n.shape);
```

```
wt/=wt.sum(2, k
```

```
    keepdims=True)
```

```
    trans=rng.uniform(.01, 1, (3, 3, 3)); trans/=trans.sum(2, keepdims=True)
```

```
    random=synthetic(n, p, wt, trans); g=rng.uniform(0, 800, 4)
```

```
    fast=m.bellman(g, random, grid); slow=m.bellman(g, random, grid, scalar=True)
```

```
    delta=float(np.abs(fast-slow).max())
```

```
    check('bellman_all_knots', delta<1e-7, delta)
```

```
    actions=[]
```

```
    for e in [1200., 2000.3, 10799.9, 10800.]:
```

```
        for gap in [-1500, -2, 0, 100, 1500]:
```

```
            a, _=m.action(e, gap, .7, fast[1, 1], grid)
```

```
            r=np.array([0, 1000, a[0], a[1], a[2], a[3], e, a[4], 1000+gap, .7, 0])
```

```
            actions.append(max(physical(r).values()))
```

```
    check('continuous_boundary_actions', max(actions)<1e-6, max(actions))
```

```
    # Constant-price embedded 9-state chain must exactly agree on reachable states.
```

```
    joint_trans=np.zeros((3, 9, 9)); prior=np.zeros(9); prior[[1, 4, 7]]=1/3
```

```
    for i in range(3): joint_trans[:, 3*i+1, [1, 4, 7]]=trans[:, i, :]
```

```

# Unreachable source states receive an arbitrary valid row, but cannot be reached.
for i in [0, 2, 3, 5, 6, 8]: joint_trans[:, i, 1]=1
n9=np.repeat(n, 3, axis=1); wt9=np.repeat(wt, 3, axis=1)
jmdl=synthetic(n9, np.full_like(n9, .5), wt9, joint_trans, prior)
small=synthetic(n, np.full_like(n, .5), wt, trans)
diff=float(np.abs(m.bellman(g, jmdl, grid)[:,[1, 4, 7]]-m.bellman(g, small, grid)).max())
check('constant_price_exact_reduction', diff<1e-7, diff)
for badlen in [0, 3, 5]:
    try: m.bellman(np.zeros(badlen), random, grid)
    except AssertionError: pass
    else: raise AssertionError('horizon mismatch was not rejected')
check('horizon_guard', True, 'invalid plan lengths rejected')
jm=m.scenario(f, 31, 0, '43'); probs=float(np.max(np.abs(jm.transition.sum(2)-1)))
check('probabilities', probs<1e-12 and abs(jm.prior.sum()-1)<1e-12 and
np.allclose(jm.mass.sum(2), 1), dict(max
_row_error=probs, fallback=jm.fallback))
# Check actual representative provenance through retained history IDs.
pts=np.stack([f['zn43'][:18:31, 0], f['zp'][:18:31, 0]], axis=-1).reshape(-1, 2)
valid=jm.mass>0; ids=jm.representatives[valid]
n_back=(jm.demand*6-jm.center[:, None, None])/jm.scale[:, None, None]
rawp=np.maximum(0, jm.price_center[:, None, None]+jm.price_scale[:, None, None]*pts[np.maximum(jm.representativ
es
, 0), 1])
check('paired_representatives', np.allclose(n_back[valid], pts[ids, 0]) and
np.allclose(jm.price[valid], rawp[va
lid]), int(valid.sum()))
flat=np.array([[0., 1.], [0., 1.], [2., 3.]])
_, mass, _=m.compress(flat, 7)
check('compression_duplicate_mass', np.allclose(mass, [2/3, 1/3]), mass.tolist())
# Rebuild at data ingress: future prices cannot alter historical/zero-hour information.
changed=raw['price'].copy(); changed[31:]*=2
ph, sd, zp=m.price_features(changed)
check('price_causality', np.allclose(ph[:31], f['ph'][:31], equal_nan=True) and
np.allclose(ph[31, 0], f['ph'][31
, 0])
and np.allclose(sd[:32], f['sd'][:32], equal_nan=True) and
np.allclose(zp[:31], f['zp'][:31], equal_nan=Tr
ue), 'future-date ingress perturbation')
tiny=raw['price']*0+.004; ph0, sd0, zp0=m.price_features(tiny)
ff=dict(f, ph=ph0, sd=sd0, zp=zp0); const=m.scenario(ff, 31, 0, '43')
absent=[0, 2, 3, 5, 6, 8]
check('constant_price_support', np.max(np.abs(const.transition[:, :, absent]))<1e-12 and
np.allclose(const.pric
e[const.mass>0, .004], 'no artificial price states or positive-price floor')
zero=m.scenario(dict(f, ph=ph0*0, sd=sd0, zp=zp0*0), 31, 0, '43')
check('zero_price_allowed', np.max(zero.price)==0, 0)
warm=m.warmup(f)
check('warmup_physics', max(physical(warm).values())<1e-6 and
warm[0, 0, 6]==6000, dict(end=float(warm[-1, -1, 7])
, **physical(warm)))
energy=float(warm[-1, -1, 7]); cache={}
full=m.simulate(f, 31, energy, cache=cache)
check('real_day_physics', max(physical(full['rows']).values())<1e-6, physical(full['rows']))
fixed=m.simulate(f, 31, energy, version_mask=7, adjust_mask=0, cache=cache)
check('update_without_trading', not len(fixed['events']) and
np.array_equal(fixed['rows'][:, 0], fixed['rows'][:
, 1]) and fixed['rows'][-1, -1]==3, '4 issue models, same fixed plan')
frozen=m.simulate(f, 31, energy, src='42', version_mask=0, adjust_mask=0)
check('q42_forecast_family', np.array_equal(frozen['rows'][:, 0], frozen['rows'][:, 1]) and not
np.allclose(froz
en['rows'][:, 0], full['rows'][:, 0]), 'Q2 and Q3 forecasts remain separate')
# Perturb future price/net values and regenerate all affected price features/scenarios.
r2={k:v.copy() for k, v in raw.items()}; r2['price'][31, 36:]*=1.3; r2['net'][31, 36:]+=500
f2=m.features(r2); altered=m.simulate(f2, 31, energy)
e0=full['events'][full['events'][:, 0]==1, :6]; e1=altered['events'][altered['events'][:, 0]==1, :6]
check('transaction_before_observation', np.array_equal(full['snapshots'][1], altered['snapshots'][1]) and
np.a
rray_equal(e0, e1), 'recomputed features + price/net perturbation from slot36')
r3={k:v.copy() for k, v in raw.items()}; r3['q43'][31, 2:, :72:]+=1000

```

```

    later=m.simulate(m.features(r3),31,energy)
    check('unreleased_forecast_causality',np.allclose(full['rows'][:72],later['rows'][:72]),'12h/18h forecast
pe
    rturbation does not change earlier execution')
    # Physical state is carried to the next day, rather than being reset.
    nxt=m.simulate(f,32,float(full['rows'][-1,7]))
    check('cross_day_state',nxt['rows'][0,6]==full['rows'][-1,7],float(nxt['rows'][0,6]))
    check('protected_inputs',m.provenance()==provenance,'all recorded hashes unchanged')
    out=m.BASE/'tests'; out.mkdir(exist_ok=True)
    payload=dict(passed=len(reports),total=len(reports),checks=reports,elapsed=time.perf_counter()-started)
    (out/'unit_checks.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encoding='utf-8')
    print(f'All {len(reports)} tests passed; elapsed={payload["elapsed"]:.2f}s')

```

```

if __name__=='__main__': main()

```

代码/实验/q4_independent_codex_20260912/test_recovery.py

```

"""Integration checks for safe output handling and actual resumed day equivalence."""
import json
import subprocess
import sys
import numpy as np
import model as m

def main():
    result={}
    frozen={str(p):m.digest(p) for p in (m.BASE/'smoke').glob('*.*.json')}
    run=subprocess.run([sys.executable,'-X','utf8','run.py','--days','1','--output','smoke'],cwd=m.BASE,capture
e_
    output=True,text=True,encoding='utf-8')
    result['nonempty_rejected']=run.returncode!=0 and 'output not empty' in run.stderr
    result['rejection_preserves_files']=all(m.digest(m.Path(p))==h for p,h in frozen.items())
    # Deliberately incompatible signature in an isolated test fixture.
    fixture=m.BASE/'tests/bad_signature'; fixture.mkdir(parents=True,exist_ok=True)
    (fixture/'signature.json').write_text('{}',encoding='utf-8')
    run=subprocess.run([sys.executable,'-X','utf8','run.py','--resume','--output','tests/bad_signature'],cwd=m.
B
    ASE,capture_output=True,text=True,encoding='utf-8')
    result['incompatible_signature_rejected']=run.returncode!=0 and 'signature mismatch' in run.stderr
    same=True
    for d in [31,32]:
        with np.load(m.BASE/'smoke/days'/f'{d:03d}.npz') as
resumed,np.load(m.BASE/'results/days'/f'{d:03d}.npz'
        ) as straight:
            same &= set(resumed.files)==set(straight.files)
            for key in resumed.files:
                if key!='metadata': same &= np.array_equal(resumed[key],straight[key])
    result['actual_resume_matches_uninterrupted']=bool(same)
    payload=dict(passed=sum(result.values()),total=len(result),checks=result)
    (m.BASE/'tests/recovery_checks.json').write_text(json.dumps(payload,indent=2),encoding='utf-8')
    print(payload)
    assert all(result.values())

```

```

if __name__=='__main__': main()

```

代码/figures/论文数据图/问题三/figA_netload_forecast_update.py

```

"""Plot existing Q3 forecasts; no fitting, smoothing, or simulated observations.

```

```

Run: python -X utf8 figA_netload_forecast_update.py --kind raw

```

```

Optional: --kind calibrated --date 2025-03-20
"""

```

```

from pathlib import Path
import argparse
import hashlib
import json
import platform

```

```

import matplotlib
matplotlib.use('Agg')

```

```

import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.text import Text
import numpy as np
import pandas as pd

COLORS = dict(actual='#104E8B', forecast_00='#7DAEEB', forecast_06='#5BC0C9',
               forecast_12='#E5B5B5', forecast_18='#D85C5C')
STEM = 'figA_netload_forecast_update'

def sha(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--kind', choices=['raw', 'calibrated'], default='raw')
    parser.add_argument('--date', default='2025-03-20',
                        help='Default: first of the four dates specified in the problem; not selected by
accurac
y.')
    args = parser.parse_args()
    out = Path(__file__).resolve().parent
    root = next(p for p in out.parents if (p/'基础数据').is_dir())
    source = root/'代码/实验/q3_revision_a_20260912/results'
    cache_path = source/'conditional_forecasts.npz'
    actual_path = source/'main_intervals.csv'
    manifest_path = source/'manifest.json'
    manifest = json.loads(manifest_path.read_text(encoding='utf-8'))
    inputs = {str(p): sha(p) for p in [cache_path, actual_path, manifest_path]}
    for p in [cache_path, actual_path]:
        assert inputs[str(p)] == manifest['output_hashes'][p.name], f'Source hash mismatch: {p}'
    data = pd.read_csv(actual_path)
    data['date'] = pd.to_datetime(data['date'])
    data = data.sort_values(['date', 'slot']).reset_index(drop=True)
    dates = pd.date_range('2025-02-01', '2025-12-31')
    assert len(data) == 334*144
    assert np.array_equal(data['date'].to_numpy().reshape(334, 144)[: , 0], dates.to_numpy())
    assert np.array_equal(data['slot'].to_numpy().reshape(334, 144), np.tile(np.arange(1, 145), (334, 1)))
    actual = data['net_kwh'].to_numpy().reshape(334, 144)*6
    with np.load(cache_path) as f:
        load = f['load'].copy()
        pv = f['pv'].copy()
        raw = load[:, None, :]-pv
        forecast = raw if args.kind == 'raw' else f['pred'][:, :, 1, :].copy()
    assert forecast.shape == (365, 4, 144)
    forecast = forecast[31:]
    for k in range(4):
        assert np.isfinite(forecast[:, k, 36*k:]).all()
        assert np.isnan(forecast[:, k, :36*k]).all()
    date = pd.Timestamp(args.date)
    assert date in dates
    d = (date-dates[0]).days
    daily, paired = [], []
    for k in range(4):
        sl = slice(k*36, 144)
        error = forecast[d, k, sl]-actual[d, sl]
        all_error = forecast[:, k, sl]-actual[:, sl]
        daily.append(dict(issue=f'{6*k}:00', window=f'{6*k}:00 - 24:00', n=int(error.size),
                        mae_kw=float(np.abs(error).mean()), rmse_kw=float(np.sqrt(np.mean(error**2))),
                        pooled_mae_kw=float(np.abs(all_error).mean())))
    if k:
        old = float(np.abs(forecast[:, k-1, sl]-actual[:, sl]).mean())
        new = float(np.abs(all_error).mean())
        paired.append(dict(issue=f'{6*k}:00', target_window=f'{6*k}:00 - 24:00',
                        previous_version_mae_kw=old, updated_mae_kw=new,
                        reduction_percent=100*(old-new)/old,

```

```

representative_day_previous_mae_kw=float(np.abs(forecast[d,k-1,sl]-actual[d,sl])

mean()),

representative_day_updated_mae_kw=daily[-1]['mae_kw'])

available = {f.name for f in font_manager.fontManager.ttflist}
chinese = next((n for n in ['Microsoft YaHei','SimHei','Noto Sans CJK SC'] if n in available), None)
assert chinese, 'A Chinese font is required.'
plt.rcParams.update({'font.family':'sans-serif', 'font.sans-serif':[chinese,'Arial'],
                    'axes.unicode_minus':False, 'font.size':10,
                    'axes.labelsize':11, 'xtick.labelsize':10, 'ytick.labelsize':10,
                    'svg.fonttype':'path', 'pdf.fonttype':42, 'axes.linewidth':0.75})
fig = plt.figure(figsize=(8.4,4.4), facecolor='white')
ax = fig.add_axes([0.10,0.16,0.61,0.79])
x = (np.arange(144)+0.5)/6 # interval midpoints; never shift completed values into future slots
styles = [(0,(6,3)), (0,(6,2,1.5,2)), (0,(3,5,2.2)), (0,(2,1.5))]
labels = ['0:00 预测','6:00 更新预测','12:00 更新预测','18:00 更新预测']
forecast_lines = []
for k, key in enumerate(['forecast_00','forecast_06','forecast_12','forecast_18']):
    line, = ax.plot(x[36*k:], forecast[d,k,36*k:], color=COLORS[key],
                    lw=1.65 if k!=3 else 1.85, ls=styles[k], label=labels[k], zorder=3+k)
    forecast_lines.append(line)
actual_line, = ax.plot(x,actual[d],color=COLORS['actual'],lw=2.4,label='实际净负荷',zorder=8)
for h in [6,12,18]:
    ax.axvline(h,color='#AEB8C2',lw=0.85,ls=(0,(3,4)),zorder=1)
ax.set_xlim(0,24)
ax.set_xticks([0,6,12,18,24], ['0:00','6:00','12:00','18:00','24:00'])
ax.set_xlabel('时间')
ax.set_ylabel('净负荷 / kW')
ax.grid(axis='y',color='#E9EDF1',lw=0.65,zorder=0)
ax.spines[['top','right']].set_visible(False)
for edge in ['left','bottom']:
    ax.spines[edge].set_color('#6E7883')
ax.tick_params(length=3,color='#6E7883')
ax.margins(y=0.10)
fig.text(0.755,0.934,args.date,fontsize=10.5,color='#334155')
fig.legend(handles=[actual_line]+forecast_lines,loc='upper left',
            bbox_to_anchor=(0.745,0.899),frameon=False,handlelength=3.2,
            handletextpad=0.8,labelspp=0.75,fontsize=9.5)
table = ax.table(cellText=[[v['issue'],f"{v['mae_kw']:.2f}"] for v in daily],
                 colLabels=['预报时刻','MAE / kW'],cellLoc='center',colLoc='center',
                 bbox=[1.07,0.22,0.37,0.34],colWidths=[0.44,0.56])
table.auto_set_font_size(False)
table.set_fontsize(9)
for (r,c),cell in table.get_celld().items():
    cell.set_edgecolor('#E1E7ED')
    cell.set_linewidth(0.55)
    cell.set_facecolor('#F1F5F9' if r==0 else 'white')
    cell.get_text().set_color('#334155')
footnote = fig.text(0.755,0.29,'MAE: 代表日发布后至24:00\n各行评价时段长度不同',fontsize=8.2,
                    color='#64748B',linespacing=1.65,va='top')
kind_label = '负荷预测 光伏预测' if args.kind=='raw' else '历史误差校准后的中位数'
fig.text(0.755,0.177,kind_label,fontsize=8.2,color='#64748B')
for text in fig.findobj(match=Text):
    text.set_fontfamily(chinese if any(ord(c)>127 for c in text.get_text()) else 'Arial')
fig.canvas.draw()
renderer = fig.canvas.get_renderer()
assert not table.get_window_extent(renderer).overlaps(footnote.get_window_extent(renderer))
for ext in ['png','svg','pdf']:
    fig.savefig(out/f'{STEM}_{ext}',dpi=300,facecolor='white',metadata={'Creator':'Matplotlib'} if
ext!='png'
    else None)
plt.close(fig)
records = pd.DataFrame({'interval_start':np.arange(144)/6, 'interval_end':(np.arange(144)+1)/6,
                        'plot_hour_midpoint':x,'actual_netload_kw':actual[d]})
for k in range(4):
    records[f'forecast_{6*k:02d}_kw'] = forecast[d,k]
records.to_csv(out/f'{STEM}_data.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(daily).to_csv(out/f'{STEM}_mae.csv',index=False,encoding='utf-8-sig')

```

```

    meta = dict(date=args.date, kind=args.kind, selection='First date specified in problem; fixed, not chosen
by f
    forecast performance.',
        actual_field='main_intervals.csv: net_kwh * 6 = actual load - actual PV, kW',
        forecast_field='conditional_forecasts.npz: load[d, :] - pv[d, k, :]' if args.kind=='raw'
        else 'conditional_forecasts.npz: pred[d, k, 1, :]' (q=0.50; includes historical residual
correcti
on)',
        raw_forecast_field='load[d, :] - pv[d, k, :
]', date_index=(date-pd.Timestamp('2025-01-01')).days,
        issue_index={'0:00':0, '6:00':1, '12:00':2, '18:00':3}, plot_time='10-minute interval midpoint',
        dpi=300, colors=COLORS, mae=daily, same_target_window_update_checks=paired,
        caveats=['Later forecasts begin only after release; no backfilling or curve offsets.',
            'MAE across issue times covers different remaining horizons; not a pure
information-val
ue comparison.',
            'Identical late-day forecasts remain overlapping; not artificially separated.',
            'Forecast accuracy alone does not prove every purchase-plan revision is
economical.'],
        input_sha256=inputs, python=platform.python_version(), matplotlib=matplotlib.__version__,
        source_sha256=sha(Path(__file__)))
    for path, digest in inputs.items():
        assert sha(Path(path)) == digest, f'Input changed: {path}'
    (out/f' {STEM}_metadata.json').write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding='utf-8')
    print(json.dumps({'date':args.date, 'kind':args.kind, 'metrics':daily, 'paired_checks':paired,
        'output':str(out), 'input_hashes_verified':True}, ensure_ascii=False, indent=2))

if __name__=='__main__':
    main()

```

附录D 支撑文件逐项列表

1. AI使用说明.docx
2. AI使用说明.txt
3. AI工具使用详情.pdf
4. README_支撑材料说明.txt
5. 准备复现.py
6. 参考文献来源.txt
7. 图表/问题一_调度.png
8. 图表/问题三_累计费用.png
9. 图表/问题三_组合费用.png
10. 图表/问题二_累计费用.png
11. 图表/问题四_指定日库存.png
12. 图表/问题四_累计费用.png
13. 图表/问题四_组合费用.png
14. 文件校验清单.json
15. 核验/Excel精简核验.json
16. 核验/文件来源与哈希.json
17. 核验/问题一_历史运行记录.json
18. 核验/问题三_历史运行记录.json
19. 核验/问题二_历史运行记录.json
20. 核验/问题四_历史运行记录.json
21. 核验支撑包.py
22. 源码/代码/figures/论文数据图/问题三/figA_netload_forecast_update.py
23. 源码/代码/q1_solution.py
24. 源码/代码/实验/q2_fresh_20260912/fresh_engine.py
25. 源码/代码/实验/q2_fresh_20260912/run_fresh.py
26. 源码/代码/实验/q2_fresh_20260912/verify_delivery.py
27. 源码/代码/实验/q2_fresh_20260912/write_report.py
28. 源码/代码/实验/q3_revision_a_20260912/check_engine.py
29. 源码/代码/实验/q3_revision_a_20260912/deliver_q3.py
30. 源码/代码/实验/q3_revision_a_20260912/q3_engine.py

31. 源码/代码/实验/q3_revision_a_20260912/run_q3.py
32. 源码/代码/实验/q4_independent_codex_20260912/audit.py
33. 源码/代码/实验/q4_independent_codex_20260912/deliver.py
34. 源码/代码/实验/q4_independent_codex_20260912/inspect_inputs.py
35. 源码/代码/实验/q4_independent_codex_20260912/model.py
36. 源码/代码/实验/q4_independent_codex_20260912/run.py
37. 源码/代码/实验/q4_independent_codex_20260912/test_model.py
38. 源码/代码/实验/q4_independent_codex_20260912/test_recovery.py
39. 环境/requirements_q2.txt
40. 环境/requirements_q2_extra.txt
41. 环境/requirements_q34.txt
42. 结果/result1.xlsx
43. 结果/result2.xlsx
44. 结果/result3.xlsx
45. 结果/result4-2.xlsx
46. 结果/result4-3.xlsx
47. 结果/问题一/summary.json
48. 结果/问题一/完整调度.csv
49. 结果/问题三/daily_summary.csv.xz
50. 结果/问题三/delivery_checks.json
51. 结果/问题三/engine_checks.json
52. 结果/问题三/forecast_metrics.json
53. 结果/问题三/main_intervals.csv.xz
54. 结果/问题三/revision_ledger.csv.xz
55. 结果/问题三/sensitivity.csv
56. 结果/问题三/summary.json
57. 结果/问题三/指定日表1.csv
58. 结果/问题三/指定日表2.csv
59. 结果/问题三/指定日表3.csv
60. 结果/问题二/controller_selection.json
61. 结果/问题二/daily_summary.csv.xz
62. 结果/问题二/delivery_check.json

- 63. 结果/问题二/forecast_selection.json
- 64. 结果/问题二/independent_check.json
- 65. 结果/问题二/main_intervals.csv.xz
- 66. 结果/问题二/summary.json
- 67. 结果/问题二/training_trace.csv
- 68. 结果/问题四/A42_intervals.csv.xz
- 69. 结果/问题四/A43_111_intervals.csv.xz
- 70. 结果/问题四/A43_111_ledger.csv.xz
- 71. 结果/问题四/annual.json
- 72. 结果/问题四/audit.json
- 73. 结果/问题四/daily_costs.csv.xz
- 74. 结果/问题四/information_cutoffs.csv
- 75. 结果/问题四/price_metrics.csv
- 76. 结果/问题四/scenario_diagnostics.csv
- 77. 结果/问题四/sensitivity.csv
- 78. 结果/问题四/specified_table1.csv
- 79. 结果/问题四/specified_table2.csv
- 80. 结果/问题四/specified_table3.csv
- 81. 结果/问题四/static_repricing.csv